大模型应用开发八股

基础

1.Workflow 和 Agent 的差异

2.什么是reflection?

3.什么是 CoT（Chain of Thought）

4.reflection和COT的区别?

5.Agent的三种模式

6.ReAct的具体工作流程

7. 什么是 向量数据库 IVF HNSW 以及各自具体的工作流程

8.什么是重排序 为什么需要重排序 以及重排序的流程

9、什么是查询重写？以及查询重写有哪些方式?

10. Self-Query Retriever （ 自查询检索器 ）

11.RAG是如何进行更新和删除的?

12.什么是GraphRAG 以及为什么需要GraphRAG

13.GraphRAG的索引建立和查询的具体细节是什么

一、索引建立：先把文本变成“图 + 分层摘要 + 向量索引”

1）切分文档，得到 TextUnit

2）从每个 TextUnit 抽取实体、关系、可选的 claim

3）合并跨 chunk 的同名实体与关系，并做摘要归一

4）做社区发现，形成层次化社区结构

5）为每个社区生成 Community Report

6）再做向量化，写入向量存储

二、索引产物到底有什么

三、查询：不是一种查法，而是多种 Query Mode

四、Local Search 的具体流程

第一步：把用户问题映射到相关实体

第二步：以这些实体为入口向外扩展候选上下文

第三步：对候选信息做 ranking 和 filtering

第四步：把结构化 + 非结构化上下文送入 LLM 作答

五、Global Search 的具体流程
第一步：选定某一层级的 Community Reports
第二步：Map 阶段
第三步：Reduce 阶段
六、为什么 GraphRAG 能比普通 RAG 更适合复杂问题
七、你在面试里可以怎么总结
14 RAG系统评估
15 什么是MCP?
16 什么是Skills?
17.什么是openclaw?
18 Agent的记忆模式有哪些 以及具体的工作流程
19 分别说说 Agent 的 短期记忆、长期记忆和工作记忆 的流程
20.什么是多智能体协同 以及为什么需要多智能体协同?
21.Memory的多用户并发怎么实现?
22.RAG幻觉的解决办法
23 什么是A2A协议?
24 LoRA
RAG部分：
1.RAG系统中如何处理多跳回答?
2.什么是self-RAG 如何让模型自主判断是否需要检索?
3 RAG系统中如何支持多模态检索 图文检索如何实现?
4.增量更新场景下，RAG的向量库如何维护?如何处理文档删除和修改?
5.知识图谱怎么和大模型结合?检索增强生成(RAG)怎么用图谱?
6.什么是知识图谱?实体、关系、属性分别是什么?
7.DST怎么处理多轮对话?历史轮次的信息怎么利用?
8.知识图谱补全(Knowledge Graph Completion)怎么做?怎么预测缺失的关系?
9.RAG中的幻觉问题如何缓解?引用溯源(Citation)如何实现?
10.什么是GraphRAG?知识图谱如何增强RAG系统?
11.RAG系统中文档切分的策略有哪些?如何选择合适的chunk size?
12 上下文压缩(Context Compression)技术有哪些?如何减少token消耗?
13 RAG系统的评测指标有哪些?如何评估检索质量和生成质量?
14 如何选择合适的Embedding模型?开源模型vs闭源API的权衡?

15 RAG中的重排序(Reranking)如何工作?有哪些重排序模型?

16 如何构建RAG系统的GroundTruth数据集?标注方法有哪些?

17 什么是Hypothetical Document Embeddings (HyDE)?如何提升检索效果?

18 什么是Semantic Chunking?与固定长度切分有什么区别?

19 关系抽取(Relation Extraction)是什么?怎么识别实体之间的关系?

20 什么是混合检索(Hybrid Search)?稀疏检索和稠密检索如何结合?

Agnet部分：

1.什么是AIAgent?与传统软件程序有什么本质区别?

2.如何设计代理(Agent)的反思机制?何时触发反思?

3.Agent对话模块的容错能力怎么设计?用户说话不清楚或有歧义时怎么办?

4.什么是MCP(Model Context Protocol)协议?它解决了什么问题?

5.AIAgent的幻觉问题如何解决?有哪些验证策略?

6.MCP(Model Context Protocol)协议的基本架构是怎样的?Client和Server如何交互?

7.LangChain是什么?它解决了LLM应用开发的哪些问题?

8.LangChain的核心组件有哪些?各自的作用是什么?

9.LangChain的Agent执行器是如何工作的?

10.LangGraph是什么?它与LangChain有什么关系?

11 什么是对话历史窗口?保留多少轮对话合适?

12 如何使用LangChain构建多轮对话系统?

13.AIAgent的记忆机制如何设计?短期记忆和长期记忆的区别?

14 如何评估AIAgent的性能?有哪些关键指标?

15 Agent的Self-Refinement是如何实现的?需要哪些关键组件?

16 LangGraph的人机交互功能如何实现?

17 AIAgent在执行过程中可能遇到哪些错误?如何处理?

18 AIAgent开发中常见的调试问题有哪些?如何解决?

19 如何用LangChain实现一个简单的问答机器人Agent?

20 什么是ReAct模式?它如何提升AIAgent的推理能力?

21 如何调试LangGraph工作流?有哪些调试工具?

22 Agent的长期记忆如何持久化?向量数据库的选择标准是什么?

23 Agent的成本如何计算?如何在效果和成本间找到最优平衡点?

24 MCP(Model Context Protocol)协议支持哪些传输方式?各有什么特点?

25 MCP(Model Context Protocol)协议中的Resource和Tool有什么区别?各自的使用场景?

26 什么是多AI Agent系统(Multi-Agent System)?AI Agent之间如何协作?

27 如何在LangGraph中定义状态?状态管理的最佳实践?

28 Graph of Thoughts (GoT)如何表示复杂推理过程?有什么优势?

29 如何在LangGraph中实现循环和迭代?

30 LangGraph的编译和执行过程是怎样的?

31 LangChain的缓存机制如何工作?如何优化性能?

32 LangGraph的条件边(Conditional Edge) 如何使用?

33 LangChain的Memory组件如何工作?有哪些Memory类型?

34 MCP(Model Context Protocol)协议的安全机制包括哪些?如何保证通信安全?

35 如何在LangChain中实现流式输出?有什么应用场景?

36 如何在LangChain中实现RAG(检索增强生成)?

37 如何给AIAgent添加网络搜索功能?需要注意什么?

38 什么是LCEL (LangChain Expression Language)?它有什么优势?

39 Agent的可解释性与可控性如何权衡?如何在自主性和安全性间平衡?

40 如何在LangChain中集成外部工具?Tool的定义规范是什么?

41 LangChain的输出解析器有什么作用?如何处理结构化输出?

42 Multi-Agent系统中的角色分工如何设计?有哪些典型角色模式?

43 LangGraph的持久化机制是什么?如何保存执行状态?

44 如何使用MCP(Model Context Protocol)协议开发一个简单的工具服务?

45 如何在LangGraph中处理并行执行?

46 LangChain的文档处理功能包括哪些?如何处理大文档?

47 什么是代码解释器(Code Interpreter)?它在AI Agent中的作用是什么?

48 如何设计Multi-Agent的任务分配算法?负载均衡如何实现?

49 什么是Agent的工具学习(Tool Learning)?如何让Agent学会使用新工具?

牛客/小红书面经：

1.请阐述RAG的核心原理，并说明如何通过 RAG 缓解大模型的幻觉问题。

2.如何设计一个高效的 agent 上下文维护方案?

3 一个完整的 agent 智能体架构一般包括哪些部分?

4 向量检索和关键词检索的区别是什么?

5.讲解召回重排的策略

6.如果让你设计OpenClaw你会考虑哪些因素，以及你觉得他的发展限制因素有哪些

7.讲一下agent中有哪些模式

8.你编写了哪些MCP工具，介绍一下

9.聊一下muliagent，你是怎么做意图识别的

10 你是如何处理这种异常情况的，比如路由到了一个错误的任务，如果用户说不对，我要问的问题是XXX...

11 任务分解后，会根据任务执行结果调整任务列表吗

12 大文本说明书常用的切分方式有哪些

13 你说按段落/按语义切，那段落有的 100 字、有的 1 万字怎么办?

14 Agent skil怎么理解，跟tools是区别是什么

15 上下文工程了解多少，长短期记忆如何做

16 记忆覆盖问题是如何解决的?

17 prompt设计的规则

18 你认为要把一件事情做好，Agent应该怎么拆分。Agent拆分的环节数量，对于最终的交付质量有什么关系

19 对于Agent记忆的理解，当前市面上用的多的方案有哪些

20 你如何设计上下文窗口

21 如何确定写入长期记忆中是重要的

22 短期记忆摘要进行压缩，存在损失，如果减少了关键信息，你如何解决

23 如何压缩上下文

24 RAG系统完整链路如何设计(切分、召回、重排、生成、评估)?

25 向量召回、关键词召回、实体召回如何融合与定权?

26 Agent系统如何设计可观测性(日志、trace、回放)?

27 RAG有哪些分类

28 传统RAG有什么痛点;介绍GraphRAG，GraphRAG的难点是什么;GraphRAG如何应对增量场景

29 RAG优势与Split策略:讨论针对不同文档(如代码、手册)的切分逻辑差异。

30 大文档检索:针对长文档，如何避免“中间信息丢失”问题及长上下文的处理技巧。

31 面对一万个PDF文档，考察从解析、清洗、向量化入库到检索增强的全链路设计。

32 多阶段召回策略优化

33 介绍一下Function Call的流程，模型是如何知道该调用哪个工具的?

34：向量数据库索引中，IVF_FLAT和HNSW有什么区别?各自的适用场景?

35 LangGraph相比于LangChain有什么优势?LangGraph的状态快照机制了解吗，大概是怎么实现的?你的...

36 工具多导致token数过多的解决办法

37 Mcp server的构建方式

基础

1.Workflow 和 Agent 的差异

Workflow 和 Agent 的核心区别在于，**Workflow 是预定义流程驱动，Agent 是目标驱动的自主决策系统**。

Workflow 的步骤通常是提前设计好的，比如意图识别、知识检索、答案生成、结果校验，整体执行路径比较固定，所以它的优点是可控性强、稳定性高、容易落地，适合客服、审批、固定报表这类标准化任务。

而 Agent 更强调自主性。开发者通常只给它一个目标，它会根据当前上下文和外部反馈，自主决定下一步做什么，比如要不要拆解任务、要不要调用搜索或数据库、要不要调整策略重新执行。所以 Agent 更适合复杂、开放、非标准化任务，比如自动化研究、复杂问答、多工具协同。

一句话总结：**Workflow 是人先把路规划好，系统按流程执行；Agent 是人给目标，系统自己边做边决定路径。实际项目里，两者往往会结合使用，也就是用 Workflow 保证整体稳定性，在局部需要灵活决策的环节引入 Agent。

2.什么是reflection?

Reflection 可以理解为大模型或 Agent 的**自我反思、自我检查机制**。它的核心思想不是模型一次性把答案直接给出来，而是先生成一个**初步结果**，再回过头**检查**这个结果有没有错误、遗漏或者偏离目标的地方，然后再进行**修正和优化**。

它和普通推理的区别在于，普通推理更强调“怎么一步一步得到答案”，而 Reflection 更强调“答案出来之后，能不能自我复盘和纠错”。

在实际系统里，Reflection 通常用在复杂任务中，比如代码生成、论文写作、任务规划、多工具调用。比如模型先写一版代码，运行后发现报错，再根据报错分析原因、修改代码，这其实就是一种 Reflection。

所以我一般会把它总结成一句话：Reflection 就是让模型从“直接回答”升级为“先做、再检查、再改进”的闭环机制。

它的价值在于能提升复杂任务的正确率、稳定性和完成质量，尤其在 Agent 系统里很重要，因为 Agent 不只是要会做事，还要能根据结果不断修正自己的策略。

3.什么是 CoT（Chain of Thought）

CoT，也就是 Chain of Thought，中文通常叫**思维链**。它的核心思想是，不让大模型直接给最终答案，而是引导它把中间推理过程一步一步展开，这样可以提升复杂任务的推理能力。

比如在数学题、逻辑题或者多步分析任务里，如果模型直接回答，可能会跳步骤，容易出错；而用了 CoT 之后，模型会先分析条件，再分步骤推导，最后得出结果。这样做的好处是，模型在复杂问题上的正确率通常会更高。

从本质上讲，CoT 是把“隐藏在模型内部的推理过程”尽量显式化，让模型按步骤思考，而不是只给结论。

在实际应用里，CoT 常用于数学推理、代码分析、复杂问答和任务拆解。比如在 Agent 系统里，模型可以先通过 CoT 做任务分解，再决定调用哪些工具。

所以我一般会把它总结成一句话：**CoT 就是让大模型把答案背后的推理路径显式展开，从而提升复杂任务表现的一种提示和推理机制。**

4.reflection和COT的区别?

Reflection 和 CoT 的核心区别在于，**CoT 强调“生成答案时一步一步推理”，而 Reflection 强调“答案生成后再回头检查和修正”。**

CoT，也就是思维链，重点是把中间推理过程展开，让模型按步骤分析问题、逐步得到答案。它解决的是“模型怎么想”的问题，所以常用于数学推理、逻辑分析、任务拆解这类场景。

而 Reflection 更像“复盘”机制。模型先给出一个初步答案，然后再站在审查者的角度去检查这个答案有没有错误、遗漏或者不符合要求的地方，再进行修改。它解决的是“模型答完以后能不能自我纠错”的问题。

所以可以把两者理解成两个不同阶段：**CoT 发生在生成过程中，帮助模型更好地推理；Reflection 发生在生成之后，帮助模型做自我检查和优化。**

如果用一句话总结，我会说：**CoT 是让模型把思考过程展开，Reflection 是让模型对已有结果做复查和修正。** 在实际系统里，这两者经常结合使用，比如先用 CoT 生成初稿，再用 Reflection 做纠错和改进。

5.Agent的三种模式

智能体里常见的三种架构模式是 **ReAct、Plan-and-Solve 和 Reflection**，它们分别对应三种不同的任务处理思路。

ReAct 的核心是“边想边做”。模型一边推理，一边调用工具，再根据工具返回结果继续下一步，所以它适合信息不完整、需要实时交互和动态决策的任务，比如搜索、查询数据库、多轮工具调用。它的优点是灵活，但缺点是容易陷入局部最优，整体规划能力相对弱。

Plan-and-Solve 的核心是“先规划，再执行”。模型先把大任务拆成若干子步骤，形成一个整体计划，然后按计划逐步完成。它比 ReAct 更适合复杂任务，因为先有全局路线，执行时不容易乱，常用于复杂问答、任务编排、多步骤分析。

Reflection 的核心是“先做，再检查，再改”。模型先生成初步结果，然后回头审查自己的答案或执行过程，发现问题后再修正。所以它更强调自我纠错和迭代优化，特别适合代码生成、复杂推理、长链路任务。

如果我用一句话总结三者区别，就是：**ReAct 是边推理边行动，Plan-and-Solve 是先规划后行动，Reflection 是行动后再复盘优化。** 实际系统里，这三种模式经常结合使用，比如先用 Plan-and-Solve 做全局规划，中间用 ReAct 调工具执行，最后再用 Reflection 做结果校验和修正。

6.ReAct的具体工作流程

ReAct 的全称是 **Reason + Act**，它的核心思想是让智能体把**推理**和**行动**结合起来，也就是一边思考，一边调用工具，再根据工具返回结果继续决策。

它的具体工作流程一般可以分成四步。第一步是**理解任务**，模型先根据用户目标判断当前需要做什么。第二步是**生成 Thought**，也就是先进行一小步推理，分析下一步最合适的动作。第三步是**执行 Action**，比如调用搜索、数据库、计算器或者代码工具。第四步是接收 **Observation**，也就是工具返回的结果。然后模型会基于这个 Observation 再进入下一轮 Thought，决定下一步怎么做。这个过程会不断循环，直到任务完成。

所以它本质上是一个 **Thought → Action → Observation → 再次 Thought** 的闭环。

ReAct 的优势在于，它不是只靠模型内部知识硬答，而是能通过外部工具不断获取新信息并调整策略，因此特别适合开放环境下的复杂任务。

我一般会把它总结成一句话：**ReAct 就是让智能体在“思考—行动—观察—再思考”的循环中逐步完成任务。**

7. 什么是 向量数据库 IVF HNSW 以及各自具体的工作流程

向量数据库本质上是**专门用来存储、索引和检索向量数据的数据库**。在大模型应用里，文本、图片这些非结构化数据通常会先通过 embedding 模型转换成高维向量，再存进向量数据库。查询时，也是先把用户问题转成向量，然后去找语义上最相近的向量，因此它的核心能力就是**相似度检索**。

在向量检索里，IVF 和 HNSW 是两种非常常见的近似最近邻索引方法。

IVF 的全称可以理解为倒排文件索引，它的思路是**先聚类，再局部搜索**。具体流程是：先对全量向量做聚类，形成多个簇；每个向量只归到一个簇里；查询时先判断查询向量最接近哪些簇，再只在这些簇内部做相似度计算，而不是扫全库。它的优点是速度快、适合大规模数据，缺点是如果相关向量分散在没被选中的簇里，可能会损失召回率。

HNSW 是分层小世界图索引，它的思路是**把向量连成图，再沿图快速逼近最近邻**。具体流程是：建索引时把每个向量和附近的若干向量建立连接，并构建多层图结构；查询时从高层稀疏图开始快速定位大致区域，再逐层向下细化，最后在底层找到最相近的向量。它的优点是召回率高、查询效果通常很好，缺点是建图和内存开销比较大。

所以我会总结成一句话：**向量数据库负责存和找相似向量，IVF 是“先分桶再查”，HNSW 是“沿图逐步逼近”。**

8.什么是重排序 为什么需要重排序 以及重排序的流程

重排序，也就是 Rerank，指的是在检索系统里，先用一个速度快的召回模块找出一批候选结果，再用一个更精细的模型对这些候选结果重新打分和排序，把真正最相关的内容排到前面。它本质上不是“找更多”，而是**从已经召回的结果里挑更准的**。

之所以需要重排序，是因为第一阶段召回通常更关注速度和覆盖率。比如向量检索、BM25 检索，虽然能快速找出一批可能相关的文档，但排在前面的内容不一定最符合用户真实问题，可能会有语义相近但不够关键的内容混进来。这时候如果直接把结果喂给大模型，回答质量就会受影响。重排序的作用就是进一步提升相关性和准确率。

它的流程一般分三步。第一步是**召回**，先通过向量检索、关键词检索或者混合检索拿到 TopK 候选文档。第二步是**重打分**，把用户 query 和每个候选文档一起输入 reranker 模型，让模型判断这一对 query-document 的匹配程度。第三步是**重新排序并截断**，按新分数重新排序，取最相关的前几个结果送给大模型生成答案。

所以我一般会总结成一句话：**重排序就是在“先粗召回”之后，再做一次更精细的相关性判断，用准确率换取更高质量的最终检索结果。**

9、什么是查询重写？以及查询重写有哪些方式？

如果面试时用 1 分钟回答，我会这样说：

查询重写，就是在用户原始问题进入检索或推理模块之前，先把这个问题改写成**更适合系统处理的表达**。它的核心目标是提升检索效果和理解效果，因为用户的原始提问往往比较口语化、模糊，甚至带代词、省略和上下文依赖，直接拿去检索，召回结果可能不准。

比如用户说“它的优缺点呢”，人能结合上下文知道“它”指什么，但检索系统不一定能理解。这时候就需要把它重写成更完整、更明确的问题，比如“Milvus 的优缺点是什么”。

常见的查询重写方式主要有几类。第一类是**指代消解和上下文补全**，把“它、这个、上面说的”还原成完整实体。第二类是**语义改写**，把口语化表达改成更标准、更适合检索的表述。第三类是**查询扩展**，给问题补充同义词、相关术语或领域关键词，提升召回率。第四类是**多路重写**，也就是把一个问题改写成多个不同版本，分别去检索，再合并结果。第五类是**结构化重写**，比如把自然语言改写成 SQL、Cypher 或搜索过滤条件，方便访问数据库或知识图谱。

所以我一般会总结成一句话：**查询重写就是先把用户问题变成系统更容易检索、理解和执行的形式，本质上是在检索前做一次问题优化。**

HyDE 和 **后退提示（Step-back Prompting）** 都是为了提升检索或推理效果的技巧，但它们解决的问题不一样。

先说 **HyDE**，全称是 Hypothetical Document Embeddings。它的核心思想是：用户问题本身往往很短，不一定适合直接做向量检索，所以先让大模型根据问题“假想”出一段可能的答案或文档，再把这段假想文档做 embedding，用它去检索真实文档。因为这段假想文本通常比原始 query 信息更丰富、语义更完整，所以往往能提升召回效果。它特别适合用户问题很短、知识型问答或者语义检索场景。流程就是：用户问题 → LLM 生成假想文档 → 对假想文档做向量化 → 去向量库检索 → 返回真实文档。

再说 **后退提示**，也就是 Step-back Prompting。它的核心不是直接回答细节问题，而是先“退一步”，从更高层、更抽象的角度提炼出这个问题背后的通用概念或原理，再基于这些高层信息去推理或检索。比如用户问一个很具体的问题，模型先抽象成“这个问题属于哪一类原理”，然后再回答细节。这样做的好处是能减少模型陷在局部细节里，提高复杂推理和知识迁移能力。

所以我一般会总结成一句话：HyDE 是先生成一篇假想文档来增强检索，后退提示是先抽象出高层问题再回到具体问题，本质上一个偏检索增强，一个偏推理增强。

10. Self-Query Retriever （自查询检索器）

标准叫法通常是 **自查询检索器**，英文是 **Self-Query Retriever**。它的核心思想是：**不只是拿用户原始问题去做语义检索，而是先让大模型把问题解析成“语义查询 + 结构化过滤条件”，再去底层向量库或检索系统里查。** 也就是说，它不仅看“内容像不像”，还会自动理解用户问题里隐含的元数据条件，比如时间、作者、类别、评分、价格区间等。(LangChain 参考文档)

它为什么有用？因为很多用户问题本身其实同时包含两类信息：一类是语义需求，另一类是过滤约束。比如“找 2024 年之后、关于 GraphRAG 的高质量论文”，这里“关于 GraphRAG”是语义部分，而“2024 年之后”就是结构化过滤条件。普通向量检索更擅长处理前者，但不擅长稳定处理后者；自查询检索器就是把这两部分拆开，再组合起来检索。(LangChain中文网)

它的工作流程一般分成四步。第一步，**理解用户问题**；第二步，**由 LLM 生成结构化查询**，也就是把问题拆成检索文本和 metadata filter；第三步，**把这个结构化查询发给底层 VectorStore 或搜索引擎**，同时做语义相似度匹配和元数据过滤；第四步，**返回更精准的候选文档** 给后续 RAG 或问答模块使用。LangChain 官方文档对它的定义也是：利用 LLM 生成适用于向量存储的查询，并执行元数据过滤。(LangChain 参考文档)

所以我一般会总结成一句话：**自查询检索器就是让大模型先把自然语言问题翻译成“可检索的语义条件 + 可执行的过滤条件”，从而比普通向量检索更精准。**

11.RAG是如何进行更新和删除的?

RAG 的更新和删除，本质上是在维护它背后的**外部知识库**，而不是去改大模型参数。也就是说，RAG 的知识通常存放在文档库、向量数据库或者混合检索系统里，所以当知识发生变化时，我们更新的是检索层，而不是重新训练模型。

更新 的流程一般是这样的：先找到发生变化的原始文档，然后重新做文档切分、向量化和索引构建，最后把新的 chunk 和 embedding 替换掉旧的数据。如果是增量更新，就只处理新增或被修改的文档，而不是全库重建。这样成本更低，也更适合真实业务。

删除 的核心是把对应文档或 chunk 从检索系统里移除。比如在向量库里删除对应向量，同时也要删除它关联的 metadata、原文存储、倒排索引等，否则可能出现“向量删了但原文还在”或者“检索结果还能召回已失效内容”的问题。

工程上通常还会配合几个机制：第一是 **文档 ID 和 chunk ID**，便于精确定位要更新或删除的内容；第二是 **版本管理**，避免新旧知识混用；第三是 **定时重建或增量同步**，保证知识库和原始数据源一致。

所以我一般会总结成一句话：**RAG 的更新和删除，核心不是改模型，而是对外部知识库做增量维护、索引重建和数据一致性管理。**

12.什么是GraphRAG 以及为什么需要GraphRAG

GraphRAG 可以理解为 **在传统 RAG 的基础上，引入知识图谱或图结构检索能力的一种增强方案**。传统 RAG 主要是把文档切成 chunk，存到向量库里，查询时靠向量相似度去召回相关文本；而 GraphRAG 不只是检索“相似文本”，还会把文本中的**实体、关系、事件和它们之间的连接结构**显式建成图，然后基于图去做检索、聚合和推理。

之所以需要 GraphRAG，是因为普通 RAG 更擅长回答“局部事实型问题”，比如某个概念是什么、某段文档里提到了什么；但当问题涉及**多跳关系、全局总结、跨文档关联**时，传统 RAG 往往会遇到瓶颈。因为向量检索召回的是相似 chunk，但这些 chunk 之间的关系并不显式，模型需要自己在上下文里临时拼接，容易漏信息，也容易丢全局结构。

GraphRAG 的优势就在于，它把“信息点之间怎么连接”也建模出来了。这样在回答问题时，不仅能找到相关内容，还能沿着实体关系做多跳检索，或者基于社区和子图做全局总结，所以特别适合复杂知识问答、研究分析、企业知识库和多文档推理场景。

所以我一般会总结成一句话：**GraphRAG 就是把传统 RAG 的“文本相似检索”升级为“文本 + 关系结构联合检索”，从而更适合处理跨文档、多跳推理和全局理解问题。**

13.GraphRAG的索引建立和查询的具体细节是什么

GraphRAG 的“索引建立”和“查询”如果按 **Microsoft GraphRAG** 的标准实现来讲，可以分成两大部分：

一、索引建立：先把文本变成“图 + 分层摘要 + 向量索引”

GraphRAG 的核心不是直接把文档 chunk 后丢进向量库，而是先把语料加工成一个**结构化知识层**。官方标准索引流水线大致是：

1）切分文档，得到 TextUnit

原始文档先被切成 **TextUnit**。这一步类似普通 RAG 的 chunking，但在 GraphRAG 里，TextUnit 不只是为了向量检索，更是后续实体抽取、关系抽取和溯源的最小分析单元。官方数据流里也明确把 `Document -> TextUnit` 作为第一阶段。(Microsoft GitHub)

2）从每个 TextUnit 抽取实体、关系、可选的 claim

接着系统会对每个 TextUnit 用 LLM 做图抽取，得到：

- **Entity**：实体
- **Relationship**：实体之间的关系
- **Covariate / Claim**：可选的声明、事实、时间约束信息

官方文档写得很明确：标准 GraphRAG 会用 LLM 做实体抽取、关系抽取、实体摘要、关系摘要，以及可选的 claim extraction。(Microsoft GitHub)

3）合并跨 chunk 的同名实体与关系，并做摘要归一

同一个实体可能在很多 TextUnit 里都出现。GraphRAG 会先把这些局部子图合并，再对重复出现的实体描述和关系描述做 **summarization**，把多处证据压缩成一个更稳定、更紧凑的全局实体描述和关系描述。官方说明里提到：同 title+type 的实体会合并描述数组，同 source–target 相同的关系也会合并，再让 LLM 生成单一摘要。(Microsoft GitHub)

4）做社区发现，形成层次化社区结构

图建好之后，不是直接拿整个图来查，而是要做 **hierarchical community detection**。Microsoft GraphRAG 默认使用的是 **Hierarchical Leiden Algorithm**，递归地把实体图切成不同粒度的社区，直到达到设定的社区规模阈值。这样就得到一个**层次化语义簇结构**。(Microsoft GitHub)

这一层很关键，因为它把“局部实体网络”进一步组织成“主题社区”。GraphRAG 能回答全局问题，很大程度就靠这一步。

5）为每个社区生成 Community Report

有了社区之后，系统会对每个社区生成一份 **Community Report**。这份报告不是简单摘要，而是面向后续检索和推理的“社区级知识压缩表示”。官方文档说明，这些报告会引用社区中的关键实体、关系，以及可选的 claims，并且可以覆盖不同层级：顶层社区更接近全局主题，底层社区更接近局部簇。(Microsoft GitHub)

6）再做向量化，写入向量存储

GraphRAG 不是不要向量，相反它会给多个层级的数据做 embedding，包括：

- TextUnit
- Entity / Relationship 的 description
- Community Report 的 content

官方概览明确写到：默认输出表存成 Parquet，而 embeddings 写到你配置的向量存储中。(Microsoft GitHub)

二、索引产物到底有什么

官方默认会产出一组知识表，常见核心表包括：

- `documents`
- `text_units`
- `entities`
- `relationships`
- `communities`
- `community_reports`
- `covariates`

其中 `communities` 表里会保存 parent、children、level、entity_ids、relationship_ids、text_unit_ids 等字段，也就是社区层次结构和它所覆盖的知识范围。(Microsoft GitHub)

所以你可以把 GraphRAG 的索引理解成三层：

原始文本层：documents / text_units
图结构层：entities / relationships / communities / covariates
摘要与检索层：community_reports + embeddings

三、查询：不是一种查法，而是多种 Query Mode

GraphRAG 官方 Query Engine 里主要有：

- **Local Search**
- **Global Search**
- **DRIFT Search**
- **Basic Search**

其中最核心的是前两个。(Microsoft GitHub)

四、Local Search 的具体流程

Local Search 适合回答**具体实体相关问题**，比如“某个人是谁”“某个概念和谁有关”“某个对象有哪些属性”。官方定义就是：把知识图谱中的结构化数据和原文中的非结构化文本一起放进上下文，用于实体级推理。(Microsoft GitHub)

它的流程可以理解成：

第一步：把用户问题映射到相关实体

用户 query（再加上可选的对话历史）会先通过 **entity description embedding** 去匹配图中的相关实体，也就是先找“这个问题最像哪些实体描述”。(Microsoft GitHub)

第二步：以这些实体为入口向外扩展候选上下文

一旦找到入口实体，就沿图和映射关系取出多类候选上下文：

- 与实体关联的 **Text Units**

- 与实体相关的 **Community Reports**
- 邻接的 **Entities**
- 相关 **Relationships**
- 可选的 **Covariates / Claims**

官方 Local Search 数据流图就是这样展开的。(Microsoft GitHub)

第三步：对候选信息做 ranking 和 filtering

不是所有候选都进 prompt。GraphRAG 会按优先级和 token 预算，对这些候选做排序和裁剪，最终组装成一个单一上下文窗口。(Microsoft GitHub)

第四步：把结构化 + 非结构化上下文送入 LLM 作答

最后 LLM 基于这个混合上下文生成答案。

所以 Local Search 的本质是：

query → 找入口实体 → 扩展周边图信息和原文片段 → 压缩成上下文 → 生成答案

它比普通向量检索强的地方在于：不是只找“相似 chunk”，而是能从实体出发，把关系、邻居实体、社区摘要和原文片段一起带进来。

五、Global Search 的具体流程

Global Search 适合回答**全局**、**总结性**、**主题性**的问题，比如：

- 这批文档的主要主题是什么?
- 全局上有哪些关键模式?
- 数据集中最重要的几类观点是什么?

官方文档明确说，Baseline RAG 对这类问题很差，因为向量检索只能找语义相似片段，无法天然做跨整个数据集的聚合；GraphRAG 的 Global Search 就是为这种“whole dataset reasoning”设计的。(Microsoft GitHub)

它的流程是：

第一步：选定某一层级的 Community Reports

Global Search 不直接从实体和原文开始，而是从某个社区层级的 **community reports** 开始。层级越低，报告越细；层级越高，报告越宏观。(Microsoft GitHub)

第二步：Map 阶段

社区报告会被分批或分块送入 LLM。对每一批，LLM 生成一个 **intermediate response**，其中包含若干点，并给每个点一个重要性评分。官方文档把这一步明确描述为 map 阶段。(Microsoft GitHub)

第三步：Reduce 阶段

系统把所有 intermediate responses 中高价值的点筛出来、聚合起来，再交给 LLM 生成最终答案。这就是 reduce 阶段。(Microsoft GitHub)

所以它本质上是：

community reports → map 生成局部观点 → 按重要性筛选 → reduce 汇总成最终全局答案

这就是为什么 GraphRAG 很适合“Top themes / key trends / overall patterns”这类问题：它不是在 chunk 级别做命中，而是在**社区摘要级别做分治式总结**。原始论文也明确把它描述成一种面向全局问题的 query-focused summarization 方案。(arXiv)

六、为什么 GraphRAG 能比普通 RAG 更适合复杂问题

核心原因有三点。

第一，普通 RAG 主要依赖向量相似度找 chunk，擅长局部相关片段召回；GraphRAG 在索引阶段先把**实体—关系—社区层次**建出来，所以查询时能显式利用结构信息。(Microsoft GitHub)

第二，GraphRAG 通过 **Community Report** 预先把大规模语料压缩成多粒度的主题摘要，因此它对全局主题分析比 baseline RAG 更有优势。(Microsoft GitHub)

第三，Local Search 能把图谱结构和原文片段一起用，而不是二选一，因此更适合多跳关系和实体相关问题。(Microsoft GitHub)

七、你在面试里可以怎么总结

你可以直接背这一段：

GraphRAG 的索引建立，本质上是把原始文档先切成 TextUnit，再用 LLM 抽取实体、关系和可选 claim，合并并摘要成全局图；然后对图做分层社区发现，并为每个社区生成 Community Report，最后再给文本、实体描述和社区报告做向量化。查询时，Local Search 会先从 query 找相关实体，

再扩展相关关系、邻居实体、社区报告和原文片段来回答具体问题；Global Search 则直接基于某一层社区报告做 map-reduce 式聚合，回答全局总结类问题。

如果你愿意，我下一条可以继续给你整理成**“面试背诵稿的 1 分钟回答”版本**。

14 RAG系统评估

RAG 系统评估，本质上是在评估它有没有做到两件事：**第一，检索是否把对的内容找回来；第二，生成是否基于这些内容回答得准确、完整、可信。** 所以 RAG 的评估一般不会只看最终答案，而是分成**检索层、生成层和端到端系统层**三个层面。

在**检索层**，常见指标有 Recall、Precision、Hit Rate、MRR、NDCG，主要看相关文档有没有被召回、排序是否靠前。因为如果检索阶段就没把关键信息找回来，后面的生成再强也答不好。

在**生成层**，通常看 Answer Correctness、Faithfulness、Relevance，也就是答案本身对不对、是否忠于检索到的上下文、是否真正回答了用户问题。这里一个核心点是要防止幻觉，也就是模型编造检索结果里没有的信息。

在**系统层**，还要看端到端效果，比如 Exact Match、F1、人工偏好评估，以及延迟、成本、吞吐这些工程指标。因为线上 RAG 不只是要答对，还要答得快、成本可控、体验稳定。

所以我一般会总结成一句话：**RAG 评估不是只看答案好不好，而是要同时评估“检索质量 + 生成质量 + 系统性能”，本质上是在评估整个“找资料并基于资料回答”的链路是否可靠。**

15 什么是MCP?

MCP 通常指 **Model Context Protocol**，也就是**模型上下文协议**。它的核心作用是为大模型提供一种统一、标准化的方式，去连接外部工具、数据源和运行环境，让模型不只是依赖自身参数回答问题，而是能够安全、规范地获取外部上下文并执行操作。

你可以把它理解成大模型领域里的“统一接口协议”。在没有 MCP 的时候，不同工具、数据库、文件系统、业务系统往往都要分别做一套适配，模型调用外部能力的方式比较碎片化；而有了 MCP 之后，工具和数据源可以按统一协议暴露能力，模型或 Agent 只需要按这个协议去发现、读取和调用即可。

它的价值主要有三点。第一是**标准化**，降低工具接入成本；第二是**解耦**，把模型和具体工具实现分开；第三是**可扩展**，便于构建更复杂的 Agent 系统。比如一个智能体要访问本地文件、知识库、数据库、搜索工具，如果这些能力都通过 MCP 暴露，整个系统会更统一、更容易维护。

所以我一般会总结成一句话：**MCP 是让大模型以统一协议连接外部工具和上下文的一种标准，目的是把模型能力从“只会生成”扩展到“能安全、规范地使用外部世界”。**

16 什么是Skills?

Skills 可以理解为**把大模型完成某一类任务的能力，封装成可复用的功能模块**。它通常不只是一个简单 prompt，而是一套更完整的能力单元，里面可能包含提示词、工具调用、输入输出格式、执行步骤，甚至还包括异常处理和结果校验。

它为什么重要？因为如果每次都临时写 prompt、临时拼流程，系统会很难复用，也很难维护。把常见任务封装成 Skill 之后，比如“总结文档”“查询知识库”“生成周报”“调用数据库分析数据”，上层 Agent 或 Workflow 就可以像调用组件一样直接使用。

从本质上讲，Skill 更像是大模型时代的“能力模块”或者“任务插件”。它位于底层模型和上层业务之间，起到一个封装和复用的作用。这样做的好处是**降低重复开发成本、提高稳定性、增强可组合性**。

如果放到 Agent 系统里看，Agent 负责根据目标做决策，而 Skill 负责执行某一种具体能力，所以两者关系可以理解为：**Agent 是大脑，Skill 是它可调用的标准化能力单元。**

所以我一般会总结成一句话：**Skills 就是把大模型完成特定任务的流程和能力封装成可复用模块，方便在 Workflow 或 Agent 系统中稳定调用。**

17.什么是openclaw?

OpenClaw 是一个开源、可自托管的 **AI Agent / 个人 AI 助手平台**。它的定位不是单纯聊天，而是把大模型接到你的本地文件、消息渠道和工具系统上，让它能执行任务，比如读文件、调用工具、做自动化，甚至通过 Telegram、Discord、WhatsApp 这类渠道远程给它下指令。官方仓库把它描述成“Personal AI Assistant”，并且提供了技能注册与自动拉取能力。(GitHub)

从架构理解，它更像一个“**大模型 + 工具系统 + 技能机制 + 消息入口**”的执行平台。也就是说，LLM 负责理解意图和推理，OpenClaw 负责把这些能力接到本地环境和外部渠道上，再通过 skills 扩展更多具体功能，所以它本质上属于比较典型的 Agent 框架，而不是传统的单轮问答系统。(GitHub)

它为什么火，是因为它把“个人 AI 助手”这个概念做得很具体：不是只会回答问题，而是真的能长期运行、接消息、调工具、做自动化任务。不过它这类系统也有明显风险，最近公开报道里就提到过 OpenClaw 的安全漏洞问题，因此实际使用时安全、权限隔离和升级维护都很重要。(TechRadar)

所以我一般会总结成一句话：**OpenClaw 是一个开源自托管的 AI Agent 平台，核心目标是让大模型从“会聊天”升级为“能接入工具并持续帮你执行任务的个人助手”。**（GitHub）

18 Agent的记忆模式有哪些 以及具体的工作流程

Agent 的记忆模式，通常可以分成 **短期记忆、长期记忆和工作记忆** 三类，也有些系统会再补充 **情节记忆** 和 **语义记忆** 这种更细的划分。它的核心目的，是让 Agent 不只是“当前这一轮会回答”，而是能在多

轮任务中保存上下文、积累经验，并在后续步骤里继续使用。

短期记忆 一般指当前会话内的上下文，比如用户刚刚说了什么、上一轮工具返回了什么，主要用于维持连续对话和当前任务状态。**长期记忆** 则是把对未来还有价值的信息持久化下来，比如用户偏好、常用规则、历史任务经验，通常会存到数据库、向量库或知识库里。**工作记忆** 更像任务执行时的临时草稿区，保存当前计划、子任务结果、待办步骤，任务结束后可以丢弃。

它的工作流程一般是：**先感知输入，再判断哪些信息要进入短期上下文；如果信息有长期价值，就写入长期记忆；执行任务时从短期记忆和长期记忆中检索相关内容，组装成当前上下文供大模型推理；任务完成后，再决定是否把这次经验总结并存回长期记忆。**

所以我一般会总结成一句话：**Agent 的记忆机制，本质上就是“当前上下文管理 + 长期经验存储 + 执行过程暂存”的组合，用来支持多轮对话、持续学习和复杂任务执行。**

19 分别说说 Agent 的 短期记忆、长期记忆和工作记忆 的流程

Agent 的**短期记忆、长期记忆和工作记忆**，本质上对应三种不同时间尺度的信息管理方式，它们的流程也不一样。

短期记忆 主要服务当前会话。它的流程一般是：用户输入进来之后，系统先把最近几轮对话、最新的工具返回结果、当前任务状态放进上下文窗口里；模型就在这个临时上下文上推理和作答；随着对话继续，新的信息不断加入，旧的信息可能被截断、压缩或者总结。所以短期记忆的重点是**维持当前对话连续性**，它通常依赖会话历史、滑动窗口或者摘要机制。

长期记忆 主要服务跨会话和长期个性化。它的流程一般是：系统先判断当前信息有没有长期价值，比如用户偏好、固定规则、历史经验；如果有，就把它结构化后存到外部存储，比如数据库、向量库或知识库；当后续来了新任务，Agent 会先根据当前问题去长期记忆里检索相关信息，再把检索结果拼回当前上下文供模型使用。所以长期记忆的核心流程就是**写入、存储、检索、回注入上下文**。

工作记忆 则主要服务任务执行过程。它更像一个临时草稿板。流程通常是：Agent 接到复杂任务后，先把目标拆解成若干子任务，再把中间结果、待办步骤、当前计划、工具调用结果暂存在工作记忆里；模型每做一步都会更新这块内容，直到任务结束；任务完成后，这部分信息通常会被清空，或者只把有价值的经验提炼后写入长期记忆。所以工作记忆的重点是**支撑任务中的多步推理和状态跟踪**。

我一般会用一句话总结：**短期记忆管当前对话，长期记忆管跨会话沉淀，工作记忆管任务执行中的临时状态。**

20.什么是多智能体协同 以及为什么需要多智能体协同?

多智能体协同，指的是**把一个复杂任务拆给多个具有不同职责的 Agent，由它们分工合作、相互通信、共同完成目标**。它的核心思想和人类团队协作很像，不再依赖一个单一 Agent 从头到尾完成所有事情，

而是让不同 Agent 各自负责自己擅长的部分，比如有的负责任务拆解，有的负责检索资料，有的负责代码执行，有的负责结果审查和汇总。

之所以需要多智能体协同，是因为单智能体在面对复杂任务时，往往会遇到几个问题：第一是**上下文压力大**，所有信息都堆在一个 Agent 里容易混乱；第二是**角色冲突**，同一个 Agent 既要规划、又要执行、还要审查，效果通常不稳定；第三是**复杂任务链路长**，单 Agent 容易中途跑偏或者遗漏步骤。

而多智能体协同的优势在于，它可以通过**角色分工、并行处理和相互校验**来提升整体效果。比如一个研究型 Agent 系统里，可以让 Planner 先拆任务，Retriever 去查资料，Analyst 做分析，Reviewer 负责检查结果，这样既能提高复杂任务完成质量，也更容易扩展和维护。

所以我一般会总结成一句话：**多智能体协同就是让多个 Agent 像团队一样分工合作，目的是解决单智能体在复杂任务中容易过载、失稳和效率不足的问题。**

21.Memory的多用户并发怎么实现?

Memory 的多用户并发实现，核心就是一句话：**把记忆从“模型内部状态”变成“外部可隔离、可读写、可并发控制的数据层”**。也就是说，真正上线时不能把所有用户记忆混在一起，而是要按用户维度、会话维度甚至租户维度做隔离。

具体实现上，第一步是**内存分区**，通常会设计 `user_id` 、 `session_id` 、 `tenant_id` 这类主键，让每个用户的短期记忆和长期记忆都有独立命名空间。第二步是**存储分层**，短期记忆一般放 Redis 这类高速缓存，长期记忆放数据库、向量库或知识库存储。第三步是**并发控制**，当同一用户同时发起多个请求时，需要用乐观锁、版本号、事务或队列机制避免脏写和覆盖，比如更新 memory 时带上 version，写入前校验版本是否一致。第四步是**异步化处理**，把记忆写入、摘要压缩、向量化这些重操作放到后台异步任务里，避免阻塞主链路。

如果是高并发场景，还会做**读写分离、分库分表、缓存加速和事件驱动更新**，保证 memory 检索快、写入稳。

所以我一般会总结成一句话：**Memory 的多用户并发，本质上是通过用户隔离、分层存储、并发控制和异步更新，把记忆系统做成一个可扩展的共享状态服务。**

22.RAG幻觉的解决办法

RAG 的幻觉，本质上是**模型没有严格基于检索到的证据回答**，要么是没检索到关键内容，要么是检索到了但模型还是自己补了一些不可靠的信息。所以解决 RAG 幻觉，不能只盯着生成模型，而是要从**检索、上下文构造、生成约束和结果校验**四个层面一起做。

第一，**提高检索质量**。如果相关文档根本没召回，模型就容易乱编，所以要优化 chunk 切分、embedding、混合检索、重排序，尽量把真正有用的证据找回来。第二，**控制上下文质量**。不是召回越

多越好，而是要把最相关、最完整、噪声最少的内容送进模型，避免无关片段干扰。第三，**在生成阶段加约束**，比如明确要求“只能基于给定上下文回答，如果证据不足就直接说不知道”，并要求附带引用。第四，**做后处理校验**，比如用规则或二次模型检查答案是否被证据支持，发现没有依据的内容就拒答或回退。

在高要求场景里，还会加一些更强的机制，比如 **query rewrite**、**rerank**、**citation grounding**、**faithfulness 检测**、**self-reflection 或 verifier agent**。

所以我一般会总结成一句话：**解决 RAG 幻觉的核心，不是单纯让模型更聪明，而是让它“先找到对的证据，再只根据证据回答，并对结果做校验”。**

23 什么是A2A协议?

A2A 协议，通常指 **Agent2Agent Protocol**。它是一个面向 **AI Agent 之间通信与协作** 的开放协议，核心目标是让不同厂商、不同框架、不同组织里的 Agent 可以用统一方式发现彼此、交换消息、协同完成任务。这个协议最早由 Google 在 2025 年提出，后来进入了 Linux Foundation 的治理体系。

你可以把它理解成：**MCP 更像“模型怎么接工具和数据”，而 A2A 更像“Agent 和 Agent 之间怎么对话和协作”**。A2A 关注的是跨 Agent 的能力发现、任务请求、状态更新、结果返回和安全通信，让一个 Agent 不只是把另一个 Agent 当工具调用，而是把它当成一个独立协作者。

之所以需要 A2A，是因为现在很多 Agent 是基于不同框架开发的，彼此之间如果没有统一协议，就只能做大量定制集成，系统会很碎片化。A2A 的价值就在于提供一个公共语言，降低多智能体系统的互联成本，提高跨平台协同能力。

所以我一般会总结成一句话：**A2A 协议就是为多智能体系统提供统一通信标准，让不同 Agent 能像网络服务一样互相发现、协作和交换任务结果。**

24 LoRA

LoRA 的全称是 **Low-Rank Adaptation**，中文一般叫**低秩适配**。它是一种参数高效微调方法，核心思想是：**微调大模型时，不去更新全部原始参数，而是在某些线性层旁边额外加入两个低秩矩阵，只训练这部分小参数**。这样就能用很小的训练成本，让模型适配新任务。

它为什么有效，是因为很多微调带来的权重变化，其实可以用一个低秩更新来近似表示。也就是说，原来一个很大的参数矩阵，不一定需要全部改动，只需要用一个低维分解出来的增量去补充，就足够学到任务相关能力。

LoRA 的优势很明显：第一，**显存占用低**，因为冻结了大部分原模型参数；第二，**训练成本低**，只训练少量新增参数；第三，**易于切换和部署**，同一个底座模型可以挂不同的 LoRA 适配器，对应不同任务。

它特别适合大模型场景，因为全量微调成本太高，而 LoRA 能在较低资源下完成领域适配、指令微调或者下游任务微调。

所以我一般会总结成一句话：**LoRA 就是在冻结大模型主体参数的前提下，用低秩增量参数来完成微调，从而以很低成本适配新任务。**

RAG部分：

1.RAG系统中如何处理多跳回答?

多跳问答需要RAG系统通过多次检索和推理才能得到答案，比如"ChatGPT的开发公司CEO是谁"需要先查到OpenAI，再查SamAltman。处理这类问题，核心是**将复杂问题分解成检索链路**，让系统具备"边查边想"的能力。

实践中主要有几种做法。最直接的是**迭代检索**，让LLM根据初次检索结果判断是否需要继续查询，生成新的检索query，直到信息充足。这需要设计好停止条件，避免无限循环。更主动的方式是**查询分解**，先让LLM把问题拆成子问题序列，按依赖关系逐个检索并聚合答案，像上面例子就拆成"ChatGPT开发公司“和"该公司CEO"两步。

技术上还会用**图结构增强**，把知识库预处理成实体关系图，通过图遍历找多跳路径。或者采用**自我提问机制**，让模型在推理过程中主动生成需要补充的问题再检索。关键是**上下文管理**，每次检索要把前序结果带上，让模型理解完整推理链。现在也有混合方案，结合dense retrieval做初筛和reranker做精排，每一跳都保证检索质量。实际部署时要注意延迟控制，多跳会增加响应时间，通常限制在2-3跳内，并行检索可以加速但要处理好信息融合。

2.什么是self-RAG 如何让模型自主判断是否需要检索?

Self-RAG是一种让大模型**自主决定何时需要检索外部知识的增强框架**。传统RAG对所有问题都执行检索，而Self-RAG通过在**训练阶段引入特殊的反思token(reflectiontokens)**，让模型学会自我判断和控制检索行为。

核心机制是**训练模型生成几类特殊token**:Retrieve标记判断是否需要检索、ISREL评估检索内容的相关性、ISSUP判断生成内容是否有检索证据支持、ISUSE评估回答的整体质量。**模型在生成过程中会先输出Retrieve token**，如果判定为“需要检索”才会触发外部知识库查询，否则直接用自身参数知识回答。

实现方式上，你需要构建包含这些反思token的训练数据集，让模型在监督学习中掌握判断逻辑。比如面对”今天天气怎么样"这类实时信息查询，模型输出Retrieve=Yes触发检索;而"什么是梯度下降"这种通用

知识问题，模型输出Retrieve=No直接作答。这样做的好处是减少不必要的检素开销，同时通过ISREL和ISSUP这些质量评估token，能让模型在推理时对检索结果和生成内容进行自我校验，提升回答的准确性和可信度。本质上是把检索决策权交给模型本身，而不是外部规则系统。

3 RAG系统中如何支持多模态检索 图文检索如何实现?

RAG系统支持多模态检索的核心在于**将不同模态数据映射到统一的向量空间**。这不是简单地支持多种数据类型，而是让图像和文本能够真正"对话"——用户上传一张图片能找到相关文字描述，**输入文字能召回相关图像**。具体来说，你需要使用**多模态编码器**(如CLIP、BLIP等)分别对文本、图像进行编码，这些编码器经过对比学习训练，能够让语义相关的图文在向量空间中距离更近。

图文检索的实现流程分三步走:第一步是**建立索引**，用多模态编码器把知识库里的图片和文本分别编码成向量，存入支持多模态的向量数据库，同时**打上模态标签**。第二步是**查询处理**，用户无论输入图片还是文本，都用同一个编码器转成向量。第三步是**跨模态检索**，在向量空间做相似度匹配，这时图查文、文查图就自然实现了。

实际应用中效果非常直观。比如医疗影像分析系统，医生输入”肺部结节"这个文本查询，系统不仅能返回相关的病历文本，还能检索出相似的CT影像及其诊断报告。或者在电商场景，用户上传一张商品图片，系统能同时召回相似商品图片和对应的商品描述文本。技术栈上主要涉及三块:编码器层用CLIP或BLIP这类多模态模型，存储层用Milvus或Weaviate这种原生**支持多模态的向量数据库**，检索层还需要考虑模态融合策略，比如是在特征层面融合还是结果层面做重排序。

4.增量更新场景下，RAG的向量库如何维护?如何处理文档删除和修改?

RAG向量库的**增量维护**核心在于**建立文档ID与向量ID的映射关系**。这个问题的本质是要解决文档和向量的一致性问题，因为一个文档会被切分成多个chunk，每个chunk对应一个向量，所以文档的任何变动都需要精准地映射到向量层面的操作。

在**插入阶段**，每个文档切片后生成的向量都要**关联原始文档ID**和chunk位置信息，存入向量库的**metadata字段**。这样后续就能精准定位需要操作的向量。删除文档时，**通过文档ID查询出所有关联的向量ID，批量调用向量库的delete接口删除**。像Pinecone、Milvus这些向量库都支持基于metadata的过滤删除，一条delete语句就能清理干净。

修改文档更麻烦些，本质是**先删后插**。你需要先用旧文档ID删除所有向量，然后对修改后的文档重新做**embedding和切片**，生成新向量插入。这里有个优化点是可以对比新旧文档的chunk差异，只删除和插入变化的部分，**但实现复杂度会高不少**，通常全量替换就够用了。

实际应用中还要维护一张关系表，**记录文档ID、向量ID列表、文档版本号和时间戳**。这样既能追溯历史，也方便做并发控制。比如用户同时修改同一文档时，通过版本号判断是否需要重新embedding。另外要注意向量库的删除通常是标记删除，需要定期compact来真正释放存储空间，这点在设计定时任务时要考虑进去。

5.知识图谱怎么和大模型结合?检索增强生成(RAG)怎么用图谱?

知识图谱和大模型的结合主要有三种模式。**第一种是把图谱作为外部知识库**，在推理时通过实体链接和关系检索来增强模型输出比如问"阿司匹林治什么病”，先从问题中识别出"阿司匹林"实体，然后去医学知识图谱里查它的适应症关系，把这些结构化知识和原问题一起喂给大模型生成答案。**第二种是用图谱来微调或预训练模型**，把三元组转成文本或者直接用图神经网络编码图结构，让模型学习实体间的关系模式。**第三种是让大模型生成或补全图谱**，用模型的常识推理能力来做知识抽取和关系预测。

在RAG场景下用图谱会比单纯的向量检索更精准。传统RAG是把文档切片后做向量相似度匹配，容易召回不相关的内容。如果引入图谱，**可以先做实体识别和图查询，沿着关系路径精确定位到相关知识子图**，比如查”某药物的副作用和禁忌人群”，能通过图谱的has side effect和contraindication关系直接找到答案节点，然后再结合这些节点关联的文档片段生成回答，这样既保证了知识的准确性，又能追溯来源。实际应用中像金融风控、医疗问诊这种需要严格逻辑推理的场景特别适合这么做。

6.什么是知识图谱?实体、关系、属性分别是什么?

知识图谱是一种**结构化的语义知识库**，以图的形式存储和表示现实世界中的实体及其相互关系。它的核心是**用节点和边来组织知识**，让机器能够理解和推理。你可以把它想象成一张巨大的知识网络，**每个节点都是一个知识点，节点之间的连线则说明了它们的关联方式。**

这张网络里有三个核心要素。**实体**就是图中的节点，代表客观存在的事物或抽象概念。比如在医疗知识图谱中，”阿司匹林"、"心脏病”、“张三"都是实体，它们是知识的基本单元。**关系**是连接两个实体的边，描述实体之间的语义关联。像"阿司匹林–治疗–心脏病”中的"治疗"就是关系，它让独立的实体产生了意义连接。关系是有方向性的，“张三–患有–心脏病”和”心脏病–患者–张三“虽然信息相近但语义方向不同。**属性**是附加在实体上的键值对信息，用来描述实体的特征。比如实体"阿司匹林"可以有属性"剂型:片剂”、“生产厂家:拜耳"、"有效期:3年"，这些属性丰富了实体的描述维度。

在实际应用中，搜索引擎用知识图谱构建右侧的知识卡片，智能问答系统通过遍历关系路径来回答”阿司匹林能治疗什么病”，推荐系统利用实体间的关联做个性化推荐。知识图谱本质上是让碎片化的数据建立了可计算的语义网络，这是它区别于传统数据库的关键。

7.DST怎么处理多轮对话?历史轮次的信息怎么利用?

DST(对话状态追踪)处理多轮对话的核心是维护和更新一个**动态的状态表示**，这个状态会随着对话推进不断演化。在每一轮对话中，DST模型会接收当前用户输入，然后基于历史信息来更新slot-value对。

历史信息的利用有几种主流方式:最直接的是把**前几轮的对话文本拼接起**来作为上下文输入，让模型能看到完整的对话历史;更高效的做法是**只传递上一轮预测的状态**，新一轮在这个状态基础上做增量更新，这样既能保留历史信息又避免输入过长。

举个实际场景，用户订机票时第一轮说"我要去北京”，DST提取出“目的地=北京”;第二轮用户说“下周三出发”，DST需要在保留“目的地=北京”的同时，新增”出发日期=下周三”;第三轮用户说"改成周四吧”，这时DST要识别出这是对之前时间slot的修改操作，更新为“出发日期=下周四”。现代的DST模型通常会用Transformer架构编码整个对话历史，或者采用门控机制来决定哪些历史slot需要保留、更新还是删除。

8.知识图谱补全(Knowledge Graph Completion)怎么做?怎么预测缺失的关系?

知识图谱补全**本质上是在已有三元组(头实体、关系、尾实体)基础上预测缺失的关系或实体。核心思路是将图谱中的实体和关系映射到低维向量空间，通过向量运算来建模它们之间的语义关联。**想象一下，传统知识图谱里"iPhone°和”Apple"只是两个标签，计算机无法直接理解它们的语义关系。嵌入方法的价值就是给每个实体分配一个稠密向量，让语义相近的实体在向量空间里距离也近，这样就能通过向量运算来发现隐藏的关联。

主流方法可以分为几个流派。基于翻译的模型像TranSE把关系看作头实体到尾实体的平移向量，即h+r≈t，通过最小化这个距离来训练;TransH、TransR等改进版本能更好地处理一对多、多对多关系。基于语义匹配的模型如DistMult、ComplEx则用双线性函数或复数空间计算实体-关系的匹配度，不再依赖距离而是看相似性。基于神经网络的方法像ConvE用卷积网络捕捉特征交互，图神经网络(GNN)则聚合邻居信息学习实体表示，在稀疏图或新实体预测上有独特优势。

预测时的套路很直接:对于给定的(头实体，关系，?)或(?，关系，尾实体)，遍历所有候选实体计算评分，选择得分最高的作为预测结果。实际应用遍布各个领域，电商场景预测"商品-适用人群"关系帮助完善商品属性，医疗领域补全"疾病-治疗药物“链接辅助诊断决策，搜索引擎通过补全实体关联提升查询理解能力。训练过程通常采用负采样策略，随机替换正确三元组中的实体构造负样本，用对比学习方式优化嵌入向量，让模型学会区分正确和错误的关系。

9.RAG中的幻觉问题如何缓解?引用溯源(Citation)如何实现?

RAG中的幻觉问题主要源于三个层面:检索质量低导致喂给模型的上下文就是错的、上下文与问题不匹配产生的理解偏差、以及LLM自身倾向于用预训练知识”自作聪明”地填补空白。缓解策略需要从检索和生

成两端同时入手，建立分层防御体系。

在检索侧，核心是提升召回精度。可以采用**混合检索策略**，让稠密向量处理语义理解，稀疏检索和关键词匹配补足专有名词、数字、日期这类精确匹配的短板。比如用户问”iPhone 15 Pro Max的电池容量"，纯向量检索可能召回所有iPhone的续航讨论，但加上关键词过滤就能精准定位到具体型号的参数文档。重排序模型是第二道防线，用Cross-encoder对粗筛结果做深度交互打分，把最相关的topk文档留下来。chunk切分的粒度也很关键,按段落或固定token数切分时保留一定overlap，避免关键信息被割裂在边界上。

在生成侧，prompt工程是核心武器。不是简单写一句"别瞎编”，而要包含明确的角色定位、**严格的回答边界、以及输出格式要求。**比如角色定位用"你是一个严谨的知识库助手"而不是”AI助手”，能激活模型更保守的生成模式。回答边界要明确写”仅根据以下上下文回答，如果上下文中没有相关信息，请明确回复根据现有信息无法回答”。配合**降低temperature到0.1-0.3**，让模型更倾向于选择高概率token，减少天马行空的发挥。

引用溯源的实现思路是让模型在生成答案时标注信息来源，让整个过程变得可追溯、可验证。最直接的方法是在检索到的文档块中添加唯一标识，在prompt里要求模型引用这些标识。更精细的做法是在后处理阶段做验证，提取模型答案中的关键语句与原始文档做相似度匹配，自动补充引用链接或高亮原文位置。在法律问答、医疗咨询这类高风险场景，把引用的原文片段直接展示给用户，既增强可信度又方便人工审核。

10.什么是GraphRAG?知识图谱如何增强RAG系统?

GraphRAG是**将知识图谱与检索增强生成相结合的技术方案**。传统RAG通过向量相似度检索文档片段，但这种方式容易去失实体间的关系信息。GraphRAG则先将文档构建成知识图谱，把实体作为节点、关系作为边存储，检索时不仅能找到相关实体，还能沿着图结构获取多跳关系和上下文信息。

知识图谱对RAG的增强主要体现在三个方面。首先是**关系推理能力**，比如查询"某药物的副作用"时，可以通过图谱遍历药物-成分-副作用的路径，获得比单纯文本检索更完整的答案。其次是**消除歧义**，当检索到”苹果"这个实体时，图谱中的关系边能明确区分是水果还是公司。最后是**知识一致性**，图谱的结构化特性避免了向量检索可能带来的矛盾信息。

如果你在面试中遇到这个问题，第一句话非常关键。建议直接点明本质:"GraphRAG是在RAG的检索环节引入知识图谱，让系统不仅能找到相关内容，还能理解内容之间的关联关系。“这句话能立即让面试官知道你理解了这个技术的核心定位。接下来可以用对比的方式回答:传统RAG就像用搜索引擎找资料，找到的都是独立的文档片段，系统并不知道这些片段之间有什么联系;而GraphRAG会先把知识整理成一张关系网络，检索的时候能顺着这张网找到更多相关信息。

说到价值点时，别急着列举一堆技术术语。面试官更关心的是"解决了什么实际问题"。你可以举个直观的例子:用户问"这款手机适合学生吗”，传统RAG可能只返回产品参数，但GraphRAG能沿着“手机-价格区

间–目标人群"这条路径，结合“学生–预算敏感”的关系，给出更贴合需求的回答。实际应用中，金融领域用GraphRAG分析企业关联关系，医疗场景用它构建疾病–症状–药物的知识网络来辅助诊断。技术实现上通常采用混合架构:向量检索负责召回候选节点，图遍历算法扩展相关子图，最后将子图序列化后送入LLM生成答案。这种方式让RAG系统从"检索相似文本"进化到"理解知识关联”

11.RAG系统中文档切分的策略有哪些?如何选择合适的chunk size?

RAG系统的文档切分策略主要包括**固定长度切分**、**语义边界切分**和**混合策略**这几种思路。固定长度切分就是按字符数或token数硬切，实现简单性能好，但可能会把一个完整的段落或代码块从中间切断，破坏语义完整性。语义边界切分会基于段落、句子、章节等自然结构进行分割，保留语义连贯性，常用的像按换行符、标题层级来切，或者使用NLP模型识别语义单元。混合策略则是在语义切分基础上控制chunk大小范围，兼顾两者优势，还有更进阶的递归切分策略，先按大的语义单元比如章节切分，如果某个章节还是太大就继续按段落切，再大就按句子切，这种分层降级的思路在技术文档场景特别有用。

chunk size的选择需要平衡几个因素。首先看你的**embedding模型能接受多长输入**，比如很多模型限制在512 tokens。其次**考虑检索精度**，chunk太大会包含无关信息降低召回准确率，太小则上下文不足影响理解，一般200–500tokens是个经验起点。还要看具体场景，技术文档可能需要更大的chunk保持代码完整性，问答类内容可以切得更细。实践中建议设置overlap重叠区域，比如chunk间重叠10–20%，避免关键信息被切断。最终还是要通过实验验证，用你的真实数据跑检索评估指标，观察不同size对召回率和答案质量的影响，找到最优配置。

12 上下文压缩(Context Compression)技术有哪些?如何减少token消耗?

上下文压缩技术主要是为了**在保留关键信息的前提下减少输入token数量**，这个问题的核心是在大模型应用中，如何在保证回答质量的前提下，把输入的上下文从几千甚至上万token降下来，毕竟token既影响成本也影响响应速度。

从技术实现角度来看，主要分为三个方向。**过滤型通过相似度计算把不相关的内容直接删掉**，比如使用embedding模型计算文档片段与查询问题的相似度，只保留相关性高的部分送入LM，这能直接过滤掉无关内容。检索到10个文档片段，通过相似度阈值筛选后可能只需要保留3个最相关的，token消耗立刻降到原来的30%。**压缩型用摘要或者关键信息提取把长文本变短，可以利用小模型做摘要提取，把长文档先压缩成关键句子或要点，再作为上下文输入。截断型设定优先级规则保留最重要的部分**，比如滑动窗口策略对于超长对话历史，只保留最近N轮对话加上初始系统提示，中间部分可以用一句话摘要代替。

实际应用中，客服机器人就常用滑动窗口方法，保留最近5轮对话详情，之前的对话压缩成"用户咨询了退款问题”这类简短描述。LLMLingua这类专门的压缩框架会更激进，通过删除冗余token、保留关键词的方式，能在保持80%语义的情况下压缩到原来的20-30%。本质上就是提高每个token的信息密度，用户不会等你处理几万token，他们要的是秒级响应

13 RAG系统的评测指标有哪些?如何评估检索质量和生成质量?

RAG系统的评测需要分别关注**检索和生成两个环节**。

检索质量评估主要看能否找到相关文档，核心指标包括Recall@K(前K个结果中包含相关文档的比例)、MRR(首个相关文档的排名倒数)、NDCG(考虑排序质量的归一化折损累积增益)。实际应用中，你可以用人工标注的问答对作为基准，看检索出的文档片段是否真正包含答案信息。Context Relevance Score也很重要，它衡量检索内容与问题的语义相关度。

生成质量评估则更复杂。传统指标有BLEU、ROUGE这些文本匹配度量，但它们往往不够准确。现在更推荐用Faithfulness(生成内容是否忠实于检索文档，有没有幻觉)和Answer Relevance(回答是否真正解决用户问题)。你可以用GPT-4这类强模型来自动评判这两个维度，或者结合人工评测。

端到端评估还要看最终用户满意度，比如在客服场景中统计解决率、用户反馈评分。有些团队会用A/B测试比较不同RAG配置的实际效果。另外Context Utilization也值得关注，即模型是否有效利用了检索到的信息，而不是仅依赖自身知识回答。整体来说，评测RAG不能只看单一指标，需要建立符合业务场景的综合评价体系。

14 如何选择合适的Embedding模型?开源模型vs闭源API的权衡?

选择Embedding模型主要看三个维度:**性能需求**、**成本控制和部署灵活性**。这不是个非黑即白的选择题，而是要根据你的具体场景做权衡。

如果你的应用对检索质量要求高，比如企业级知识库或复杂的语义搜索，OpenAI的text-embedding-3系列或Cohere的embed-v3通常效果最好。这些闭源API在多语言和领域泛化上优势明显，调用一个HTTP接口就能拿到高质量的向量表示。但要注意API调用会产生持续费用，每次embedding都在烧钱，数据需要传输到外部服务，还可能受限于调用频率。如果你日调用量上百万，每个月光AP费用可能就要大几千美元。

开源模型像BGE系列、GTE、M3E在中文场景表现不错，关键优势是可以本地部署，数据完全可控，没有按量付费压力。假设你租一台带GPU的云服务器，不管跑一万次还是一千万次embedding，硬件成本基本固定，边际成本几乎为零。如果你的数据敏感或调用量巨大，自建服务的RO1会远低于API。不过你需要自己承担GPU资源、模型优化和维护工作，这部分工程投入不能忽视。

实际权衡时建议这样考虑:原型验证阶段直接用AP快速迭代，一天就能把demo跑起来，确定方案可行性。**进入生产后如果日调用量超过百万次，或者处理金融、医疗等敏感数据，就该评估切换到开源模型自部署。**另外可以采用混合策略，**核心业务用AP保证质量，边缘场景用开源模型降本**，这样既控制了成本又保证了关键路径的效果。

记得选型时必须在自己的真实数据上做评估，不要只看公开benchmark。不同模型在特定领域的表现差异可能很大，电商场景最优的模型放到法律文书检索上可能就不行了。用MTEB或自建测试集跑一遍，让数据告诉你答案，这比看任何榜单都靠谱。

15 RAG中的重排序(Reranking)如何工作?有哪些重排序模型?

重排序是**RAG检索后的精排环节**，工作原理是**对初筛的候选文档与查询进行精确相关性打分，重新排列顺序**，把最相关的内容送给大模型。初筛阶段通常用向量检索快速召回几十上百个候选，但向量模型对语义细节捕捉不够精准，这时重排序模型会对每个<查询,文档>对进行深度交互计算，输出0-1之间的相关性分数，根据分数降序取Top-K。

核心区别在于交互深度:向量检索是分别编码查询和文档再做相似度计算，而重排序模型让查询和文档在模型内部充分交互，通过Cross-Attention机制捕捉细粒度的语义匹配信号，**准确性更高但计算成本也更大**，所以放在二阶段处理。

常见的重排序模型包括**:Cohere Rerank系列(商用API效果好)、bge-reranker(智源开源，中英文支持)、jina-reranker(支持多语言长文本)、以及基于BERT的Cross-Encoder架构模型。**实际应用中，比如法律文档检索，初筛可能召回50条相关法条，重排序会精准识别出与案件事实最匹配的3-5条，显著提升RAG答案质量。工程上要平衡精度和延迟，通常初筛召回20-100条，重排后取5-10条送入LLM。

16 如何构建RAG系统的GroundTruth数据集?标注方法有哪些?

RAG系统的Ground Truth数据集构建**核心在于建立查询-文档-答案的标准映射关系**。这个映射关系不是简单的问答对,而是一个**三元组的集合**—每条数据包含用户的查询问题、系统应该检索到的相关文档片段、以及基于这些文档的标准答案。很多人以为只要有问题和答案就够了,但RAG系统的特殊性在于它有个检索环节,所以必须明确标注出”哪些文档是相关的,这样才能分别评估检索模块和生成模块的性能。

主要构建方法包括从**真实业务日志中提取用户查询和对应的正确答案**,这是最贴近实际场景的方式,或者基于现有文档库,通过合成数据生成用LLM阅读文档片段后自动生成问答对,再人工审核修正;还可以采用专家标注,让领域专家针对特定文档编写高质量的问答样本。

标注方法上需要关注几个维度。首先是相关性标注,标记哪些文档片段与查询真正相关,通常用0-3分的分级或二分类。这里的分级很有讲究3分代表强相关必须包含完整答案,2分是中等相关只包含部分信息需要

结合其他文档，1分是弱相关仅提供背景知识,0分是不相关。其次是答案质量标注,评估生成答案的准确性、完整性和忠实度。还要做检索准确率标注,记录正确文档是否被召回以及排序位置。

实际操作中建议采用混合策略:先用LLM批量生成候选数据,然后人工抽检20-30%进行质量控制,重点标注那些检索失败或答案质量差的困难样本。对于企业场景,可以持续收集用户反馈(点赞/点踩)作为弱监督信号,逐步扩充数据集。标注工具方面Label Studio或自研的标注平台都能满足需求,关键是要定义清晰的标注规范,确保多人标注的一致性。

17 什么是Hypothetical Document Embeddings (HyDE)?如何提升检索效果?

HyDE (Hypothetical Document Embeddings)是一种**先生成假设性答案再进行检索的技术方案，**本质上是在解决用户提问方式和知识库文档表达方式不匹配的问题。传统RAG直接用问题去检索文档，但问题和文档的语义空间往往存在gap。比如用户问"如何优化数据库性能”，这是典型的口语化表达，但知识库文档的标题可能是”数据库性能调优最佳实践:索引设计与查询优化”，直接检

索效果会比较一般。

HyDE的核心思路很巧妙:**先让LLM根据问题生成一个假设性的回答文档，然后用这个生成的文档去做embedding检索，而不是用原始问题。**具体来说，用户提问后，系统先调用LLM生成一段可能的答案，比如”可以通过索引优化、查询重写、分区表设计、连接池配置来提升数据库性能”。注意这段生成内容即使不完全准确也没关系，它的作用是包含更丰富的领域术语、表达方式和上下文，与知识库中真实文档的语义更接近。然后对这个假设文档做向量化，去向量数据库中检索最相似的真实文档，最后基于检索到的真实文档再生成最终答案。

提升检索效果的关键在于缩小查询与文档的语义鸿沟。**假设文档包含了索引、分区等具体术语，与知识库文档的表达习惯更匹配检索召回率会明显提高。**这种方法特别适合专业领域知识检索场景，能有效解决用户口语化问题与正式文档之间的表达差异问题。

18 什么是Semantic Chunking?与固定长度切分有什么区别?

Semantic Chunking是**基于语义理解进行文本分块的技术**，它会分析文本的语义边界、主题转换和逻辑结构来决定在哪里切分。具体来说，它会计算句子间的语义相似度，当相似度出现明显下降时就认为是一个语义单元的结束，从而在那里进行切分。这样切出来的每个chunk在语义上是完整且连贯的。

与之相比，**固定长度切分**就简单粗暴得多，它只是**按照预设的字符数或token数机械地切分文本**，完全不管语义是否完整。比如你设定500个token一切，它就严格按这个数字来，可能会把一个完整的段落、甚至一句话拦腰斩断。

在实际应用中区别就很明显了。假如你在做RAG系统，用户问”这个产品的退货政策是什么”，如果用固定长度切分，可能退货政策的说明被切成了两半，前半段在chunk A，后半段在chunk B，检索时可能只召回其中一个，导致答案不完整。但用SemanticChunking的话，完整的退货政策说明会被保留在同一个chunk里，检索和生成的质量就会好很多。当然Semantic Chunking的计算成本更高，需要embedding模型来计算语义相似度，处理速度会比固定切分慢，但换来的是更高的信息完整性和检索准确性。

19 关系抽取(Relation Extraction)是什么?怎么识别实体之间的关系?

关系抽取是从文本中识别实体之间语义关系的任务，比如从”乔布斯创立了苹果公司”中抽取出(乔布斯，创始人，苹果公司)这样的**三元组**。如果说实体识别解决的是"找出来”的问题，那关系抽取解决的就是”连起来”的问题。它通常是完成命名实体识别之后的下一步工作，两者配合才能构建出完整的知识图谱。

识别实体关系主要有几种技术路线。基于规则的方法通过模式匹配来识别，像定义”X创立Y"这样的模板，但覆盖面有限。监督学习方法需要标注大量训练数据，早期用CNN、RNN等网络将实体对的上下文编码后分类，现在更多使用BERT等预训练模型，把句子和实体对一起输入，通过[CLS]或实体位置的向量表示来预测关系类型。

实际应用中还会遇到远程监督的做法，利用知识图谱自动生成训练数据，虽然有噪声但能大幅降低标注成本。对于跨句子的关系，需要用文档级关系抽取模型，通过图神经网络或Transformer建模多个句子间的依赖。在金融风控中，关系抽取可以从新闻中识别”公司A收购公司B”、“高管C任职于公司D"等关系构建企业知识图谱;在医疗领域能从病历中抽取”药物–疾病–治疗"关系辅助诊断。

20 什么是混合检索(Hybrid Search)?稀疏检索和稠密检索如何结合?

混合检索是将**稀疏检索**和**稠密检索**两种方法结合起来，充分利用它们各自优势的检索策略。这不是简单的技术叠加，而是让两种检索方式优势互补，既能准确匹配重要词项，又能理解用户真正想表达的意思。

稀疏检索基于传统的词项匹配，比如**BM25算法**，它把文本表示成高维稀疏向量，向量维度对应词表大小，只有出现的词对应位置有值。这种方法在精确匹配关键词时表现很好，比如搜索产品型号、专有名词这类场景。

稠密检索则使用深度学习模型**(如BERT、Sentence–BERT)**将文本编码成低维稠密向量，能够捕捉语义相似性，即使查询词和文档用词不同，只要意思相近也能检索到。

两者结合主要有几种方式:最常见的是加权融合，**分别用稀疏和稠密方法检索后，对两份结果的分数进行加权求和重新排序**，权重可以根据场景调优;还有级联方式，先用计算成本低的稀疏检索召回候选集，再

用稠密检索精排;或者互补召回，两路检索并行执行后合并结果集再统一排序。实际应用中，比如企业知识库搜索，用户输入”如何修改密码”，稀疏检索能精确匹配包含“密码"的文档，稠密检索能找到”重置账户凭证"这类语义相关的内容，混合检索把两者结果综合起来，召回率和准确率都会提升。

Agnet部分：

1.什么是AIAgent?与传统软件程序有什么本质区别?

AI Agent是一个**能够感知环境、自主决策并执行动作的智能系统**，它通过大语言模型作为核心推理引擎，结合工具调用、记忆机制和规划能力来完成复杂任务。

与传统软件程序的本质区别在于执行逻辑的根本不同。传统程序遵循预定义的代码逻辑，输入确定则输出确定，比如一个订单处理系统会按照固定的业务流程执行。**而AI Agent具备自主推理能力，它会根据当前状态动态生成执行计划，同一个输入在不同上下文下可能产生不同的处理策略。**

具体表现在:传统程序的分支逻辑由开发者硬编码，AI Agent的决策分支由模型实时生成;传统程序需要为每种情况编写对应代码，AI Agent能够处理训练时未见过的场景组合;传统程序的错误处理依赖异常捕获机制，AI Agent能够自我反思和调整策略。

举个实际应用场景，传统的客服系统只能按照预设规则匹配用户问题，而AI Agent客服能够理解复杂的用户意图，主动调用查询工具获取信息，甚至在对话过程中调整自己的回答策略。这种从确定性执行到概率性推理的转变，是AI Agent与传统软件程序的核心差异。

2.如何设计代理(Agent)的反思机制?何时触发反思?

Agent的反思机制本质上是让系统评估自身行为质量并调整策略的过程。核心设计思路是在执行流程中嵌入一个元认知层，让Agent审视刚完成的动作、生成的输出或整个任务链路。

设计上通常包含三个要素。首先是**反思的对象**，可以是单次工具调用结果、一轮对话的完整性、或者多步推理的逻辑连贯性。其次是评估标准，比如输出是否回答了用户问题、工具调用参数是否合理、推理步骤有没有矛盾。最后是改进动作，根据反思结论决定是重新执行、修正参数还是切换策略。

触发时机主要看任务复杂度和风险容忍度。对于关键节点比如工具调用失败、生成内容与预期格式不符、用户反馈负面时，应该立即触发。对于多步任务，可以在阶段性完成后做定期反思，检查中间结果是否偏离目标。有些场景会设置质量阈值，当置信度低于某个值时自动触发。

实际应用中，**比如代码生成Agent执行代码后发现报错**，反思机制会分析错误栈、检查生成代码的语法逻辑，然后修正重新生成。或者客服Agent回复后发现用户追问同一问题，就会反思是否理解有误，重新组

织答案。关键是把反思嵌入到执行循环中，而不是事后补救，这样才能形成真正的自我优化能力。

3.Agent对话模块的容错能力怎么设计?用户说话不清楚或有歧义时怎么办?

对话系统的容错能力**核心在于多层识别与主动澄清机制**。当遇到用户输入不清晰时，首先通过置信度阈值判断识别结果的可靠性，低于阈值就触发容错流程。

具体来说，你可以设计意图消歧策略:当识别到多个可能意图时，通过追问来确认，比如用户说”我要查一下”，系统回复”您是要查询订单、余额还是物流信息?"。对于slot填槽不完整的情况，**用自然的方式引导补充**，像用户说”帮我订票”，系统就问"好的，请问您要订哪天从哪里出发的票?

还要建立上下文修正机制，**允许用户随时纠正前面的错误输入**。比如用户说"明天"后又说”不对，是后天”，系统能识别否定词并更新槽位值。**另外模糊匹配和同义词扩展**也是基础能力，用户说"退钱”你得理解成"退款”，说”不要了"要关联到"取消订单”。实在理解不了的时候，直接坦诚告知比胡乱猜测要好，说"抱歉我没理解，您可以换个说法吗"总比答非所问让用户更抓狂来得强。记住容错的目标是让对话继续下去，而不是卡死在某个识别失败的节点上。

4.什么是MCP(Model Context Protocol)协议?它解决了什么问题?

MCP (Model Context Protocol)是Anthropic推出的一个标准化协议，专门用于大型语言模型与外部数据源之间的安全连接和交互。这个**协议的核心目标是让AI模型能够直接访问和操作各种外部系统**，比如数据库、AP接口、文件系统、云服务等，而不需要每次都重新开发专门的集成方案。

MCP解决的主要问题是AI模型与外部世界交互的标准化缺失。在MCP出现之前，每当你想让Claude或其他A模型访问你的数据库、读取特定文件或调用某个API时，都需要开发定制化的连接器和安全验证机制，这个过程既繁琐又容易出错。MCP通过提供统的接口规范和安全框架，让开发者可以快速构建AI模型与各种数据源的桥梁。

举个实际场景:当你需要让AI助手查询公司的客户管理系统、分析销售数据并生成报告时，通过MCP协议，AI模型可以直接连接到CRM数据库，执行查询操作，获取实时数据进行分析。这种连接是安全的、标准化的，避免了传统方式中需要导出数据、手动传递给AI的繁琐流程，真正实现了AI与企业系统的无缝集成。

5.AIAgent的幻觉问题如何解决?有哪些验证策略?

AIAgent的幻觉问题主要通过**多层验证策略来解决**。

首先在**输入端进行事实核查**，通过检索增强生成(RAG)技术将Agent的回答与可信知识库进行实时比对，确保信息来源的准确性。

在**推理过程中**，我建议采用**多模型交叉验证**，让不同的模型对同一问题进行独立判断，通过一致性检查来识别潜在的幻觉内容。

对于**输出验证**，置信度评分机制非常关键，当Agent对某个回答的置信度低于阈值时，应该主动标记不确定性或拒绝回答。同时配合逻辑一致性检查，验证Agent前后回答是否存在矛盾。

在实际应用中，比如智能客服场景，可以设置人工审核触发点，当检测到高风险回答时自动转人工处理。对于金融或医疗等关键领域，还需要建立专业知识图谱验证，确保Agent的回答符合行业标准和法规要求。

另外，持续学习和反馈循环也很重要，通过收集用户纠错和专家标注，不断优化Agent的判断能力。最有效的方法是建立分层验证架构，根据问题的复杂度和风险等级采用不同强度的验证策略，既保证准确性又维持响应效率。

6.MCP(Model Context Protocol)协议的基本架构是怎样的?Client和Server如何交互?

MCP(Model Context Protocol)采用**双向通信架构**，基于J**SON-RPC2.0协议**实现Client-Server交互模式。**Client通常是AI应用或模型接口，Server则提供具体的工具、资源或数据源能力。**

核心交互机制包括三种消息类型:**Request/Response**用于同步调用，Notification用于单向通知，Progress用于长时间操作的进度反馈。Client通过标准化的接口向Server请求能力发现、资源访问和工具调用。

连接建立阶段，Client首先发送initialize请求获取Server支持的能力清单，Server返回可用的tools、resources和prompts列表。

随后Client可以根据需要调用具体功能，比如执行文件读写操作、数据库查询或API调用等。

状态管理方面，MCP支持有状4态和无状态两种模式。Server可以维护会话状态，也可以设计为纯功能性的无状态服务。Client负责管理多个Server连接，协调不同能力的组合使用。

安全机制通过能力声明和权限控制实现，Server明确暴露哪些功能可被调用，Client在调用前需要获得相应权限。这种设计让A|模型能够安全、标准化地扩展外部能力，实现工具调用、知识库访问、系统集成等复杂场景

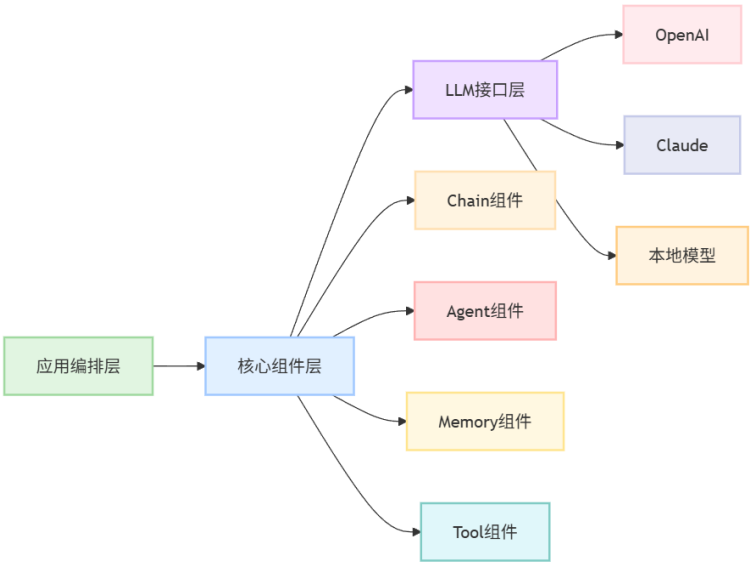
7.LangChain是什么?它解决了LLM应用开发的哪些问题?

LangChain是一个专门为大语言模型(LLM)**应用开发设计的框架**，它通过提供**标准化的组件和抽象层**来简化复杂应用的构建过程。在实际开发中，我们经常遇到几个核心痛点:**提示词管理混乱、多步骤调用逻辑复杂、外部数据源集成困难**，LangChain正是为解决这些问题而生的。

具体来说，LangChain解决了几个关键问题。首先是**链式调用的编排问题**·当你需要让AI先分析用户意图，再查询数据库，最后生成回答时，LangChain的**Chain组件**让这个流程变得可控可维护。其次是记忆管理–通过**Memory组件**，你可以轻松实现对话历史的保存和上下文传递，不用自己处理复杂的状态管理。

另一个重要价值是外部工具集成，比如让AI调用搜索引擎、计算器或数据库查询，**LangChain的Agent和Tool机制提供了标准化的接口。最后是提示词模板化，通过PromptTemplate组件，你可以将提示词逻辑从代码中分离，便于维护和优化。**

总的来说，LangChain让原本需要大量胶水代码的LLM应用开发变成了组装积木式的编程，显著降低了开发复杂度和维护成本。



8.LangChain的核心组件有哪些?各自的作用是什么?

LangChain的核心组件主要包含以下几个关键部分:**Models**是整个框架的基础，负责与各种语言模型(如GPT、Claude)进行交互，提供统一的接口来调用不同厂商的模型。**Prompts**组件专门处理提示工程，包括提示模板的创建、管理和优化，让你能够动态生成结构化的输入内容。

Chains是LangChain的核心抽象，它将多个组件串联起来形成完整的工作流程，比如你可以创建一个”检索–增强–生成”的链条来构建问答系统。**Memory**组件负责管理对话历史和上下文信息，确保多轮对话的连贯性，这在聊天机器人场景中特别重要。

Agents是更高级的组件，它能够根据用户输入动态选择使用哪些工具和执行什么操作，具备一定的推理和决策能力。**Tools**组件则提供了各种外部工具的集成接口，比如搜索引擎、计算器、数据库查询等。

9.LangChain的Agent执行器是如何工作的?

LangChain的Agent执行器是一个**循环推理执行框架（React）**，核心工作机制是思考–行动–观察的迭代过程。

执行器接收用户输入后，Agent会分析当前状态并决定下一步行动，这个决策过程依赖于内置的推理逻辑和预定义的工具集。

Agent可以选择使用特定工具(如搜索引擎、AP调用、数据库查询)或直接给出最终答案。

工具执行完成后，Agent会观察返回结果，然后基于新信息重新评估情况，决定是否需要使用其他工具或已经可以回答用户问题。

这个循环会持续进行，直到Agent认为已经收集到足够信息给出满意答案，或者达到预设的最大迭代次数。

比如用户问”今天北京天气如何，适合穿什么衣服”，Agent会先调用天气AP获取北京当前天气数据，然后基于温度、湿度、风力等信息推理出合适的着装建议。整个过程中，执行器负责协调Agent的推理逻辑、工具调用和结果整合，确保每个步骤都能正确执行并传递给下一个环节。

执行器还会处理异常情况，比如工具调用失败、超时或返回错误格式数据时的容错机制，保证整个推理链路的稳定性。

10.LangGraph是什么?它与LangChain有什么关系?

LangGraph**是LangChain生态系统中的一个专门模块**，用于构建具**有状态管理和循环控制的复杂AI应用程序**。它**基于图结构来编排多个处理节点，每个节点可以是LLM调用、工具执行或数据处理步骤**。

与传统的LangChain链式调用不同，LangGraph**允许你创建非线性的执行流程**。比如在构建一个代码审查助手时，LangGraph可以让程序在"分析代码"—"发现问题"—"生成修复建议"—"验证修复”之间循环执行，直到代码质量达标才结束流程。而普通的LangChain只能按预设顺序执行。

LangGraph的核心优势在于**状态持久化和条件分支**。它**维护一个全局状态对象**，各节点可以读取和修改状态，支持基于状态内容进行路径选择。这让你能够构建真正的AIAgent，而不仅仅是简单的问答系统。

在实际应用中，**LangGraph特别适合构建需要多步推理、错误重试、动态决策的场景**，比如自动化客服系统需要在"理解问题”—”查询知识库"—"生成回答"—"确认用户满意度”之间灵活跳转。它本质上是LangChain从简单链式调用向复杂图状态机的进化，为构建生产级AI应用提供了更强大的控制能力。

11 什么是对话历史窗口?保留多少轮对话合适?

对话历史窗口是指在多轮对话中**保留的历史消息数量**，用于为大模型提供上下文信息。由于模型的token限制和成本考虑，我们不能无限制地保留所有历史对话，需要通过**窗口机制来截取最近的若干轮对话**。这就像人的短期工作记忆，只能记住最近发生的几件事，时间久远的细节就会自然遗忘。

保留多少轮其实没有固定答案，这取决于你的具体场景。一般来说3-10轮是比较常见的选择。如果是客服场景，可能5轮左右就够了，因为用户的问题往往比较聚焦，”我的订单为什么还没发货”怎么申请退款"这类咨询基本几轮就能解决。但如果是技术咨询或复杂的任务型对话，可能需要保留10轮甚至更多，因为后续问题经常依赖前面的大量上下文，比如用户先描述系统架构，然后问数据库设计，接着讨论性能优化，这些环节环环相扣。

实际应用中你需要权衡几个因素:**token成本会随窗口增大线性增**长，窗口每扩大一倍，每次调用的费用就跟着翻倍;响应延迟也会变长，模型需要对输入的所有token做Attention计算，输入越长计算量越大;但窗口太小又会导致上下文丢失，模型回答变得答非所问，用户明明刚问过的事情，下一轮就"失忆"了。比较实用的做法是采用滑动窗口策略，保留最近N轮对话，或者使用智能截断优先保留system提示词和最近几轮，对中间的历史做摘要压缩。有些场景还会根据对话的重要性动态调整窗口大小，比如检测到话题切换时就清空历史，重新开始计数。关键是要在你的实际业务中测试和调优，找到成本和效果的最佳平衡点。

12 如何使用LangChain构建多轮对话系统?

使用LangChain构建多轮对话系统的核心在于对话记忆管理和上下文维护。你需要通过conversationchain 配合合适的Memory组件来实现对话状态的持续跟踪。

最基础的实现是使用conversationBufferMemory，它会保存完整的对话历史。当你需要控制上下文长度时，可以选择ConversationBufferwindowemory来限制保留的轮次，或者使用ConversationsummaryMemory 对历史对话进行摘要压缩。对于更复杂的场景，ConversationSummaryBufferMemory能够结合摘要和缓冲机制。

在实际应用中，比如客服系统，你可以通过 Conversationchain.from_l1m()初始化对话链，然后在每次用户输入时调用predict()方法，LangChain会自动将历史对话作为prompt的一部分传递给大语言模型。关键是要正确配置memory的memory_key 和input_key参数，确保对话历史能够正确注入到prompt模板中。

如果你需要更精细的控制，可以自定义conversationchain的prompt模板，明确指定对话历史的格式和位置。对于需要持久化的场景，可以将Memory组件与数据库结合，实现跨会话的对话状态保存。整个过程中，LangChain会透明地处理对话历史的序列化和反序列化工作。

13.AIAgent的记忆机制如何设计?短期记忆和长期记忆的区别?

AI Agent的记忆机制是系统维持状态连续性和知识积累的核心组件。**短期记忆**主要存储当前会话或任务执行过程中的临时信息，包括用户输入历史、中间计算结果、上下文状态等，通常采用有**限长度的缓冲区或滑动窗口实现**，当超出容量限制时会自动清除最早的记录。

长期记忆则负责持久化存储重要知识和经验，包括用户偏好、历史交互模式、学习到的规则等，常通过**向量数据库、知识图谱或结构化存储实现。**

在技术实现上，短期记忆直接影响当前推理过程，需要快速读写访问，而长期记忆需要检索机制来激活相关信息。比如在客服Agent中，短期记忆保存本次对话的问题描述和解决步骤，长期记忆则存储用户的产品使用历史和问题解决偏好。两者的关键区别在于生命周期、存储容量和访问模式:短期记忆具有会话级生命周期、有限容量和顺序访问特性;长期记忆具有持久化生命周期、大容量存储和基于相似度的检索访问特性。设计时需要考虑记忆更新策略、检索效率和存储成本的平衡。

14 如何评估AIAgent的性能?有哪些关键指标?

AIAgent性能评估需要从多个维度综合考量。**任务完成率**是最核心指标，衡量Agent在特定任务中的成功执行比例，比如客服Agent正确解决用户问题的百分比。响应时间直接影响用户体验，包括首次响应延迟和整个对话流程的耗时。

准确性指标涵盖理解准确性和执行准确性，前者评估Agent对用户意图的理解程度，后者衡量执行动作的正确性。在代码生成Agent中，这体现为代码的语法正确性和功能实现度。鲁棒性测试Agent面对异常输入、边界条件或攻击性prompt时的稳定表现。资源效率包括计算资源消耗、内存使用和AP调用频次，这些直接影响部署成本。

可解释性评估Agent决策过程的透明度，特别是在金融、医疗等关键领域。用户满意度通过实际用户反馈收集，包括交互体验和结果质量评价。评估方法上，需要构建标准化测试集进行基准测试，同时进行A/B测试对比不同版本性能。长期监控运行指标变化趋势，识别性能退化。针对不同应用场景，这些指标的权重会有所调整，比如实时交易系统更注重响应时间，而内容创作Agent更关注创造性和准确性的平衡

15 Agent的Self-Refinement是如何实现的?需要哪些关键组件?

Agent的Self-Refinement本质是**让Agent通过自我反馈循环来迭代优化输出结果**。你可以把它类比成我们写文档的过程——先快速写个初稿，然后回过头审视哪里逻辑不通、哪里表达不清，再针对性修改。实现机制是**Agent先生成初始响应，然后对这个响应进行criti 评估，发现问题后重新生成改进版本，这个过程可以多轮迭代直到满足质量标准。**

关键组件主要包括三个部分:**Generator生成器**负责产生初始输出，**Critic评估器**对输出进行批判性分析找出缺陷，**Refiner改进器**根据评估反馈重新生成优化后的结果。这三个角色可以是同一个大模型切换不同

prompt实现，成本很可控。整个过程就是个闭环生成结果后立刻自我批判，发现具体问题就带着改进建议重新生成，这个循环可以跑多轮直到质量达标。

实现时需要设计好评估维度，比如代码生成场景会检查语法正确性、逻辑完整性、边界条件处理等;文本生成则关注事实准确性逻辑连贯性、表达清晰度。终止条件也很关键，可以设置最大迭代次数，或者当Critic评分达到阈值时停止。核心价值在于将人类的迭代思考过程内化到Agent的执行流程中，相比传统的单次生成，这种机制能显著提升复杂任务的成功率。

16 LangGraph的人机交互功能如何实现?

LangGraph的人机交互功能主要通过**interrupt机制**和**人工节点**来实现。你可以在图中设置特定的节点作为中断点，当执行流到达这些节点时，系统会暂停执行并等待人工干预。

核心实现方式是使用**interrupt_before或interrupt_after参数**来标记需要人工介入的节点。比如在一个文档审核流程中，你可以在AI生成初稿后设置中断点，让人工审核员检查内容质量。审核员可以直接修改生成的内容，或者提供反馈指令，然后通过**invoke 或stream 方法恢复执行**。

状态持久化是关键特性，**LangGraph会保存中断时的完整状态，包括所有变量和执行上下文。你可以使用内存存储或数据库来持久化这些状态，确保即使系统重启也能从中断点继续执行。**

实际应用中，这种机制特别适用于需要人工判断的场景，比如法律文档生成中的合规性检查、客服对话中的复杂问题升级、或者代码生成后的安全审查。人工干预可以是简单的确认操作，也可以是复杂的内容编辑和参数调整，系统会根据人工输入更新状态图并继续后续的自动化流程。

17 AIAgent在执行过程中可能遇到哪些错误?如何处理?

AI Agent在执行过程中常遇到的错误主要包括网络连接异常、AP调用失败、输入数据格式错误和推理逻辑偏差等情况。

当网络连接中断或外部服务不可用时，Agent需要实现重试机制和降级策略，比如在调用第三方API失败时自动切换到备用服务或使用本地缓存数据。对于输入数据异常，应该在Agent接收输入时进行严格的数据验证和清洗，确保数据类型、格式和范围符合预期。

在推理过程中，Agent可能因为上下文理解偏差或知识库信息不准确导致错误决策。这需要通过多轮验证机制来处理，让Agent在执行关键操作前进行结果确认，并设置置信度阈值，当不确定性过高时主动寻求人工介入。

对于工具调用失败的情况，Agent应该具备异常捕获能力，记录错误详情并尝试替代方案。比如在文件操作失败时检查权限设置，在数据库查询异常时调整查询语句。

最重要的是建立完整的日志记录体系，记录Agent的每个决策节点和执行结果，便于问题追踪和模型优化。同时设计熔断机制，防止错误在多Agent系统中传播扩散。

18 AIAgent开发中常见的调试问题有哪些?如何解决?

在AI Agent开发中，最常遇到的调试问题集中在几个核心环节。**推理链路断裂**是最棘手的问题，Agent在多轮对话中会丢失上下了或逻辑跳跃，这时需要增加详细的日志记录每个推理步骤，并检查prompt设计是否过于复杂。**工具调用失败也很常见**，比如AP时、参数格式错误或权限问题，解决方法是添加重试机制和异常捕获，同时对工具返回结果进行格式验证。

幻觉问题在知识检索场景中尤其突出，Agent会编造不存在的信息，需要通过RAG向量检索增强、设置置信度阈值，以及在prom中明确要求”不知道就说不知道”来缓解。性能瓶颈通常出现在复杂任务规划中，Agent会陷入无限循环或执行效率低下，可以通过置最大执行步数、优化planning算法，以及使用更小粒度的子任务分解来解决。

调试时，结构化日志是关键工具，记录每个decision point的输入输出和中间状态。同时建议使用deterministic的测试用例，固随机种子确保问题可复现。对于复杂Agent,分模块单独测试tool calling、memory management、planning等组件，再进行成测试，这样能快速定位问题根源。

19 如何用LangChain实现一个简单的问答机器人Agent?

用LangChain实现问答机器人主要分为两种方式:基础链式调用和Agent模式。

基础问答机器人使用**conversationchain** 即可实现。你需要先初始化LLM模型(如OpenAI的GPT)，然后创建对话链，添加记忆组件来维持上下文。代码核心就是 **Conversationchain(11n=11m, memory=ConversationBuferremory())**，这样机器人就能记住之前的对话内容。

Agent模式更强大，它能调用外部工具来回答问题。你需要定义工具集(Tools)，比如搜索工具、计算器、数据库查询等，然后使用initialize-agent 创建Agent。Agent会自动判断需要调用哪个工具来回答用户问题。例如用户问“今天北京天气如何”，Agent会自动调用天气查询工具。

实际应用中，我建议结合RAG模式(检索增强生成)。先用vectorStore存储知识库文档，当用户提问时，系统先检索相关文档片段，再让LLM基于这些内容回答。这种方式特别适合企业内部知识问答系统。

关键组件包括:Prompt模板定义机器人角色和回答风格，Memory组件维持对话状态，Output Parser格式化回答内容。整个流程就是接收用户输入一检索相关信息一调用LLM生成回答一返回结果，LangChain把这些步骤封装得很简洁。

20 什么是ReAct模式?它如何提升AIAgent的推理能力?

ReAct模式是**Reasoning+Acting**的缩写，这是一种让AI Agent交替进行推理和行动的框架模式。传统的AI Agent要么只能思考不能行动，要么只能执行预设操作，而ReAct打破了这种局限。

具体来说，ReAct让Agent在每个步骤中都经历**“思考–行动–观察”的循环**。Agent首先分析当前情况并推理下一步应该做什么，然后执行具体的行动(比如调用API、搜索信息、运行代码)，最后观察行动结果，再基于新的信息继续推理。这个过程会持续进行直到完成任务。

这种模式显著提升了推理能力，主要体现在几个方面:首先是动态信息获取，Agent可以根据推理需要主动获取外部信息，而不是仅依赖初始输入;其次是错误自我纠正，当行动结果不符合预期时，Agent能够重新推理并调整策略;最后是复杂任务分解，Agent可以将大任务拆分成多个推理–行动步骤。

比如在数据分析场景中，Agent会先推理需要什么数据，然后查询数据库，观察查询结果后再推理下一步的分析方法，最终逐步完成复杂的分析任务。这种方式让AIAgent具备了更接近人类的问题解决能力。

21 如何调试LangGraph工作流?有哪些调试工具?

LangGraph工作流调试主要通过内置日志系统和可视化工具来实现。你可以在每个节点中添加system.out.println()语句或使用Logger来跟踪数据流转，这样能清楚看到状态在节点间的传递过程。

LangSmith是最核心的调试工具，它能提供完整的执行轨迹可视化，包括每个节点的输入输出、执行时间和错误信息。配置方式是设置LANGcHAIN_TRACING_V2=true 环境变量，然后在LangSmith界面查看详细的执行图谱。

对于实时调试，你可以使用断点调试，在关键节点设置断点来暂停执行，检查当前状态内容。另外，graph.getGraph().drawMermaid()方法能生成工作流的结构图，帮助你理解节点连接关系。

在处理复杂工作流时，建议使用**状态检查器功能**，通过 graph.invoke(input，debug=true)启用详细输出模式。如果遇到条件路由问题，可以在路由函数中添加日志记录，明确显示路由决策的依据。

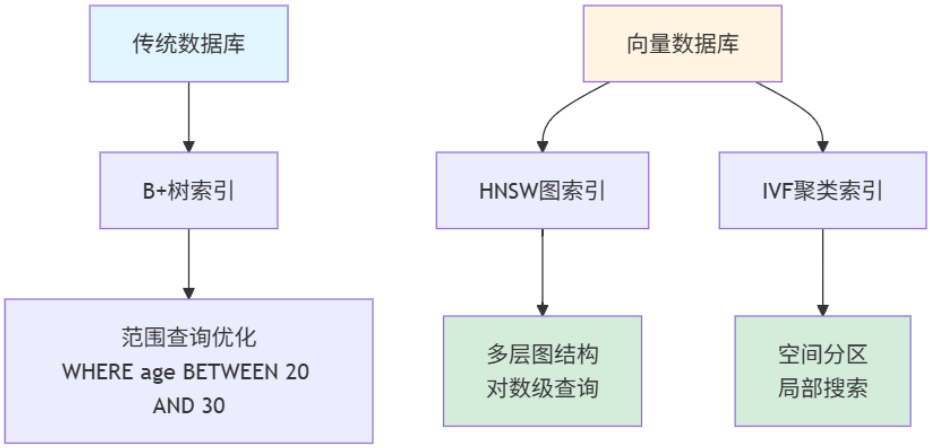
性能调试方面，LangSmith的时间轴视图能帮你识别瓶颈节点，而内存使用情况可以通过性能监控库来监控。这些工具组合使用基本能覆盖LangGraph工作流的所有调试需求。

22 Agent的长期记忆如何持久化?向量数据库的选择标准是什么?

Agent的长期记忆持久化**主要通过向量数据库存储embedding实现**。具体流程是将对话历史、用户偏好、知识片段等文本经过embedding模型转换为向量，存入向量数据库，后续通过语义检索召回相关记忆。这个过程的核心在于解决一个关键问题:当用户有成千上万条历史记录时，怎么快速找到语义相关的那几条?传统数据库擅长精确匹配，但Agent需要语义理解。比如用户问"上次那个便宜的推荐”，系统要能关联到历史对话里的价格敏感信息，这就需要向量相似度检索而不是SQL的模糊查询。

选择向量数据库时需要关注几个核心维度。性能层面看查询延迟和吞吐量，生产环境通常要求毫秒级响应，像Milvus和Qdrant在百万级数据下能保持较好性能。索引算法很关键，**HNSW适合高维稠密向量的快速检索**，IVF系列适合大规模场景。过滤能力决定能否结合元数据筛选，比如按时间范围或用户ID过滤记忆，这在多租户场景必不可少。还要考**虑混合检索支持**，很多场景需要向量检索和关键词检索结合，像Weaviate原生支持这种能力。数据规模也影响选型，小规模可用Chroma这类嵌入式方案，企业级应用更适合Pinecone或自建Milvus集群。最后是生态适配性，要看是否有成熟的LangChain或LlamaIndex集成，能否方便接入现有技术栈。

实际应用中，个人助理类Agent可能只需Chroma配SQLite存几万条记忆，但客服系统可能需要Milvus集群存储千万级用户交互历史，并支持实时更新和分布式部署。选型的本质是在性能、成本、复杂度之间找平衡点。



23 Agent的成本如何计算?如何在效果和成本间找到最优平衡点?

Agent的成本主要由API调用费用构成，包括输入token和输出token的消耗。计算时需要考虑单次对话的prompt长度、模型推理生成的响应长度，以及Agent的迭代次数——也就是规划–执行–反思的循环轮数。如果Agent需要调用外部工具或进行RAG检索，还要加上这些服务的成本。

成本优化的关键在于减少不必要的token消耗和迭代。你可以通过精简system prompt、优化few–shot示例数量来控制输入成本;设置明确的停止条件避免Agent陷入过多轮次的自我循环;对于复杂任务，用较便宜的模型做初筛或简单步骤，只在关键决策节点使用GPT–4这类强模型。

找平衡点需要建立评估体系:先用小规模数据测试不同配置下的效果指标(准确率、任务完成率等)和成本数据，绘制成本–效果曲线。比如客服场景中，你可能发现GPT–3.5处理80%的常见问题就够了，只有复杂投诉才路由到GPT–4，这样既保证体验又大幅降本。实践中还要设置预算上限和监控告警，实时追踪每个Agent实例的token消耗，防止异常情况导致成本失控。本质上这是个工程问题，需要持续监控数据并迭代优化策略。

24 MCP(Model Context Protocol)协议支持哪些传输方式?各有什么特点?

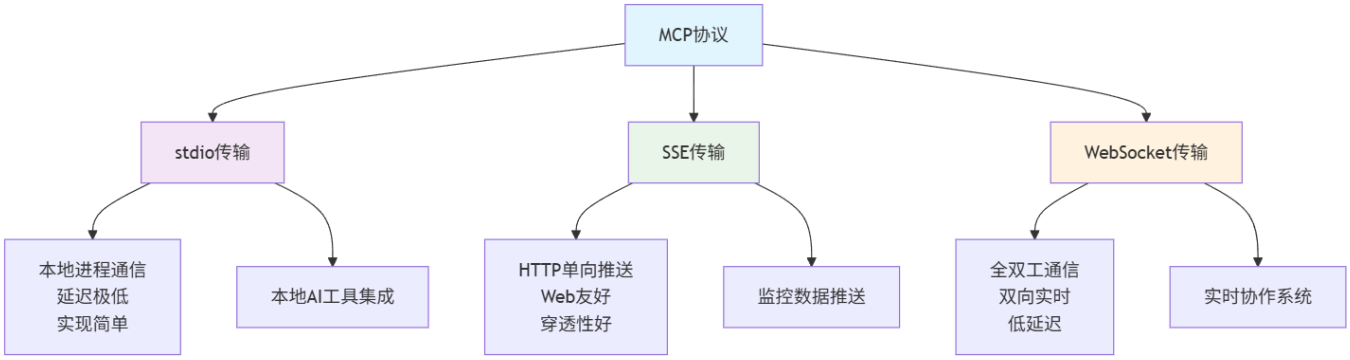
MCP协议主要支持三种传输方式，每种都针对不同的应用场景进行了优化。

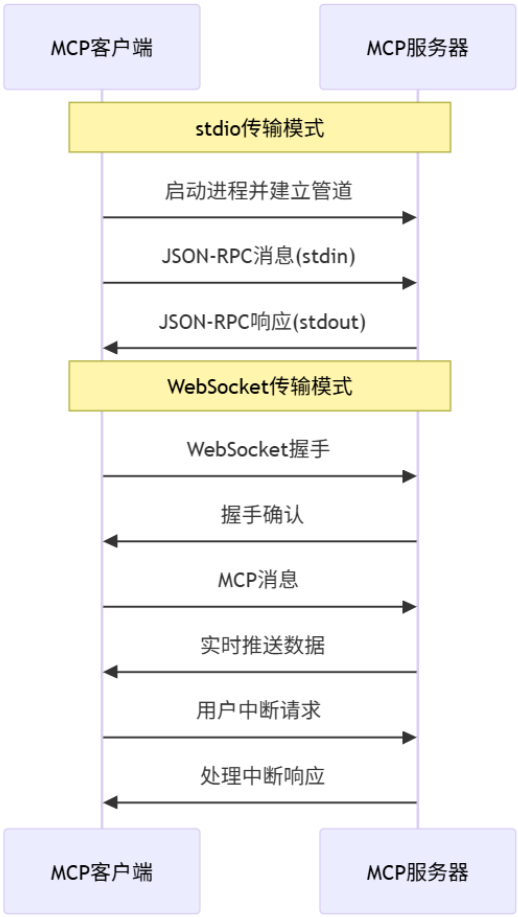
stdio传输是最常用的方式，通过标准输入输出流进行通信。这种方式实现简单，特别适合本地进程间通信，比如AI客户端与本地工具服务器的交互。由于直接使用进程的stdin/stdout，延迟极低，但仅限于同一主机内的通信。

SSE(Server-Sent Events)传输基于HTTP协议，采用单向流式推送机制。服务器可以持续向客户端推送数据，而客户端通过HTTP请求发送消息。这种方式特别适合需要实时数据推送的场景，比如监控系统或实时状态更新，同时具备良好的防火墙穿透能力和Web兼容性。

Websocket传输提供全双工通信能力，支持客户端和服务器之间的双向实时数据交换。相比SSE，WebSocket的开销更低，延迟更小，特别适合需要频繁双向交互的场景，比如实时协作工具或交互式数据处理。

在实际应用中，stdio主要用于本地集成，SSE适合云端服务推送场景，而WebSocket则是分布式系统实时交互的首选。选择传输方式时需要考虑部署环境、网络条件和交互模式这三个关键因素。





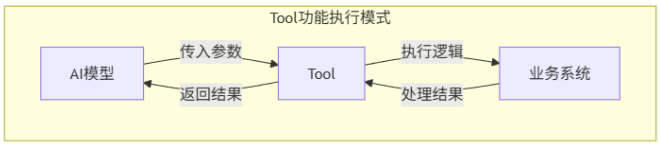
25 MCP(Model Context Protocol)协议中的Resource和Tool有什么区别?各自的使用场景?

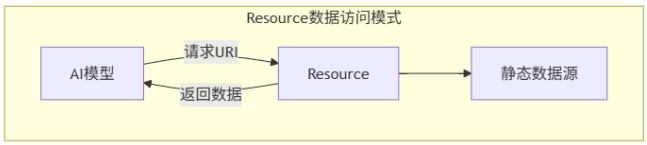
Resource和Tool在**MCP协议中本质上是两种完全不同的数据交互机制**。这个问题的核心在于理解静态数据访问和动态功能执行的区别。

Resource是**静态数据源**，它提供可读取的信息内容，比如文件、数据库记录、API响应等。当你需要获取某个特定的数据时，你通过Resource URI来访问，系统返回的是数据本身。典型场景包括读取配置文件、获取用户档案、访问文档内容等。Resource的交互是单向的读取操作，你请求什么URI，就返回对应的数据内容。

Tool则是**可执行的功能单元**，它不是数据而是动作。当你调用Too时，实际上是在执行某个操作或计算过程，然后获得执行结果。比如发送邮件、执行数据库查询、调用计算函数、触发业务流程等。To0l的交互是双向的执行操作，你传入参数，To0l执行逻辑后返回执行结果。

简单说，Resource回答"这里有什么数据"，Tool回答"我能帮你做什么事”。当你需要获取已存在的信息时用Resource，当你需要执行某个操作来产生结果时用Tool。在实际应用中，一个完整的MCP服务往往同时提供Resource来暴露数据访问能力，提供To0l来暴露功能执行能力。





26 什么是多AI Agent系统(Multi-Agent System)?AI Agent之间如何协作?

多AIAgent系统是由多个具备感知、决策和行动能力的自主智能体组成的分布式系统，**每个Agent都能独立处理任务并与其他Agent交互协作。相比单一AI系统，多Agent系统通过分工合作能够处理更复杂的问题。**

Agent之间的协作主要通过**消息传递、任务分解**和角色分工实现。在消息传递机制中，Agent通过定义好的通信协议交换信息，比如在自动驾驶系统中，路况监控Agent会将交通数据传递给路径规划Agent。任务分解是将复杂问题拆分成子任务，分配给不同专长的Agent处理，就像软件开发中的代码生成Agent、测试Agent和文档生成Agent各司其职。

协作模式包括**协调式协作和竞争式协作**。协调式如智能客服系统中，意图识别Agent、知识检索Agent和回复生成Agent按流程协同工作;竞争式如金融交易系统中，多个策略Agent同时分析市场，系统选择最优结果。

关键技术挑战在于冲突解决、负载均衡和一致性保证。当多个Agent给出不同建议时，需要仲裁机制;当某个Agent过载时，需要动态调度;当处理同一数据时，需要保证状态同步。这些机制确保整个系统能够高效稳定地运行。

27 如何在LangGraph中定义状态?状态管理的最佳实践?

在LangGraph中，状态是通过**TypedDict**或自定义类来定义的，它作为图中所有节点间共享的数据结构。你需要在创建StateGraph时明确指定状态类型，比如stateGraph(Mystate)，其中MyState包含了所有需要在节点间传递的数据字段。

状态管理的核心在于保持不可变性和明确的数据流向。每个节点函数接收当前状态作为输入，返回状态的更新部分，LangGraph会自动合并这些更新。你应该避免在节点内直接修改状态对象，而是返回包含变更字段的字典。

最佳实践方面，首先要合理设计状态结构，将相关数据分组，避免过度扁平化。对于复杂的对话系统，可以将用户输入、对话历史、中间结果分别作为独立字段管理。其次，利用状态的历史追踪能力，LangGraph支持checkpointing，**这让你能够回溯和恢复到任意执行点**。

在处理大型数据时，考虑使用引用传递而非直接存储，比如将大文档的ID存储在状态中，实际数据存储在外部系统。同时，要注意状态的序列化兼容性，确保所有字段都能被正确序列化和反序列化，这对于分布式执行和持久化至关重要。合理使用条件边来控制状态流转，避免无效的状态传播。

28 Graph of Thoughts (GoT)如何表示复杂推理过程?有什么优势?

Graph of Thoughts (GoT)通过**有向无环图(DAG)**结构来表示推理过程，其中节点代表中间思维状态，边代表思维之间的依赖关系。这种表示方式允许多条推理路径并行存在，思维可以分叉、合并、回溯，形成网状的推理网络。

具体来说，**GoT会将复杂问题分解后**，让不同的推理分支独立探索，再通过**聚合节点**整合多个分支的结论。比如在代码调试场景中，可以同时探索"语法错误"和"逻辑错误"两个方向，各自推进后再汇总成最终诊断结果。GoT的核心优势在于突破了链式推理的线性限制。传统的Chain-of-Thought只能按固定顺序逐步推理，而GoT支持思维的任意组合和重用。当某个推理分支遇到瓶颈时，可以从图中其他节点寻找突破口，甚至将之前的中间结果用于新的推理路径。

这种灵活的拓扑结构特别适合处理需要多角度验证的任务，比如数学证明需要同时验证必要性和充分性，或者在策略规划中需要评估多个并行方案。另外，图结构天然支持显式的推理状态管理，可以清晰追溯每个结论的推导依据，便于调试和优化推理过程。相比树状结构的Tree-of-Thoughts，GoT通过允许节点间的交叉连接，避免了重复计算相同的中间状态，提升了推理效率。

29 如何在LangGraph中实现循环和迭代?

在LangGraph中实现循环和迭代主要通过**条件边(conditional edges)**和**状态管理**来完成。与传统编程的for/while循环有本质区别，LangGraph的循环是基于图的状态流转来实现的。你需要在图的**节点中定义判断逻辑，当满足特定条件时让执行流程回到之前的节点，形成循环结构。**

核心实现方式是使用add_conditiona1-edges方法，在节点执行后根据状态值决定下一步走向。比如在RAG系统中，你可以设计一个检索-验证-重试的循环:检索节点获取文档后，验证节点判断结果质量，如果不满足要求就通过条件边回到检索节点重新执行，直到达到质量标准或达到最大迭代次数。

状态控制是关键所在，你需要在图的状态中维护循环计数器和终止条件。例如设置retry-count字段记录迭代次数，在条件函数中检查这个值来决定是继续循环还是退出。同时要确保每次迭代都更新相关状态，避免无限循环。实际应用场景包括多轮对话中的意图澄清、AP调用失败重试、多步推理中的结果验证等。记住要设置合理的最大迭代限制和明确的退出条件，确保图的执行能够正常终止。

30 LangGraph的编译和执行过程是怎样的?

LangGraph的编译过程主要是**将图定义转换为可执行的状态机**。当你调用graph.compile()时，LangGraph会解**析节点间的连接关系，验证图结构的合法性，**然后生成一个优化的执行计划。编译器会检查条件边的逻辑、循环依赖，并为每个节点分配执行顺序。

执行阶段采用**基于状态的流式处理模式**。编译后的图从起始节点开始，按照预定义的边关系传递状态对象。每个节点接收当前状态，执**行自己的逻辑(比如LLM调用、工具使用、数据处理)，然后更新状态并传递给下一个节点**。条件边会根据当前状态动态决定下一步的执行路径，这使得复杂的分支逻辑成为可能。

整个执行过程是增量式的状态更新，每次节点执行都会产生状态快照，支持中断和恢复。比如在构建AIAgent时，可能有”**分析用户意图一选择工具一执行任务一生成回复**”这样的流程，每个步骤都会更新对话状态，如果某个环节需要人工干预，可以暂停执行并从断点继续。

LangGraph的这种设计让复杂的AI工作流变得可控和可观测，特别适合需要多步推理和工具调用的场景。

31 LangChain的缓存机制如何工作?如何优化性能?

LangChain的缓存机制主要通过**缓存层**拦截重复的LLM调用来工作。当你发起相同的提示词请求时，系统会先检查缓存中是否存在对应的响应，如果命中则直接返回缓存结果，避免重复的AP调用和计算开销。这种设计解决了LLM应用中的两个关键痛点——AP调用成本和响应延迟。

LangChain支持多种缓存后端，包括**内存缓存**、**SQLite**、**Redis和**语义缓存。内存缓存适合开发测试，Redis适合生产环境的分布式场景。特别值得注意的是语义缓存，它不仅匹配完全相同的输入，还能识别语义相似的查询并返回相关缓存结果。比如”手机续航"和"电池使用时间"其实是在问同一件事，语义缓存能够智能识别这种关联。

性能优化方面，需要根据应用场景选择合适的缓存策略。对于高频重复查询的场景，启用缓存能显著降低延迟和成本。合理设置TTL过期时间很关键，既要保证数据新鲜度，又要最大化缓存命中率。在处理大量用户请求的生产环境中，使用Redis集群可以提供更好的并发性能

32 LangGraph的条件边(Conditional Edge) 如何使用?

LangGraph中的条件边(Conditional Edge) 用于根据节点执行结果动态决定下一个执行路径。通过add_conditional_edges()方法实现，该方法接收三个关键参数:源节点、条件函数和目标节点映射。

条件函数接收当前状态作为参数，返回字符串键值来指定下一个节点。例如在聊天机器人场景中，可以根据用户输入类型路由到不同处理节点

33 LangChain的Memory组件如何工作?有哪些Memory类型?

LangChain的Memory组件通过存储和管理对话历史来为AI应用提供上下文连续性。它的核心工作机制是在每次交互时保存用户输入和AI输出，并在后续对话中将相关历史信息注入到prompt中，让模型能够理解对话的上下文关系。

Memory组件主要包含几种类型:**ConversationBufferMemory**是最基础的，直接存储完整的对话历史;**ConversationBufferWindowMemory**只保留最近的K轮对话，避免上下文过长;**ConversationSummaryMemory**会定期总结对话内容，用摘要替代原始对话来节省token;**ConversationSummaryBufferMemory**结合了窗口和摘要机制，对较早的对话进行总结，保留最近的完整对话。

还有**VectorstoreRetrieverMemory**基于向量检索，能够从大量历史对话中检索最相关的片段，特别适合长期记忆场景。

EntityMemory专门提取和存储对话中的实体信息，比如在客服系统中记住用户的姓名、订单号等关键信息。

在实际应用中，你可以根据场景选择合适的Memory类型。比如构建聊天机器人时用BufferWindowMemory控制成本，做知识问答系统时用VectorstoreRetrieverMemory检索相关历史，做个人助手时用EntityMemory记住用户偏好。Memory组件通过save_context()方法保存对话，通过load_memory_variables()方法加载历史信息到当前会话中。

34 MCP(Model Context Protocol)协议的安全机制包括哪些?如何保证通信安全?

MCP协议的安全机制主要围绕认**证授权**、**传输加密和访问控制**三个核心维度构建。在认证层面，MCP支持基于token的身份验证机制，客户端需要提供有效的API密钥或OAuth令牌才能建立连接，服务端会验证这些凭证的合法性和权限范围。

传输安全通过强制使用TLS/SSL加密实现，所有的模型调用请求、响应数据都经过端到端加密传输，防止中间人攻击和数据窃听。协议还支持证书绑定验证，确保客户端连接到正确的服务端点。

在**访问控制方面**，MCP实现了细粒度的权限管理机制，可以针对不同的模型、功能模块设置不同的访问权限。比如某个应用可能只被授权访问文本生成模型，而无法调用图像生成或敏感数据处理功能。协议还包含请求频率限制和资源配额管理，防止恶意客户端进行DoS攻击或资源滥用。

另外，**MCP还集成了审计日志机制**，记录所有的AP调用、权限变更等关键操作，便于安全监控和事后追溯。在实际部署中，比如企业级AI应用场景，这些安全机制能够有效保护模型服务不被未授权访问，同时确保数据传输过程的机密性和完整性。

35 如何在LangChain中实现流式输出?有什么应用场景?

在LangChain中实现流式输出主要通过streaming参数和回调机制来完成。你可以在初始化LLM时设置streaming=True，然后使用streamingstdoutcal1backHandler 或自定义回调处理器来实时处理token输出。具体实现时，创建LLM实例时传入callbacks=[streamingstdoutcal1backHandler()]，这样模型生成的每个token都会立即输出而不是等待完整响应。

对于更复杂的场景，你还可以继承 Basecal1backHandler 自定义回调类，重写on_11m_new_token 方法来处理每个新生成的token,比如实时更新UI界面或者将输出流式传输到前端。在使用Chain时，同样可以通过callbacks参数启用流式输出。

主要应用场景包括:聊天机器人界面需要逐字显示回复提升用户体验，长文本生成任务让用户看到实时进度避免等待焦虑，实时代码生成场景让开发者即时查看生成结果，以及需要低延迟响应的交互式应用。流式输出特别适合处理大模型的长响应时间问题，通过渐进式内容展示显著改善用户感知的响应速度。在生产环境中，结合WebSocket或Server-Sent Events可以构建真正的实时交互体验。

36 如何在LangChain中实现RAG(检索增强生成)?

在LangChain中实现RAG需要将文档检索与语言模型生成能力结合。首先你需要准备向量数据库，使用**Document Loaders**加载文档，通过**Text Splitters**将文档切分成合适的**chunk**，然后用**Embeddings**将文本转换为向量并存储到**Vector Stores**中，常用的有Chroma、 FAISS或Pinecone.

核心实现是构建检索链，你可以使用 RetrievaloA 或 ConversationalRetrievalchain。检索过程中，用户查询首先被转换为向量然后在向量数据库中找到最相似的文档片段，这些片段作为上下文传递给LM。LangChain提供了 **vectorstoreRetriever**来处理这个检索过程，你可以配置返回的文档数量和相似度阈值。

生成阶段，检索到的文档会被格式化成**prompt**模板的一部分，与用户问题一起发送给语言模型。**PromptTemplate**在这里起关键作用，它定义了如何将检索内容与问题组合。你还可以使用**RetrievalqAwithsourceschain**来追踪答案来源，或者用ConversationalRetrievalchain 实现多轮对话的RAG。

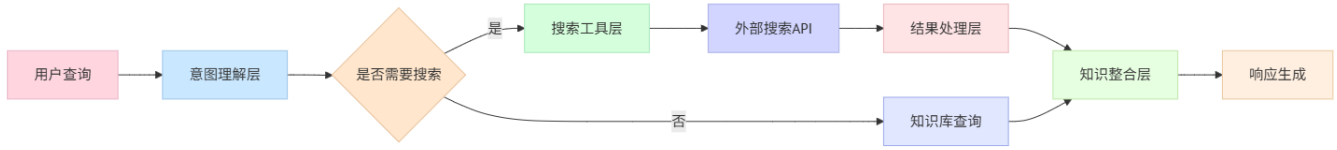
整个流程是:查询一向量化一检索相关文档一构建增强prompt一LM生成答案。LangChain的模块化设计让你可以灵活替换各个组件，比如切换不同的embedding模型或向量数据库，适应具体的业务场景需求。

37 如何给AIAgent添加网络搜索功能?需要注意什么?

给AI Agent添加网络搜索功能主要通过集成搜索API实现。你可以选择Google Search API、Bing Search API或DuckDuckGo等服务，将搜索接口封装为Agent的工具函数。实现时需要定义搜索工具的schema，包括查询参数、返回格式等，然后在Agent的工具链中注册这个搜索功能。

遇到这个问题时，面试官其实想考察你对AIAgent架构和系统集成的理解深度。可以从技术实现和工程考虑两个维度来回答，展现系统性思维。搜索功能不仅仅是技术集成问题，更是需要考虑性能、成本、安全的工程问题。

核心注意事项包括几个方面:API限制管理是首要考虑，大多数搜索服务都有调用频率和配额限制，需要实现合理的缓存机制和重试策略;结果质量控制同样重要，搜索返回的内容可能包含噪音或不相关信息，需要添加结果过滤和排序逻辑;成本控制不容忽视，频繁的搜索调用会产生AP费用，建议设置搜索触发条件和结果缓存。在安全层面要防范搜索注入攻击，对用户输入进行sanitization处理。同时要考虑数据时效性，搜索结果可能包含过期信息，需要在prompt中提醒Agent关注信息的时间戳。



38 什么是LCEL (LangChain Expression Language)?它有什么优势?

LCEL(LangChain Expression Language)是LangChain框架中的声明式编程语言，用于构建和组合大语言模型应用的处理链路。它采用管道式语法，通过 | 操作符将不同组件串联起来，形成数据处理流水线。

LCEL的核心优势体现在几个方面。首先是**简洁性**，你可以用一行代码 **prompt| 11m| output_parser** 就构建完整的对话链路，而传统方式需要编写大量样板代码。其次是**异步支持**，LCEL天然支持异步执行和流式输出，这对实时对话应用特别重要。可观测性也是重要特性，每个链路都自动支持调试、监控和日志记录，便于排查问题。

在实际应用中，LCEL让复杂的RAG系统变得清晰明了。比如构建文档问答系统时，你可以写成 **retriever|format_docs| prompt|11m| parser**，每个环节的职责一目了然。LCEL还支持并行执行和条件分支，可以根据输入类型动态选择不同的处理路径。

最重要的是，LCEL具备类型安全特性，在编译时就能发现组件间的接口不匹配问题，避免运行时错误。这种设计让复杂的AI应用开发变得更加可靠和高效。

39 Agent的可解释性与可控性如何权衡?如何在自主性和安全性间平衡?

Agent的可解释性与可控性权衡本质上是透明度与灵活度的博弈。当你增强可解释性时，比如要求Agent输出详细的推理链路和决策依据，势必会限制其使用复杂的黑盒模型或多层嵌套规划，这会降低某些场景下的性能表现。反过来说，如果追求极致的自主决策能力，让Agent自由组合工具、动态调整策略，你就很难完全追溯它每一步的内在逻辑。

实际平衡方案通常采用分层控制架构:在高风险操作层设置硬约束和人工审批点，比如金融交易Agent必须在执行前展示完整决策树并等待确认;而在低风险的信息检索或数据分析层允许更大自主度。同时可以引入可信度评分机制，让Agent自评每个决策的确定性，当置信度低于阈值时自动触发解释模块或请求人工介入。

另一个关键是渐进式放权:初期给Agent设定严格的规则边界和白名单工具集，随着在受限环境中验证其行为模式的可预测性，逐步扩大自主权限。这种做法在客服Agent部署中很常见——先处理标准问题，表现稳定后再开放复杂场景。核心思路是不追求绝对的可解释或绝对的自主，而是根据具体应用场景的风险承受度动态调节两者配比。

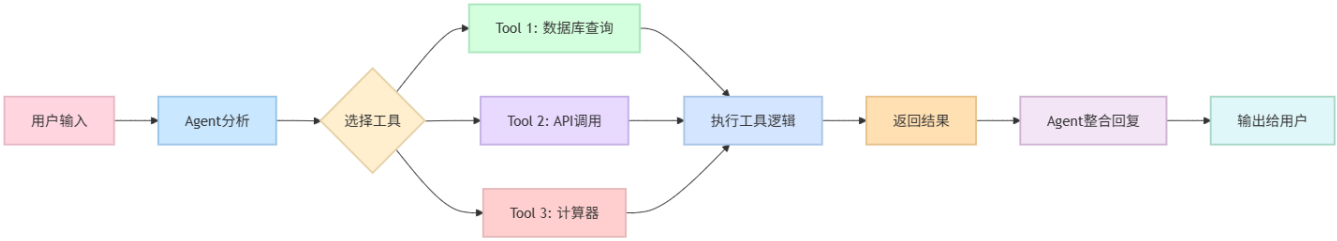
40 如何在LangChain中集成外部工具?Tool的定义规范是什么?

在LangChain中集成外部工具主要通过Tool类实现，你需要定义工具的核心属性和执行逻辑。 Tool的定义规范包含三个必要元素:(工具名称)、description(功能描述)和func(执行函数)。name用于LLM识别工具，description告诉LLM何时使用name该工具，func是实际的执行逻辑。你可以通过继承BaseTool类或使用@tool装饰器来创建工具。

```
from langchain.tools import tool

@tool
def search_database(query: str) -> str:
    """搜索产品数据库中的商品信息"""
    # 实际的数据库查询逻辑
    return search_result
```

集成时，你需要将工具添加到Agent的工具列表中，Agent会根据用户输入和工具描述自动选择合适的工具执行。关键是description要准确描述工具的功能和使用场景，这直接影响LLM的工具选择准确性。对于复杂工具，你还可以定义args.schema来规范输入参数格式，使用returndirect控制是否直接返回结果给用户。实际应用中，常见的外部工具包括AP调用、数据库查询、文件操作、计算器等，通过这种标准化的Tool接口，你可以轻松扩展LangChain的功能边界。



接下来谈到工具定义的两种方式时，面试官更想听到你对场景选择的判断。你可以说:“继承BaseTool类适合复杂工具，比如需要异步操作或者有复杂状态管理的场景。而@tool装饰器更适合简单的函数式工具，代码更简洁。

41 LangChain的输出解析器有什么作用?如何处理结构化输出?

LangChain的输出解析器主要用于将大模型生成的自然语言文本转换为结构化数据，解决了原始文本输出难以直接集成到应用程序中的问题。当你需要从模型获取JSON格式的用户信息、列表形式的推荐结果或者特定字段的数据时，解析器能确保输出符合预期结构。

输出解析器的核心作用包括数据格式化和类型转换。比如在构建客服系统时，你可能需要模型输出包含"问题分类”、“紧急程度"、"建议操作"等字段的结构化响应，而不是一段描述性文字。LangChain提供了多种解析器类型，PydanticOutputParser是最常用的，它基于Pydantic模型定义输出schema，能够进行类型验证和数据清洗。JSoNoutputParser专门处理JSON格式输出，而ListOutputParser则用于解析列表数据。

解析器还具备错误处理和重试机制，当模型输出格式不符合要求时，能够自动修正或重新请求。这种机制大大提高了生产环境中的稳定性，避免了因格式问题导致的应用崩溃，让大模型的输出能够可靠地集成到下游业务逻辑中。

42 Multi-Agent系统中的角色分工如何设计?有哪些典型角色模式?

Multi-Agent系统的角色分工设计核心是**按能力边界和任务属性拆分职责**，确保每个Agent专注特定领域，通过协作完成复杂目标。

这个过程本质上跟拆分微服务很像，都是要找到合理的边界。

设计时首先要明确任务流的关键节点，然后映射到具体角色。典型的角色模式包括:**规划者(Planner)**负责拆解用户需求、制定执行方案，比如接收”分析竞品”需求后拆分成数据采集、对比分析、报告生成等子任务;**执行者(Executor)**专注特定技能，像代码编写Agent处理编程任务、数据查询Agent对接数据库;评审

者(Reviewer)验证输出质量，检查代码bug或内容准确性;协调者(Coordinator)管理多Agent间的通信和任务分配，避免冲突。

实际应用中常见的还有领域专家角色，比如在客服系统里设置售前咨询Agent、售后支持Agent和投诉处理Agent，各自维护专属知识库;在研发场景中可能有需求分析Agent、架构设计Agent、测试Agent的分工。关键是避免角色能力重叠导致决策混乱，同时要设计好消息传递机制一–是采用中心化调度还是点对点协商，这直接影响系统的可控性和灵活性。角色粒度的把握也很重要，过细会增加协调成本，过粗则失去分工优势，通常以单一职责原则为准绳，确保每个Agent的prompt和工具集高度内聚。这样设计的好处是可以让每个Agent的能力保持专注，既降低了单个Agent的复杂度，又能通过组合应对各种场景。

43 LangGraph的持久化机制是什么?如何保存执行状态?

LangGraph的持久化机制基于**状态快照**和**检查点系统**，让你可以在任意执行节点保存和恢复图的完整状态。

核心实现是通过**Checkpointer接口**，LangGraph提供了内存、文件和数据库等多种存储后端。当图执行到特定节点时，系统会自动序列化当前状态包括变量值、执行位置、消息历史等关键信息，生成一个带时间戳的检查点。

在实际应用中，比如构建多轮对话机器人时，你可以在用户会话中断后通**过thread.id和checkpoint id**精确恢复到之前的对话状态，包括上下文记忆和执行进度。对于长时间运行的工作流，比如数据处理管道，可以在每个处理阶段创建检查点，当某个节点失败时直接从最近的检查点重新开始，而不用重跑整个流程。

配置持久化很简单，你只需要在创建图时指定checkpointer参数，比如 Memorysaver()用于内存存储或sqlitesaver()用于SQLite持久化。LangGraph还支持条件检查点，让你根据业务逻辑决定何时保存状态，比如在关键决策节点或用户交互点自动创建快照。这种机制确保了复杂AI应用的容错性和可恢复性，特别适合生产环境中需要高可靠性的场景。

详细解释

LangGraph的持久化机制本质上是一套状态管理和恢复系统，它通过检查点技术实现了AI工作流的中断恢复和状态追踪。与传统的session管理或简单的内存缓存相比，LangGraph解决了复杂AI工作流的容错问题，提供了细粒度的状态控制，支持条件检查点和增量恢复。

检查点系统的工作原理类似于数据库的事务日志，每当图执行到关键节点时，系统会将当前的完整状态序列化保存。这个状态不仅包含数据变量，还包含执行位置、消息上下文等元信息。检查点的数据结构设计很巧妙，它采用了类似Git快照的思想，每个检查点都是独立完整的状态副本，而不是增量变更。这种设计虽然存储开销稍大，但恢复时非常可靠，避免了增量恢复可能导致的状态不一致问题。

LangGraph的序列化策略很有讲究，它不是简单的对象序列化，而是智能识别状态中的不同数据类型。比如对于大型的AI模型实例，它会保存模型引用而不是模型本身；对于消息历史，它会压缩重复内容；对于执行上下文，它会重构执行栈信息。这种分层序列化策略既保证了状态完整性，又控制了存储成本。

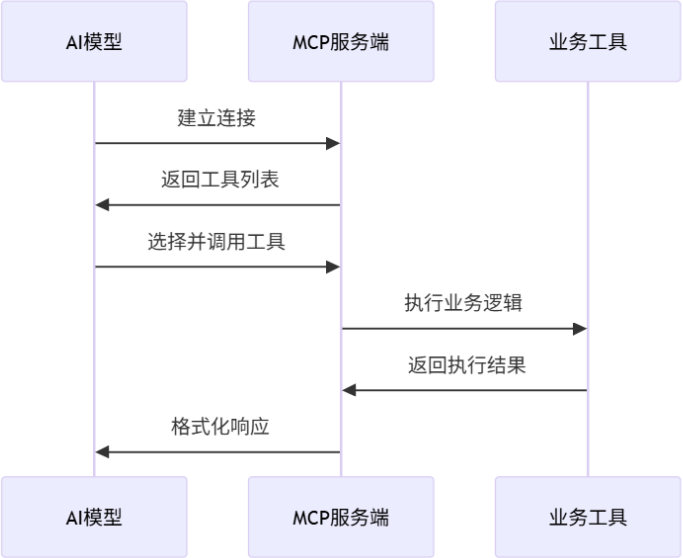
44 如何使用MCP(Model Context Protocol)协议开发一个简单的工具服务?

开发MCP工具服务需要实现服务端和客户端两个核心组件。服务端通过继承MCPServer类定义工具接口，使用@server.ca11_tool装饰器注册工具函数，每个函数需要定义name、description和parameters字段来描述工具功能和输入参数。比如开发一个文件操作工具，你可以注册read file和write file两个工具，分别处理文件读写操作。

关键实现步骤包括:首先定义工具schema，指定参数类型和必填字段;然后实现具体的工具逻辑函数，处理输入参数并返回结构化结果;最后启动MCP服务监听客户端请求。客户端通过MCPClient连接服务端，调用ca11_–too1方法执行远程工具，传入工具名称

和参数字典。

在实际应用中，你可以开发数据库查询工具、AP调用工具或文档处理工具等。MCP协议的优势在于标准化了工具接口，使得不同的AI模型都能无缝调用你开发的工具服务。记住要处理好错误异常和参数验证，确保工具服务的稳定性。整个开发过程就是定义工具接口、实现业务逻辑、启动服务这三个核心环节。



45 如何在LangGraph中处理并行执行?

LangGraph通过并行节点执行机制来处理并发任务，这是它相对于传统工作流引擎的核心优势之一。当你在图定义中将多个节点设置为同一层级时，它们会自动并行执行，而不是按顺序等待前一个节点完成。

实现方式相当直观，你只需要使用add_edge方法将同一个父节点连接到多个子节点，这些子节点就会并发运行。比如在文档分析任务中，可以让文本提取、图像识别、元数据解析三个节点同时处理同一份文

档，避免了串行等待带来的时间浪费。

关键技术点在于状态管理和结果聚合。LangGraph的状态对象天然支持并发访问，每个并行节点可以独立更新状态的不同字段，而不会产生竞态条件。当所有并行节点完成后，你需要设置一个聚合节点来收集和合并结果。

在代码层面，通过 compiledGraph.ainvoke()方法启动异步执行，LangGraph内部使用asyncio来管理并发。如果某个并行分支出现异常，可以通过错误处理节点来处理，不会影响其他分支的执行。这种设计特别适合多数据源查询、多模型推理对比、或者需要同时调用多个AP的场景。需要注意的是，并行度受限于系统资源和外部服务的并发限制，你应该根据实际情况合理设计并行分支数量

46 LangChain的文档处理功能包括哪些?如何处理大文档?

LangChain的文档处理功能主要围绕文档加载、切分、向量化和检索四个核心环节。在文档加载方面，支持PDF、Word.Markdown、CSV、JSON等多种格式，通过相应的Loader类实现统一接口调用。对于大文档处理，文档切分是关键技术LangChain提供了多种TextSplitter策略:CharacterTextSplitter按字符数切分、RecursiveCharacterTextSpliter递归切分保持语义完整性、TokenTextSplitter按token数量控制，还有针对特定格式的PythonCode TextSplitter等。

处理大文档时，你需要合理设置chunk size和chunk overlap参数，通常chunk size设为500–1500字符，overlap设为10%–20%确保上下文连贯。切分后的文档片段通过Embeddings转换为向量存储到Vectorstore中，支持Chroma、Pinecone.FAISS等向量数据库。在检索阶段，可以使用相似度搜索、MMR(最大边际相关性)搜索等策略获取相关片段。

对于超大文档，建议采用分层处理策略:先按章节粗切分，再对每个章节细切分，或者使用MapReduce模式并行处理。结合RetrievalQA链和ConversationalRetrievalChain，能够实现高效的大文档问答和对话功能，特别适用于企业知识库、技术文档查询等场景。

47 什么是代码解释器(Code Interpreter)?它在AI Agent中的作用是什么?

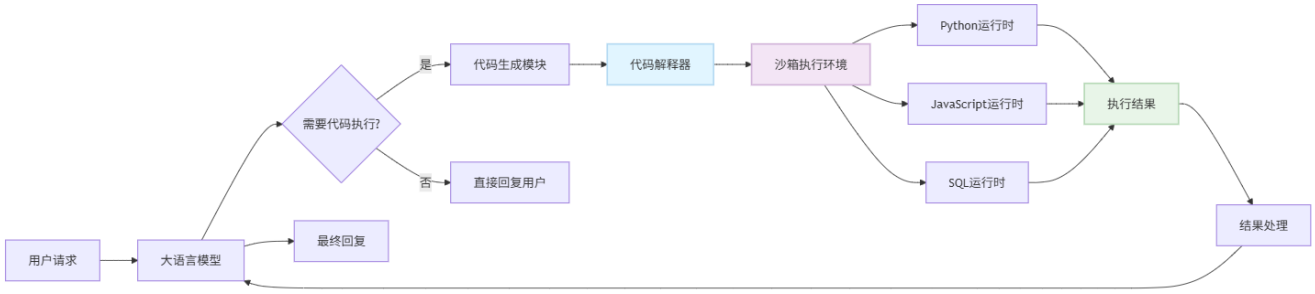
代码解释器(Code Interpreter)是一个能够动态执行和解释代码的组件，它接收文本形式的代码输入，实时解析并返回执行结果。与传统的编译执行不同，代码解释器采用逐行或逐块解释的方式，支持交互式编程体验。

在AIAgent中，代码解释器扮演着核心执行引擎的角色。当AI Agent需要处理复杂的数据分析、数学计算或算法实现时，它会生成相应的Python、JavaScript或其他语言代码，然后通过代码解释器执行这些代码获得结果。这种机制让AI Agent具备了真正的计算能力，而不仅仅是文本生成。

具体应用场景包括:AIAgent接收到”分析这份CSV数据的趋势"请求时，会生成pandas数据处理代码并通过解释器执行，产生可视化图表;或者在处理"计算复杂数学函数"时，生成NumPy计算代码获得精确结果。

代码解释器的价值在于桥接了AI的推理能力与实际的计算执行，使AIAgent从纯粹的对话系统升级为具备实际问题解决能力的智能体。它确保了AII生成的代码能够被验证和执行，大大提升了AIAgent在科学计算、数据分析、自动化任务等领域的实用性和可靠性。

下面这个架构图展示了代码解释器在AI Agent中的核心地位:



48 如何设计Multi-Agent的任务分配算法?负载均衡如何实现?

Multi-Agent任务分配的核心是任务拆解、能力匹配和动态调度。你需要先定义Agent的能力模型，比如每个Agent擅长的任务类型、当前负载、响应速度等元数据。任务进来时，通过任务路由器解析任务需求，匹配合适的Agent。

具体算法上，可以采用基于优先队列的调度:维护每个Agent的任务队列，根据Agent当前队列长度、历史执行时间、任务优先级计算分配权重。比如在客服场景中，新会话进来时，选择当前会话数最少且擅长该问题域的Agent接手。更复杂的可以用强化学习训练分配策略，让系统学习最优分配模式。

负载均衡实现主要靠实时监控和动态调整。你需要追踪每个Agent的CPU、内存占用、任务队列深度、平均响应时间等指标，当某个Agent负载超过阈值时，触发任务迁移或重新分配。可以设置心跳机制，Agent定期上报状态到协调器。当检测到负载倾斜，使用最少连接算法或加权轮询重新路由新任务。在分布式环境下，建议用一致性哈希保证Agent扩缩容时任务迁移量最小，同时配合背压机制，当下游Agent处理不过来时主动拒绝新任务，避免雪崩。

49 什么是Agent的工具学习(Tool Learning)?如何让Agent学会使用新工具?

Tool Learning是指Agent学习理解和使用外部工具AP来扩展自身能力的过程，核心在于让Agent理解工具的功能描述、参数规范，并在合适的时机正确调用。这就像人类学会使用计算器、搜索引擎来增强自

己的能力一样，大语言模型虽然知识丰富，但受限于训练数据的时效性和专业领域的深度，无法实时获取股票报价、执行数据库查询或调用支付接口。工具学习让Agent突破了这些限制，能够完成真实世界的复杂任务。

当前主流方法包含几个关键环节。首先通过工具描述文档(通常是JSON Schema格式)告诉Agent工具的名称、用途、参数类型和返回值，这些描述会被编码到prompt中。然后Agent需要学会意图识别，判断用户需求是否需要调用工具。接着进行参数映射，从对话上下文中提取并填充正确的参数值。最后解析工具返回结果并生成回复。

让Agent学会使用新工具主要有三种路径。In-Context Learning是最轻量的方式，直接在prompt中加入新工具的文档和调用示例，LLM通过上下文理解就能使用。Few-shot示例更进一步，提供几个该工具的成功调用案例，帮助Agent理解调用模式。对于复杂场景则需要微调训练，构建包含工具调用的监督数据集，让模型学习工具选择和参数生成的能力。实际应用中，像GitHubCopilot调用代码搜索API、ChatGPT调用Python解释器、AutoGPT调用文件系统操作，都是Tool Learning的体现，关键难点在于工具描述的准确性和Agent对复杂多步工具链的规划能力。

牛客/小红书面经：

1.请阐述RAG的核心原理，并说明如何通过 RAG 缓解大模型的幻觉问题。

RAG 的全称是 **Retrieval-Augmented Generation**，也就是**检索增强生成**。它的核心原理是：**先从外部知识库中检索与用户问题相关的信息，再把这些信息作为上下文提供给大模型，由模型基于这些外部证据生成答案。** 所以它不是只依赖模型参数里的静态知识，而是把“检索”和“生成”结合起来。

它的基本流程通常分为四步：第一步，先对知识库文档做切分和向量化，建立索引；第二步，用户提问后，把 query 也向量化；第三步，在向量数据库或混合检索系统里召回最相关的文档片段；第四步，把这些片段连同用户问题一起输入大模型，让模型基于检索结果回答。

RAG 之所以能缓解大模型幻觉，是因为它给模型提供了**可参考的外部证据**。传统大模型如果只靠参数作答，遇到知识缺失、知识过时或长尾问题时，就容易凭概率“编一个看起来合理的答案”；而 RAG 通过先检索相关资料，把回答建立在外部知识之上，能显著降低这种无依据生成。

不过要注意，RAG 不是天然完全消除幻觉，它只是降低幻觉概率。真正要缓解幻觉，还要保证**检索质量高、上下文干净、并且在生成阶段明确约束模型只能基于证据回答**。

所以我一般会总结成一句话：**RAG 的本质是“先找资料，再基于资料回答”，它通过引入外部知识，把模型从“凭记忆作答”变成“参考证据作答”，从而有效缓解幻觉问题。**

2.如何设计一个高效的 agent 上下文维护方案?

设计一个高效的 Agent 上下文维护方案，核心目标是 **在有限上下文窗口里，始终保留当前任务最有价值的信息**，同时控制 token 成本和系统延迟。我的思路一般是做成 **分层上下文架构**，而不是把所有历史对话直接塞进模型。

第一层是**当前轮上下文**，保留用户最新问题、系统指令、最近几轮关键交互和最新工具返回结果，保证 Agent 能处理眼前任务。第二层是**工作记忆**，专门存当前任务的计划、子任务状态、中间结论和待办事项，它更像一个 task state，而不是普通聊天记录。第三层是**长期记忆**，把用户偏好、固定规则、历史经验这类跨会话有价值的信息持久化到数据库或向量库，需要时再检索回来，而不是一直放在 prompt 里。

具体流程上，我会先对输入做**记忆分类**，判断哪些信息该进入短期上下文，哪些该写入长期记忆；然后在每轮调用前做**上下文组装**，只保留当前任务强相关的信息；当历史过长时，用**摘要压缩**替代原始对话；对于外部知识和历史经验，用**按需检索**而不是全量注入。再配合 **优先级策略**，保证 system prompt、当前目标、最新 observation 和关键约束始终优先保留。

所以我一般会总结成一句话：**高效的 Agent 上下文维护，不是简单堆历史，而是通过“分层存储、按需检索、动态压缩、优先级组装”来让模型始终看到最关键的信息。**

3 一个完整的 agent 智能体架构一般包括哪些部分?

一个完整的 Agent 智能体架构，一般可以分成 **目标层、感知与上下文层、决策规划层、执行层、记忆层、反馈反思层** 这几个核心部分。

首先是目标输入层，也就是接收用户任务、系统指令和约束条件，明确 Agent 到底要解决什么问题。其次是感知与上下文层，负责理解当前输入，并结合对话历史、环境状态、外部知识和当前任务上下文，形成模型推理所需的输入。

再往下是决策规划层，这是 Agent 的“大脑”，负责任务分解、路径规划、决定下一步要不要调用工具、是否需要搜索、是否要调整策略。然后是执行层，也就是实际去调用工具、访问数据库、执行代码、发送请求，把规划转化成动作。

同时，一个成熟的 Agent 还一定要有记忆层，用于管理短期记忆、长期记忆和工作记忆，保证它不仅能处理当前任务，还能利用历史经验和用户偏好。除此之外，还需要反馈与反思层，也就是根据工具返回结果、自检结果或者环境反馈，判断当前步骤是否有效，必要时进行纠错、重试和策略修正。

在工程实现上，通常还会补充安全与监控模块，比如权限控制、日志追踪、异常处理、成本和延迟监控。

所以我一般会总结成一句话：一个完整的 Agent 架构，本质上就是“目标输入 + 上下文感知 + 规划决策 + 工具执行 + 记忆管理 + 反馈反思”的闭环系统。

4 向量检索和关键词检索的区别是什么？

向量检索和关键词检索的核心区别在于，**关键词检索靠“字面匹配”，向量检索靠“语义相似”**。

关键词检索，比如 BM25，主要看 query 里的词有没有出现在文档里，以及词频、逆文档频率这些统计特征。所以它特别适合那些关键词非常明确的场景，比如产品型号、专有名词、报错信息、代码函数名这类查询。它的优点是速度快、可解释性强、对精确匹配很有效；但缺点是语义理解能力弱，如果用户换了说法、用了同义词，检索效果可能就会下降。

向量检索 则是先把 query 和文档都编码成 embedding 向量，再通过向量距离去找最相似的内容。它的优势是能做语义级匹配，即使 query 和文档没有完全相同的词，只要表达的意思接近，也能召回相关内容。所以它特别适合自然语言问答、知识库检索和语义搜索。缺点是对精确实体、数字、缩写和关键词约束有时不如关键词检索稳定。

所以实际系统里，两者往往不是二选一，而是做**混合检索**：关键词检索保证精确命中，向量检索保证语义召回。

我一般会总结成一句话：**关键词检索解决“词对不对”，向量检索解决“意思像不像”，两者结合通常效果最好。**

5.讲解召回重排的策略

召回重排策略，本质上就是把检索分成两个阶段：**第一阶段尽量“多找一些可能相关的内容”，第二阶段再把这些候选结果“排得更准”**。这么设计的原因是，检索系统很难同时兼顾高召回和高精度，所以通常会先粗召回，再精重排。

在**召回阶段**，目标是不要漏掉有用信息，常见策略包括 **关键词召回、向量召回和混合召回**。关键词召回比如 BM25，适合专有名词、报错信息、数字这类精确匹配；向量召回适合语义检索；混合召回就是把两者结合，提高覆盖率。有些系统还会做 **多路召回**，比如原始 query、查询重写、HyDE、多 query 并行召回，再把结果合并去重。

在**重排阶段**，目标是从这些候选里把最相关的排到前面。常见做法是用 **Cross-Encoder Reranker** 或 LLM rerank，对 query 和每个候选文档做更细粒度的匹配打分。相比召回阶段的快速粗筛，重排更关注相关性准确率。

一个完整策略通常是：**先用 BM25 + 向量检索做 TopK 召回，再合并去重，然后用 reranker 重新打分排序，最后取前几个最优结果送给大模型。**

所以我一般会总结成一句话：**召回重排策略就是“先广泛找，再精确排”，核心是在保证不漏信息的前提下，把最相关的内容尽量送到最前面。**

6.如果让你设计OpenClaw你会考虑哪些因素，以及你觉得他的发展限制因素有哪些

如果让我设计 OpenClaw 这类平台，我会优先考虑五个因素。第一是 **安全性**，因为它本质上是一个能接消息、调工具、甚至执行本地操作的 Agent 网关，所以权限隔离、身份认证、最小权限、审计日志和技能安全检查一定要放在第一位。OpenClaw 官方定位就是自托管的个人 AI 助手网关，能连接多种聊天渠道和 AI agent，因此它天然比普通聊天产品更接近“高权限执行系统”。(GitHub)

第二是 **多渠道接入与稳定编排**。既然它主打 WhatsApp、Telegram、Discord 等多入口统一接入，那么消息路由、状态同步、失败重试和渠道一致性体验会直接影响产品价值。第三是 **技能与工具生态**，因为这类平台的上限不只取决于模型，而取决于能否稳定接入工具、沉淀 Skills，并形成可复用生态。官方资料和社区资源也都强调了其多渠道能力和 skills 生态。(GitHub)

第四是 **上下文和记忆管理**，要支持多轮任务、工具调用历史、长期偏好和任务状态恢复；第五是 **可部署性和可运维性**，包括本地、自托管、云端部署体验，升级、监控、故障恢复和成本控制，否则再强也很难真正落地。OpenClaw 官方就强调它是 self-hosted gateway，面向开发者和 power users。(OpenClaw)

我觉得它的发展限制因素主要有三类。第一类是 **安全风险**，最近公开披露的 ClawJacked 漏洞就说明，这类“本地高权限 Agent + 多渠道接入”系统一旦安全边界设计不好，风险非常大；相关漏洞已被修复，但这件事本身说明安全会长期限制其企业化和大规模普及。(OASIS Security) 第二类是 **生态复杂度**，接入的渠道、工具和 skills 越多，兼容性、治理和质量控制越难。第三类是 **用户信任与运维门槛**，自托管虽然给了控制权，但也提高了部署、升级和排障成本；再加上近期还出现了借 OpenClaw 热度传播恶意安装包的案例，这会进一步提高普通用户的使用门槛。(TechRadar)

所以我会用一句话总结：**如果让我设计 OpenClaw，我会把它当成一个高权限的 Agent 操作系统来做，核心抓手是安全、编排、生态、记忆和运维；而它能不能继续发展，最大的瓶颈其实不是模型能力，而是安全可信、生态治理和落地成本。**

7.讲一下agent中有哪些模式

如果面试时用 1 分钟回答，我会这样说：

Agent 里常见的模式，通常可以按**任务执行方式**来分，比较典型的有 **ReAct**、**Plan-and-Execute**、**Reflection**、**Tool-Use**、**Multi-Agent Collaboration** 这几类。

ReAct 是最经典的模式，核心是“边思考边行动”，也就是模型先做一小步推理，再调用工具，根据返回结果继续下一步，所以适合动态环境下的检索、查询和多步任务。

Plan-and-Execute 是“先规划再执行”，模型先把复杂任务拆成若干步骤，形成整体计划，然后再逐步完成。它比 ReAct 更适合长链路、复杂任务，因为有全局路线，不容易中途跑偏。

Reflection 模式强调“先做再检查再修正”，也就是模型完成一步或一版结果后，会回头自查、自我纠错，特别适合代码生成、复杂推理和长任务执行。

Tool-Use 模式强调工具调用能力，核心是让 Agent 不只是聊天，而是能根据需要调用搜索、数据库、代码执行器、API 等外部工具完成任务。

Multi-Agent Collaboration 就是多智能体协同，把复杂任务拆给不同角色的 Agent，比如 Planner、Retriever、Executor、Reviewer 分工合作，用来提升复杂任务处理能力。

如果再总结一下，我会说：**Agent 的常见模式，本质上是在回答三个问题：要不要边做边想，要不要先整体规划，要不要做结果反思，以及是单 Agent 执行还是多 Agent 协作。**
实际系统里，这些模式往往不是单独存在的，而是组合使用，比如先 Plan，再用 ReAct 执行，最后再加 Reflection 做校验。

8.你编写了哪些MCP工具，介绍一下

在项目里，我写过的 MCP 工具，主要可以分成三类：**数据访问类、检索增强类、业务执行类**。我的设计思路不是把 MCP 当成“简单 API 包装”，而是把它当成 **给大模型提供标准化外部能力接入口**。

第一类是**数据访问类工具**，比如文件读取、目录检索、文档解析、数据库查询这类工具。它们的作用是让 Agent 能访问本地知识、结构化数据或者业务数据源。设计时我会重点处理输入参数校验、权限控制和返回结果结构化，避免模型拿到一堆不可控原始文本。

第二类是**检索增强类工具**，比如知识库检索、向量检索、混合检索、指定文档搜索这类 MCP 工具。它们主要服务 RAG 场景，让模型能按统一协议去查资料，而不是把检索逻辑硬编码在 Agent 里。这里我会特别注意返回结果里带上 `source`、`score`、`metadata`，方便后续做引用和可追溯。

第三类是**业务执行类工具**，比如发送请求、触发工作流、查询某个业务状态、生成报表这类工具。这类工具更接近真正的“执行能力”，所以我会更强调幂等性、异常处理、超时控制和审计日志。

如果面试官继续追问“你怎么理解 MCP 工具开发的重点”，我会补充一句：**重点不是把函数暴露出去，而是把能力标准化、可控化、可组合化，让大模型既能调用，又不会乱调用。**

所以我一般会总结成一句话：**我写的 MCP 工具主要围绕“数据获取、知识检索、业务执行”三类展开，核心目标是把外部能力以统一、可控、可复用的方式暴露给 Agent。**

9.聊一下muliagent，你是怎么做意图识别的

在 Multi-Agent 系统里，我做意图识别的核心思路，不是把它当成一个孤立的分类模型，而是把它当成**整个任务路由的入口层**。也就是说，用户问题一进来，我首先要判断它到底属于哪一类任务，再决定应该交给哪个 Agent、走哪条流程。

具体做法上，我一般会先定义一个**意图体系**，比如常见会分成知识问答、结构化查询、文档检索、工具执行、闲聊兜底、敏感问题拦截这几类。然后在实现上，我通常采用**“规则 + LLM 分类”**的两阶段方案。第一层用关键词、正则和一些显式规则做快速粗分，比如用户有没有提到“查数据库”“帮我总结文档”“生成报表”这种强特征；第二层再交给 LLM 做语义级判断，输出结构化标签，比如 `intent` 、 `confidence` 、 `是否需要检索` 、 `是否需要工具调用` 。

如果置信度比较高，就直接路由到对应 Agent；如果置信度低，或者问题本身是复合意图，比如既要查资料又要做分析，我会进入一个 **planner 或 router agent**，把任务拆成多个子意图，再分别交给不同 Agent 处理。

我这样设计的原因是，纯规则虽然快，但泛化差；纯 LLM 虽然灵活，但成本更高、稳定性也没那么强。两者结合，会更适合工程落地。

所以我一般会总结成一句话：**我做意图识别，本质上是在做多智能体系统的任务路由，通过“意图体系设计 + 规则粗分 + LLM 精分 + 低置信度兜底”来决定任务该由哪个 Agent 接手。**

10 你是如何处理这种异常情况的，比如路由到了一个错误的任务，如果用户说不对，我要问的问题是XXX，你这个agent会不会纠正自己的行为，还是再跑一次流程

会的，我不会简单地“原流程再跑一遍”，而是把这种情况当成一次**显式纠错信号**来处理。也就是说，当用户说“不是这个意思”或者“我要问的是 XXX”时，系统会认为前一次意图识别或路由结果失败了，接下来进入一个 **纠错分支**，而不是无脑重试。

具体做法上，我一般会分三步。第一步是**捕获用户否定反馈**，像“不对”“不是这个”“我想问的是”这类表达，会触发 re-route 机制。第二步是**保留当前对话上下文和上一次路由结果**，把它们一起交给路由层重新判断，这样模型不仅看到用户的新说明，也知道“刚才判成了什么、为什么被否定”。第三步是**重新做意图识别和任务路由**，必要时直接切到新的 Agent，而不是原 Agent 再硬答。

如果用户只是轻微补充信息，我会做 **局部修正**；如果用户明确否定了任务方向，我会做 **全量重路由**。另外我通常还会加一个 **反思或校验机制**，也就是在重路由前让系统先判断：是意图分类错了，还是参数抽取错了，还是工具执行阶段理解偏了。这样可以减少“明明是槽位错了，却把整个流程全推翻”的情况。

所以我一般会总结成一句话：**遇到用户纠正时，我会把它当成高优先级反馈信号，触发带上下文的重路由和纠错，而不是简单把原流程再跑一次。**

11 任务分解后，会根据任务执行结果调整任务列表吗

会的，**任务分解不是一次性静态生成后就不变了**，而是一个**动态规划和动态调整**的过程。因为 Agent 在真正执行子任务时，会不断拿到新的信息，比如检索结果不符合预期、工具调用失败、某个前提条件不成立，或者中间发现用户真正想要的问题和最初理解不一样。这些都会影响后续任务列表。

所以我通常会把任务列表设计成一种 **可更新的执行计划**。初始阶段先由 planner 给出一个主计划，把大任务拆成若干子任务；然后每完成一步，系统都会根据执行结果做一次状态检查，判断当前计划是不是还成立。常见的调整方式包括：**删除已经不需要的任务、补充遗漏的任务、调整任务顺序、或者在必要时重新规划整条链路。**

比如原本计划是“先检索资料，再分析，再输出总结”，但如果检索阶段发现资料不足，那后面可能就不能直接分析，而是要先增加一个“扩大检索范围”或者“向用户澄清需求”的子任务。

所以我一般会总结成一句话：**任务分解后的任务列表不是静态脚本，而是会根据执行反馈持续更新的动态计划，这样 Agent 才能真正适应复杂和开放的任务环境。**

12 大文本说明书常用的切分方式有哪些

大文本说明书的切分，核心目标是 **既要保留语义完整性，又要方便后续检索和定位**。常见的切分方式主要有四类。

第一类是 **固定长度切分**，比如按 500 或 1000 个 token 一段，再加一定 overlap。它实现简单、工程上最常见，但缺点是可能把一个完整语义切断。

第二类是 **按自然结构切分**，也就是按照标题、章节、小节、段落、列表、表格说明来切。这种方式特别适合说明书、技术文档、论文，因为它能尽量保持一个 chunk 内部主题一致，检索质量通常比纯固定切分更好。

第三类是 **递归切分**，也就是优先按章节、再按段落、再按句子逐层往下切，直到满足长度限制。这个方式比较平衡，既能保留结构，又能控制 chunk 大小，所以在实际 RAG 里很常用。

第四类是 **语义切分**，就是根据内容主题变化来切，而不是只看长度。比如同一段里如果话题已经变了，就在那里断开。它效果通常更好，但实现成本更高。

工程上还常配合两种增强方式：一个是 **overlap 重叠切分**，避免关键信息被切断；另一个是 **父子块切分**，小块用于召回，大块用于返回上下文。

所以我一般会总结成一句话：**说明书切分常见有固定长度切分、结构化切分、递归切分和语义切分，实际项目里通常会优先选择“按文档结构 + 递归切分 + overlap”的方案。**

13 你说按段落/按语义切，那段落有的 100 字、有的 1 万字怎么办?

这也是为什么实际做切分时，**不能只机械地按段落切，也不能只说按语义切**，而是要做 **分层递归切分**。因为真实文档里，段落长度差异会非常大，有的段落很短，直接切就可以；但有的段落可能特别长，比如一整页说明、长表述或者连续步骤说明，这时候如果整段当一个 chunk，就会超出模型上下文限制，也不利于检索。

所以我的做法一般是这样的：**先按文档自然结构切，比如章节、二级标题、段落；如果某一段太长，就继续往下递归切分，比如按句子、分号、步骤编号、列表项再拆；如果某一段太短，就可以和相邻的同主题段落合并。** 也就是说，段落不是最终单位，而只是第一层边界。

对于特别长的段落，我会再加两个策略。第一是 **设置 token 上下限**，比如希望 chunk 大小落在一个合理范围内；超过上限就继续拆，小于下限就尝试合并。第二是 **加 overlap**，避免关键信息正好落在边界上被切断。

所以本质上不是“按段落切”或者“按语义切”二选一，而是：**先尊重文档结构，再用长度约束做二次递归调整。**

我一般会总结成一句话：**遇到段落长短不一时，正确做法不是死按段落切，而是采用“结构优先、长度约束、递归细分、短段合并”的动态切分策略。**

14 Agent skil怎么理解，跟tools是区别是什么

Agent 里的 **Skill**，我一般理解成**面向某一类任务的能力封装**；而 **Tool** 更像是**一个具体的外部工具或原子功能接口**。两者的核心区别在于抽象层级不同。

Tool 通常是单点能力，比如搜索网页、查数据库、读取文件、调用天气 API、执行一段代码。它解决的是“我能做什么具体动作”。所以 Tool 更偏底层、原子化、接口化。

而 **Skill** 往往不是一个单独动作，而是一套可复用的任务处理流程。它里面可能会包含 prompt、规则、多个 tool 的编排、输入输出格式、异常处理，甚至还有结果校验。比如“文档总结”可以算一个 Skill，但它背后可能会先读文件、再切分、再检索、再总结、最后输出结构化结果。

所以如果类比的话，**Tool 像函数或 API，Skill 更像一个封装好的业务能力模块**。
在 Agent 系统里，通常是 Agent 负责决策，Skill 负责完成某类任务，而 Skill 内部又可以继续调用多个 Tool。

我一般会总结成一句话：**Tool 是原子能力，Skill 是对一类任务流程的能力封装；Tool 回答“怎么做一个动作”，Skill 回答“怎么完成一类事情”。**

15 上下文工程了解多少，长短期记忆如何做

我对**上下文工程**的理解是，它本质上是在解决一个问题：**怎么把“当前任务真正需要的信息”以合适的形式、合适的顺序、合适的粒度送进大模型**。因为模型效果很多时候不只取决于模型本身，还取决于上下文是不是组织得合理。上下文工程通常会包括 system prompt 设计、历史对话管理、工具返回结果裁剪、RAG 检索内容注入、任务状态维护、记忆检索和上下文压缩等。核心目标就是两点：**提高有效信息密度，控制 token 成本。**

在长短期记忆上，我一般会分层做。**短期记忆**主要服务当前会话和当前任务，通常保存最近几轮对话、当前计划、最新工具结果和关键约束，常见做法是滑动窗口加摘要压缩，也就是保留最近原始内容，较早内容压成 summary。**长期记忆**主要服务跨会话复用，比如用户偏好、固定规则、历史经验和高价值事实，这部分一般不会一直塞进 prompt，而是存到数据库、向量库或知识库中，等当前问题需要时再检索回来。

如果从流程上说，就是：**用户输入进来后，先从短期记忆里拿当前上下文，再根据任务去长期记忆里按需检索，把两部分和当前问题一起组装成 prompt；任务结束后，再判断哪些信息值得沉淀到长期记忆。**

所以我一般会总结成一句话：**上下文工程是让模型“在有限窗口里看到最重要的信息”，而长短期记忆就是通过“当前保留 + 长期沉淀 + 按需检索”来支撑这一点。**

16 记忆覆盖问题是如何解决的?

记忆覆盖问题，本质上是指 **新信息不断写入后，把旧的但仍然重要的信息冲掉了**，或者同一类记忆被后续低质量内容反复覆盖，导致 Agent 后面检索到的是错误、过时或噪声更大的内容。这个问题在长期记忆里尤其常见。

我一般会从四个层面解决。第一是 **分层存储**，不是所有信息都直接写进长期记忆。临时信息先放短期记忆或工作记忆，只有真正稳定、长期有价值的信息才进入长期记忆。第二是 **版本和时间戳机制**，记忆不是简单覆盖，而是保留版本、更新时间和来源，这样后面检索时可以判断是最新事实、历史事实还是过期信息。第三是 **写入门控**，也就是在写 memory 之前先做价值判断，过滤掉低价值、重复、矛盾或不确定的信息，避免“什么都记”。第四是 **检索排序和冲突消解**，即使长期记忆里存在多条相似信息，也会根据时间、置信度、来源可靠性和与当前任务的相关性重新排序，而不是谁最后写入谁就生效。

如果是用户偏好这类会变化的信息，我还会做 **显式更新策略**，比如新偏好可以覆盖旧偏好，但要保留变更记录；如果是知识事实，就更偏向追加版本而不是直接覆盖。

所以我一般会总结成一句话：**解决记忆覆盖问题，核心不是“多存一点”，而是通过分层存储、写入门控、版本管理和检索时冲突消解，保证重要记忆不会被噪声或过时信息轻易覆盖。**

17 prompt设计的规则

我理解的 Prompt 设计规则，核心就是一句话：**让模型清楚知道“你是谁、要做什么、按什么标准做、输出成什么样”。** 一个好的 Prompt 不是写得越长越好，而是要**角色清晰、任务明确、约束具体、输出可控**。

具体来说，第一是 **角色定义清楚**，比如让模型扮演面试官、代码助手、RAG 问答助手，不同角色会影响回答风格和关注点。第二是 **任务目标明确**，不能只说“帮我分析一下”，而要说明分析什么、重点看什么。第三是 **提供必要上下文**，比如背景信息、输入数据、参考资料、业务场景，没有上下文模型就容易泛化乱答。第四是 **明确约束条件**，比如只能基于提供材料回答、不要编造、控制字数、使用某种格式输出。第五是 **规定输出格式**，像 JSON、分点、表格、固定字段，这对工程落地尤其重要。

另外一个很重要的原则是 **少模糊，多具体**。比如不要只说“写得专业一点”，而是直接说“用面试回答风格，控制在 1 分钟内”。如果任务复杂，我还会把 Prompt 拆成步骤，比如先分析、再判断、最后输出。

所以我一般会总结成一句话：**Prompt 设计的本质，是通过清晰的角色、目标、上下文、约束和输出格式，把模型从“自由发挥”变成“按要求稳定完成任务”。**

18 你认为要把一件事情做好，Agent应该怎么拆分。Agent拆分的环节数量，对于最终的交付质量有什么关系

我认为要把一件事情做好，Agent 的拆分原则不是“拆得越多越高级”，而是要按**职责边界、复杂度和可验证性**来拆。通常我会优先把一个复杂任务拆成几个关键环节：**任务理解、任务规划、信息获取、执行处理、结果校验**。如果业务更复杂，还可以继续细分成 Router、Planner、Retriever、Executor、Reviewer 这类角色。这样做的核心目的是让每个 Agent 只负责一类清晰职责，降低单个 Agent 的认知负担，也方便定位问题。

至于拆分环节数量，我觉得它和最终交付质量不是简单正相关，而是一个**平衡问题**。拆得太少，单个 Agent 负担过重，容易上下文混乱、角色冲突、出错后也很难定位；但拆得太多，又会带来链路过长、通信成本高、误差累积和整体延迟增加的问题。也就是说，环节越多，不一定质量越高，反而可能因为协同复杂度上升，把系统做得更脆弱。

所以我的判断标准一般是：**如果一个环节有独立目标、独立输入输出，而且结果可以单独评估，那就值得拆；如果只是机械拆分、拆完没有明显边界价值，那就不该拆。**

我一般会总结成一句话：**Agent 拆分的关键不是数量，而是是否形成清晰职责、低耦合和可验证的协作链路；合理拆分能提升质量，过度拆分反而会降低交付稳定性。**

19 对于Agent记忆的理解，当前市面上用的多的方案有哪些

我对 Agent 记忆的理解是：**记忆本质上是把“当前任务需要的信息”和“跨会话长期有价值的信息”分层管理，并在合适时机写入、检索、注入上下文。** 现在主流框架基本都在按这个思路做。比如 LangGraph

明确区分了线程内的短期记忆和跨会话的长期记忆；LlamaIndex 也把 memory 作为 Agent 核心组件，支持短期会话记忆和多种长期 memory block；Mem0 则更强调把记忆做成独立的 memory layer，用分层方式管理当前任务、会话和长期用户记忆。

当前市面上用得比较多的方案，我一般会分成四类。第一类是**会话缓冲型短期记忆**，也就是保留最近几轮对话和当前状态，超出 token 后再做摘要压缩，这类方案在 LangChain/LangGraph、LlamaIndex 里都很常见。第二类是**向量检索型长期记忆**，把用户偏好、历史事实或旧会话片段向量化存储，需要时按相似度召回；LlamaIndex 官方就把 vector memory block 作为长期记忆的一种典型方式。第三类是**结构化/事实型记忆**，把偏好、规则、事实抽取成 key-value、profile 或 facts，而不是只存原文，LlamaIndex 的 static block 和 fact extraction block、以及 LangMem 这类做法都属于这一类。第四类是**独立记忆层方案**，像 Mem0 这种，把 memory 从 Agent 主流程里抽出来，单独负责写入、检索、作用域管理和上下文注入。

如果一句话总结，我会说：**现在主流 Agent 记忆方案，已经从“简单保存聊天历史”发展成“短期状态 + 长期检索 + 结构化事实 + 独立 memory layer”的组合架构。** 实际落地时，最常见的是“短期窗口/摘要 + 长期向量记忆”，而在更成熟的系统里，会再叠加事实抽取、用户画像和命名空间隔离来提升稳定性和个性化。

20 你如何设计上下文窗口

我设计上下文窗口的核心原则是：**不是把所有信息都塞进去，而是只让模型看到当前任务最需要、最有价值的信息。** 因为上下文窗口本质上是稀缺资源，放得太多会增加噪声、成本和延迟，反而降低效果。

具体上，我一般会按 **优先级分层** 来设计。第一层一定是 **system prompt 和核心约束**，因为这是模型行为边界；第二层是 **当前用户问题和当前任务目标**；第三层是 **最近几轮强相关对话**，用来维持当前会话连续性；第四层是 **当前任务的工作记忆**，比如计划、子任务状态、最新工具返回结果；第五层才是 **按需检索进来的长期记忆或外部知识**，而不是把所有历史都放进去。

如果上下文太长，我不会简单截断，而是做两件事：一是 **摘要压缩**，把早期历史总结成结构化 summary；二是 **按需检索**，只把和当前问题强相关的历史记忆、知识片段或工具结果重新取回。也就是说，我更倾向于“动态组装上下文”，而不是“堆聊天记录”。

另外在工程上，我还会做 **token 预算分配**，比如给系统指令、对话历史、检索内容、工具结果分别预留预算，避免某一部分把窗口全部占满。

所以我一般会总结成一句话：**上下文窗口设计的关键是分层、排序、压缩和按需检索，让模型在有限窗口里始终看到最关键的信息。**

21 如何确定写入长期记忆中是重要的

判断一条信息要不要写入长期记忆，我一般不会让系统“什么都记”，而是看它是否同时满足 **长期价值、可复用性和稳定性** 这几个条件。因为长期记忆不是聊天日志备份，而是给后续任务提供持续帮助的知识层。

具体来说，我通常会从四个维度判断。第一是 **时间维度**，这条信息是不是在未来多轮对话或跨会话里还可能有用，比如用户长期偏好、固定规则、常用术语。第二是 **复用价值**，它是不是能显著改善后续决策或回答质量，比如用户明确说“以后都按这种格式回答”。第三是 **稳定性**，如果只是一次性的临时状态，比如“我今天下午有会”，通常不适合直接写入长期记忆；但如果是长期偏好或明确约束，就适合沉淀。第四是 **可信度**，不是所有输入都直接写，我会先做去重、冲突检测和置信度判断，避免把噪声、猜测或矛盾信息写进去。

工程上我一般会设计一个 memory write gate，也就是写入门控机制。先由规则或模型判断这条信息是不是“值得记”，再决定存成哪种形式，比如用户偏好、事实、经验总结，而不是原样整段存进去。

所以我一般会总结成一句话：**写入长期记忆的标准，不是“出现过”，而是“未来还会有用、相对稳定、可复用、并且足够可信”。**

22 短期记忆摘要进行压缩，存在损失，如果减少了关键信息，你如何解决

这个问题确实是短期记忆设计里的一个核心难点。我的思路不是“只做一次摘要然后一直用”，而是通过 **分层保留 + 结构化摘要 + 可回溯机制** 来降低信息损失。

第一，**我不会把原始上下文完全替换掉**。通常会采用“最近几轮原文 + 更早内容摘要”的混合方式，也就是说，最新且高价值的信息仍然保留原文，只有较早的历史才压缩成 summary，这样能减少关键细节被过早丢失。

第二，**摘要不会只写成一段自然语言**，我更倾向于做结构化摘要，比如单独保留“用户目标、关键约束、已完成步骤、待办事项、用户偏好、被纠正的信息”这些字段。因为自由文本摘要最容易漏掉约束和状态，而结构化字段更稳定。

第三，**我会给关键事实做显式锚定**。比如用户明确提出的限制条件、任务目标、纠错信息，不只放进摘要里，还会提升为高优先级状态单独保存，避免在多轮压缩中被覆盖。

第四，**保留可回溯能力**。如果模型在后续任务中发现信息不足，或者用户说“不是，我前面说过……”，系统可以回查原始对话或更长历史，而不是只能依赖压缩摘要。

所以我一般会总结成一句话：**解决摘要压缩信息损失，关键不是让摘要更长，而是通过“原文与摘要分层保留、结构化压缩、关键事实单独锚定、必要时支持回溯”来保证重要信息不丢。**

23 如何压缩上下文

上下文压缩的核心目标，不是单纯把内容变短，而是**在减少 token 的同时尽量保留对当前任务最有价值的信息**。我一般会从四个层面做。

第一是 **删除冗余**，比如重复对话、重复检索片段、无关寒暄、已经失效的工具返回结果，这部分先清掉。第二是 **摘要压缩**，把较长历史对话或长文档总结成更短的内容，但我不会只做自由摘要，而是尽量做成结构化摘要，比如保留“用户目标、关键约束、已完成步骤、待办事项”这些字段。第三是 **优先级保留**，像 system prompt、当前目标、用户明确约束、最新 observation 这类高价值信息要优先保留，不能和普通历史一视同仁。第四是 **按需检索替代全量注入**，也就是不把所有历史和知识都塞进窗口，而是需要时再从长期记忆、向量库或日志里检索回来。

如果是工程上，我通常会采用一种混合策略：**最近几轮保留原文，较早历史压成摘要，外部知识按需召回，关键事实单独锚定。** 这样既能控制 token，又能降低关键信息丢失的风险。

所以我一般会总结成一句话：**上下文压缩的本质，是通过去冗余、做结构化摘要、设置优先级和按需检索，让模型在更少 token 里看到最关键的信息。**

24 RAG系统完整链路如何设计(切分、召回、重排、生成、评估)?

我理解一个完整的 RAG 系统链路，通常要按 **切分**、**召回**、**重排**、**生成**、**评估** 五个环节来设计，而且每一层都要围绕“**提高相关性**、**降低幻觉**、**控制成本**”这三个目标。

第一步是 **切分**。我不会简单按固定长度硬切，而是优先采用 **文档结构切分 + 递归切分 + overlap** 的方式。比如先按标题、章节、段落切，如果某段太长再按句子或列表往下拆；太短则适当合并。这样做是为了让 chunk 既保持语义完整，又方便检索。

第二步是 **召回**。我通常会采用 **混合召回**，也就是 **BM25 关键词检索 + 向量检索**。因为关键词检索擅长精确命中，向量检索擅长语义召回，两者结合更稳。如果 query 较复杂，还可以加查询重写、多路召回或者 HyDE 来提升 recall。

第三步是 **重排**。召回阶段追求的是“别漏”，重排阶段追求的是“排得准”。我一般会把召回出来的 TopK 候选交给 **Cross-Encoder reranker** 或 LLM reranker 重新打分，最后取最相关的前几个 chunk 送给大模型。

第四步是 **生成**。生成时我会明确约束模型 **只能基于检索到的上下文回答**，如果证据不足就直接说不知道，并尽量附带引用来源，这样可以减少幻觉。

第五步是 **评估**。我会分层评估：检索层看 **Recall**、**MRR**、**NDCG**，生成层看 **Correctness**、**Faithfulness**、**Relevance**，系统层再看 **延迟**、**成本**、**稳定性**。因为 RAG 不是只看最终答案，而是要看整条链路是不是可靠。

所以我一般会总结成一句话：**一个好的 RAG 系统，本质上就是用合理切分保证知识粒度，用混合召回保证覆盖，用重排保证精度，用受约束生成降低幻觉，再通过分层评估持续优化整条链路。**

25 向量召回、关键词召回、实体召回如何融合与定权?

向量召回、关键词召回和实体召回的融合，本质上是在综合三种不同信号：**向量召回解决语义相似，关键词召回解决字面精确匹配，实体召回解决结构化对象和关键实体命中。** 我的思路一般不是只选一种，而是做 **多路召回 + 统一打分 + 动态定权**。

第一步是**并行召回**。同一个 query 同时走向量检索、BM25 关键词检索和实体检索，比如从知识图谱、实体索引或 metadata 里查命中的实体。这样做的目的是提高覆盖率，避免单一路径漏召回。

第二步是**结果归一化和去重**。因为三路召回的分数体系不一样，不能直接相加，所以我会先对各路 score 做归一化，比如 min-max 或 rank-based normalization，然后按文档或 chunk 级别合并重复结果。

第三步是**加权融合**。一个常见做法是线性加权，比如最终分数 = α × 向量分 + β × 关键词分 + γ × 实体分。权重怎么定，要看场景：如果是通用问答或语义检索，向量权重大一些；如果是产品型号、报错信息、专业术语很强的场景，关键词权重要提高；如果问题里实体非常关键，比如人名、地名、药名、设备名，实体召回权重就应该更高。

更进一步，我会做**动态定权**，也就是先判断 query 类型再分配权重。比如 query 里有明确实体和过滤条件，就提高实体和关键词权重；如果 query 比较自然语言、描述性强，就提高向量权重。最后再接一个 reranker 做精排。

所以我一般会总结成一句话：**三路召回融合的核心是“并行召回保覆盖，归一化后做加权融合，再根据 query 类型动态调权，最后用 reranker 统一精排”。**

26 Agent系统如何设计可观测性(日志、trace、回放)?

Agent 系统的可观测性，我一般会按 **日志**、**Trace**、**回放** 三层来设计，目标不是简单“记日志”，而是要做到 **问题能定位**、**过程能还原**、**结果能复盘**。在工程底座上，我会优先采用 OpenTelemetry 这类标准化观测方案，因为它本身就支持 **traces**、**metrics**、**logs** 三种信号，并且通过 **Trace ID / Span ID** 把日志和链路关联起来。

第一层是 **日志**。我会记录用户输入、路由结果、工具调用参数、模型输出、异常信息和关键状态变更，但日志一定要结构化，比如带上 `session_id` 、 `user_id` 、 `agent_name` 、 `tool_name` 、 `trace_id` ，这样后面才能检索和关联。第二层是 **Trace**。我会把一次用户请求作为一个 root trace，然后把意图识别、任务规划、检索、工具调用、模型生成这些步骤都做成 spans，因为 trace 本质上就是由多个 span 组成的一条完整请求链路。

第三层是 **回放**。也就是把一次运行时的关键输入输出、prompt、上下文、工具返回结果、状态快照保存下来，出现问题时可以按原始顺序重放，复现“当时为什么这样决策”。像 LangSmith 这类 Agent 观测平台，本质上就是把 run、trace、prompt、tool call 这些过程记录下来，方便调试和回看。

所以我一般会总结成一句话：**Agent 的可观测性不是只看最终答案，而是通过结构化日志看事件、通过 trace 看全链路、通过回放做问题复现和策略优化。**

27 RAG有哪些分类

RAG 一般可以从**检索方式、知识组织方式和系统能力复杂度**几个角度来分类。最基础的一类是 **Naive RAG**，也就是最经典的“文档切分、向量化、检索、拼接上下文、交给大模型生成”，它实现简单，适合作为 baseline。

第二类是按**检索方式**来分，比如 **向量 RAG、关键词 RAG 和混合 RAG**。向量 RAG 依赖 embedding 做语义检索，关键词 RAG 依赖 BM25 这类字面匹配，混合 RAG 则把两者结合，兼顾语义召回和精确匹配，这也是现在最常见的工程方案。

第三类是按**检索增强能力**来分，比如 **带查询重写的 RAG、带重排序的 RAG、多路召回 RAG、HyDE RAG、自查询 RAG**。这类系统不是只做一次简单检索，而是通过 query 优化、rerank 和 metadata filter 来提升召回质量。

第四类是按**知识组织形式**来分，比如 **传统文本 RAG** 和 **GraphRAG**。传统文本 RAG 主要基于 chunk 检索，GraphRAG 则进一步把实体和关系建成图，更适合多跳推理和全局总结。

第五类是按**系统复杂度**来分，可以分成 **单轮 RAG** 和 **Agentic RAG**。单轮 RAG 是一次检索一次生成，而 Agentic RAG 会加入规划、反思、多轮检索、工具调用等能力。

所以我一般会总结成一句话：**RAG 可以从“怎么检索、怎么组织知识、系统有多复杂”这几个维度分类，实际工程里最常见的是混合检索 + 重排的增强型 RAG，而更复杂的方向就是 GraphRAG 和 Agentic RAG。**

28 传统RAG有什么痛点;介绍GraphRAG，GraphRAG的难点是什么;GraphRAG如何应对增量场景

传统 RAG 的痛点主要有三个。第一，它更擅长回答局部事实问题，但对“整批文档的主题是什么、跨文档之间有什么关系”这类**全局总结和多跳关联问题**比较弱，因为它本质上还是在做 chunk 相似度召回。第二，它召回到的是离散文本片段，**片段之间的实体关系和结构信息不显式**，所以模型要自己在上下文里临时拼关系，容易漏信息。第三，当知识分散在多个文档里时，传统 RAG 往往会出现**召回碎片化、上下文冗余和答案不够全面**的问题。Microsoft GraphRAG 论文就是为了解决这类“global questions”提出的。

GraphRAG 可以理解成：在传统 RAG 上增加一层**知识图谱和社区摘要索引**。它会先从文本里抽取实体和关系，构建图，再做社区发现，并为每个社区生成 summary。查询时，针对具体实体问题走 **Local Search**，沿实体邻居和相关概念展开；针对全局问题走 **Global Search**，直接基于 community reports 做 map-reduce 式聚合总结。这样它比传统 RAG 更适合跨文档、多跳推理和全局归纳。

GraphRAG 的难点主要在索引和维护成本。因为它不是只做 embedding，还要做实体抽取、关系抽取、实体合并、社区发现和社区摘要生成，所以构建成本更高、链路更复杂；而且图是联动的，一个新文档进来，可能不只是多几个 chunk，而是会影响实体归并、社区边界和 summary 一致性。

增量场景下，GraphRAG 的应对思路通常是“增量更新 + 局部重算，而不是每次全量重建”。Microsoft 社区讨论里提到，新增文档后重新运行时，系统会处理新文档并利用缓存减少重复开销，近版本也加入了 incremental indexing 的入口；但严格来说，删除、社区漂移和全局 summary 一致性仍然比普通 RAG 更难，所以工程上通常会做分层策略：新增内容先局部抽实体和关系，做实体归并，受影响的子图和社区再局部重算；如果图结构变化太大，再触发阶段性全量重建。

我一般会总结成一句话：**传统 RAG 擅长局部检索，GraphRAG 擅长结构化关联和全局总结；它的价值来自“图”，但它的难点也正来自“图”的维护和增量更新。**

29 RAG优势与Split策略:讨论针对不同文档(如代码、手册)的切分逻辑差异。

RAG 的核心优势，我一般总结为三点：**第一，能引入外部知识，弥补大模型参数知识的不足和时效性问题；第二，能让回答尽量基于证据，降低幻觉；第三，知识更新成本低，因为更新的是知识库和索引，不是重新训练模型。** 所以在企业场景里，RAG 特别适合做知识库问答、文档助手、代码助手这类“依赖外部资料”的系统。

但 RAG 效果很大程度取决于 **split，也就是切分策略**。因为切得不好，后面检索再强也很难救回来。我的理解是：**切分的核心目标不是机械分段，而是让每个 chunk 在语义上尽量完整，同时大小又适合检索。**

针对不同文档，切分逻辑差异会很大。
比如 **普通产品手册、说明文档**，我会优先按**标题、章节、段落、列表**做结构化切分，再结合递归切分和 overlap。因为这类文档天然有层级结构，按结构切可以尽量保证一个 chunk 只讲一个主题。

而 **代码文档或源码**，就不能按普通段落切了，我会更偏向按**文件、类、函数、方法、注释块**来切。因为代码的最小语义单元通常不是一段文字，而是一个函数或一个类。如果把一个函数从中间切断，检索回来就很可能失去上下文，比如参数定义、返回值、异常处理全被拆散了。

所以我一般会总结成一句话：**RAG 的优势在于“让模型基于外部知识回答”，而 Split 策略决定了知识能不能被正确召回；手册类文档更适合按结构和主题切，代码类文档更适合按语义单元，比如类、函数和注释块来切。**

30 大文档检索:针对长文档，如何避免“中间信息丢失”问题及长上下文的处理技巧。

长文档检索里“中间信息丢失”一般有两种情况：**一种是切分时把关键内容切散了，另一种是虽然检索到了，但在长上下文拼接时被淹没了。** 所以解决这个问题，核心不能只靠扩大上下文窗口，而是要同时优化 **切分、召回和上下文组织**。

第一，在**切分阶段**，我不会只按固定长度切，而是更倾向于 **结构化切分 + overlap + 父子块策略**。比如先按章节、段落、标题切，再用 overlap 保证跨边界信息不丢；子块用于精准召回，父块用于补充完整上下文，这样能减少“中间一段被切断后语义残缺”的问题。

第二，在**检索阶段**，我会避免只取单个 chunk，而是采用 **邻近扩展** 或 **窗口扩展**，也就是某个 chunk 命中后，把它前后相邻的片段也一并带回来。因为长文档里，真正有用的信息往往不只在一个块里，而是在相邻几段之间。

第三，在**长上下文处理**上，我不会把大量原文直接堆给模型，而是做 **分层组织**：高相关片段优先保留原文，低相关但有背景价值的内容做摘要压缩；同时按“标题—结论—证据”这种结构拼接，让模型更容易定位重点。必要时我还会用 **map-reduce 式总结**，先分段理解，再全局汇总。

所以我一般会总结成一句话：**避免长文档中间信息丢失，关键是“切分时不断语义、召回时带上邻域、拼接时分层压缩”，而不是单纯依赖更大的上下文窗口。**

31 面对一万个PDF文档，考察从解析、清洗、向量化入库到检索增强的全链路设计。

面对一万个 PDF 文档，我会把整个链路设计成 **“解析清洗、结构切分、索引入库、混合检索、重排生成、评估运维”** 六个阶段，核心目标是 **可扩展、可更新、可追溯**。

第一步是 **解析和清洗**。我不会直接把 PDF 当纯文本处理，而是先区分文本型 PDF 和扫描型 PDF；文本型优先做版面解析，扫描型再走 OCR。清洗时会去掉页眉页脚、目录噪声、重复页码，同时尽量保留标题层级、表格、章节编号和元数据，比如文档名、来源、时间、版本。因为后面检索效果很大程度取决于前面的结构保留得好不好。

第二步是 **切分**。我一般会采用 **结构化切分 + 递归切分 + overlap**。先按标题、章节、段落切，如果某段太长再往下细分；太短则合并。这样既保证语义完整，又方便召回。对于手册或规范类文档，我还会保留父子块关系，子块用于召回，父块用于返回更完整上下文。

第三步是 **向量化和入库**。我会做两套索引：一套是 **向量索引**，支持语义检索；另一套是 **关键词或全文索引**，支持精确匹配，所以整体更偏向 **混合检索架构**。每个 chunk 入库时都要带 metadata，比如文档 ID、页码、标题路径、版本号，方便后续过滤和溯源。

第四步是 **检索增强**。查询进来后先做 query rewrite，再走 **BM25 + 向量召回**，合并去重后再做 rerank，最后把高质量 chunk 送给大模型生成答案，并要求带引用来源。

第五步是 **评估和运维**。要持续监控解析成功率、召回率、延迟、成本和答案忠实性，同时支持增量更新、去重、重建索引和版本管理。

所以我一般会总结成一句话：**面对一万个 PDF，关键不是把它们都塞进向量库，而是要把“解析质量、结构保留、混合检索、来源追踪和持续更新”做成一条完整可运营的 RAG 链路。**

32 多阶段召回策略优化

多阶段召回策略优化，本质上是在解决一个平衡问题：**第一阶段尽量别漏，后续阶段尽量更准。** 因为单一召回方式很难同时兼顾召回率、精度、延迟和成本，所以实际系统里我通常会设计成 **粗召回 → 融合召回 → 精排筛选** 的多阶段链路。

第一阶段是 **粗召回**，目标是扩大覆盖面，常见做法是并行走 **BM25 关键词召回、向量召回、实体召回**，必要时再加 query rewrite、HyDE 或 multi-query，尽量把可能相关的候选都找回来。这个阶段更关注 Recall，不追求最终排序绝对准确。

第二阶段是 **融合与过滤**，把多路召回结果做去重、归一化打分和 metadata 过滤，比如时间、文档类型、权限范围。这里我通常会根据 query 类型做动态调权：如果问题里有明显术语、编号、报错信息，就提高关键词权重；如果是自然语言描述型问题，就提高向量权重；如果实体特别关键，就提高实体召回权重。

第三阶段是 **重排和上下文优化**，也就是把候选交给 reranker 做精排，只保留最相关的 TopN，再做邻域扩展、父子块回填和上下文压缩，最终送给大模型。这样既保证不漏关键信息，又避免把大量噪声塞进上下文。

所以我一般会总结成一句话：**多阶段召回优化的核心，就是前面多路召回保覆盖，中间融合过滤提纯，后面重排压缩保精度，最终在召回率、答案质量和系统成本之间取得平衡。**

33 介绍一下Function Call的流程，模型是如何知道该调用哪个工具的?

Function Call 的流程，本质上是**“模型先判断需不需要调用外部工具，如果需要，就按工具定义生成结构化参数，再由系统执行工具并把结果返回给模型继续完成回答”**。也就是说，模型本身并不直接执行工具，它负责的是**选择工具和生成调用参数**。

一个完整流程通常分四步。第一步，系统会把可用工具的描述提供给模型，包括 **工具名、功能说明、参数 schema 和使用条件**。第二步，用户问题进来后，模型会结合问题语义和这些工具描述，判断是直接

回答，还是发起某个 function call。第三步，如果决定调用工具，模型会输出一个结构化调用结果，比如工具名和 JSON 参数。第四步，外部程序真正执行这个工具，把返回结果再交给模型，模型基于工具结果生成最终回答。

至于模型为什么知道该调用哪个工具，核心原因是：**工具在 prompt 或协议里已经以“可理解的能力描述”暴露给模型了**。模型会把用户意图和工具说明做匹配。比如用户问天气，而工具列表里有“查询天气”的函数，模型就会判断这个问题更适合调用工具而不是靠记忆硬答。换句话说，模型不是“真的理解系统代码”，而是通过 **工具描述、参数定义和上下文语义匹配** 来做选择。

所以我一般会总结成一句话：**Function Call 的本质是让模型先做“工具选择和参数生成”，再由外部系统执行工具，模型之所以知道该调用哪个工具，是因为开发者提前把工具能力以结构化描述提供给了模型。**

34：向量数据库索引中，IVF_FLAT和HNSW有什么区别?各自的适用场景?

IVF_FLAT 和 HNSW 的核心区别，在于索引结构完全不同。
IVF_FLAT 是“**先聚类分桶，再在少量桶里做精确扫描**”；它会先把向量分成很多簇，搜索时先找到最相关的几个簇，再只扫描这些簇里的原始向量，所以它的关键参数通常是 `nlist` 和 `nprobe`。Milvus 官方就是这样定义的：IVF_FLAT 先把向量划分为 `nlist` 个聚类单元，查询时只在 `nprobe` 个最相近簇中比较。

HNSW 则是“**基于分层图的近邻搜索**”；建索引时会把每个向量和若干近邻连成图，查询时从高层稀疏图快速定位，再逐层向下逼近最近邻，所以它通常查询延迟更低、召回率也很高，但代价是**内存开销更大**。Milvus 官方文档明确提到，HNSW 是 graph-based index，精度和低延迟表现很好，但需要较高的内存来维护分层图结构。

适用场景上，我一般这样选：
如果数据量很大、想兼顾性能和成本，而且可以接受通过参数调优来换速度，比如大规模 RAG、推荐检索，**IVF_FLAT 更合适**；它对大规模数据比较友好，而且存的是原始向量，精度通常也比较稳。
如果数据规模没有大到极端夸张，或者你更看重**高召回、低延迟**，并且机器内存比较充足，比如高质量语义搜索、交互式检索，**HNSW 往往更合适**。Faiss 的官方选型建议里也提到，如果内存足够或者数据集不算特别大，HNSW 往往是很快且很准的选择。

还有一个工程上的点要注意：**HNSW 的维护成本通常更高**。比如 Faiss 官方文档提到，HNSW 不支持直接删除向量，因为会破坏图结构；而 IVF 类索引在增删改管理上通常更好处理一些。

所以我会总结成一句话：**IVF_FLAT 是“先分桶再扫桶”，更适合大规模、性价比优先的场景；HNSW 是“图上逐步逼近”，更适合内存充足、追求高召回和低延迟的场景。**

35 LangGraph相比于LangChain有什么优势?LangGraph的状态快照机制了解吗，大概是怎么实现的?你的向量记忆库是如何更新用户画像的?如何区分短期记忆和长期记忆?

第一，LangGraph 相比 LangChain 的优势，我会概括成一句话：**LangChain 更适合快速搭应用，LangGraph 更适合做复杂、可控、可恢复的生产级 Agent。** 官方文档里也明确说，LangChain 的 agents 是构建在 LangGraph 之上的；LangGraph 提供更底层的图式编排、显式状态管理、持久化、流式输出、人机协同和调试能力，更适合长流程、多分支、多智能体和需要精细控制的场景。

第二，LangGraph 的状态快照机制，官方叫 **checkpoint / StateSnapshot**。它的思路不是只保存最终结果，而是在图执行的每个 **super-step** 把当前线程状态保存下来；一个 checkpoint 里会包含当前配置、metadata、状态值 `values` 、下一步要执行的节点 `next` ，以及任务信息 `tasks` 。这样系统就能支持中断恢复、time-travel、human-in-the-loop 和失败后续跑。从实现理解上，本质就是“**thread_id + checkpointer + 分步快照持久化**”：每次节点执行完，把共享 state 和执行位置写到持久层，比如内存、SQLite、Postgres 等，之后可以按 thread_id 取回并继续跑。

第三，你的向量记忆库如何更新用户画像，我的做法一般是“**抽取—去重—版本化—写入—检索融合**”。也就是先从对话里识别出值得长期保留的偏好、约束或事实，比如“以后回答都用中文”“用户更关注后端开发岗位”；然后做去重和冲突检测，再带上 `user_id` 、时间戳、来源、置信度写入向量库或结构化存储。更新时我不会简单覆盖，而是更倾向于**版本化或追加写入**，检索阶段再综合时间、相关性和置信度排序，这样能避免旧画像被噪声直接冲掉。LangGraph 官方也把长期记忆定义成跨 session 的 user-specific 或 application-level data，而不是线程内状态本身。

第四，如何区分短期记忆和长期记忆，我一般这样讲：**短期记忆是 thread-level / session-level 的当前上下文，长期记忆是 cross-session 的持久化事实和偏好。** 在 LangGraph 官方文档里，短期记忆是 agent state 的一部分，主要用于多轮对话连续性；长期记忆则是独立存储在外部 store 里的，可跨线程、跨会话复用。工程上我通常会把短期记忆放在 checkpoint 或会话状态里，保存最近消息、当前计划、工具结果；长期记忆放在向量库或数据库里，按需检索回注入上下文。

我一般会把这题最后总结成一句话：**LangChain 偏高层封装，LangGraph 偏底层可控；LangGraph 的 checkpoint 是按执行步保存状态快照；用户画像更新要做抽取、版本化和冲突管理；短期记忆管当前线程，长期记忆管跨会话沉淀。**

36 工具多导致token数过多的解决办法

工具太多导致 token 过多，本质上是因为 **模型每一轮都要看到大量 tool schema、描述和参数定义**，这会同时带来上下文膨胀、推理干扰和成本上升。我的处理思路一般不是“把所有工具都直接给模型”，而是做 **分层暴露、动态选择、工具压缩**。

第一种做法是 **工具分组和分域路由**。也就是先做一个 router，把问题先分到某个领域，比如检索类、数据库类、办公类，再只把该领域的少量工具暴露给模型，而不是一次性把几十个工具全放进去。第二种做法是 **两阶段工具选择**，先用轻量分类器、规则或小模型判断候选工具集合，再让主模型在候选集合里做最终 function call，这样能大幅减少主 prompt 里的 tool 数量。

第三种做法是 **压缩工具描述**。很多时候 token 过多不是因为工具数量本身，而是因为 description 太啰嗦、schema 太长，所以我会把工具说明写得更短、更判别化，只保留“什么时候用、输入输出是什么、不要在什么情况下用”。第四种做法是 **封装高频工具为 skill**，也就是把多个底层 tool 包成一个上层能力，让模型面向“能力模块”选择，而不是直接面对很多原子工具。

如果场景再复杂，我还会做 **按会话动态加载工具**，也就是当前任务需要什么才注册什么，不需要的工具不进上下文。

所以我一般会总结成一句话：**解决工具过多导致 token 膨胀，关键不是让模型硬选更多工具，而是通过分域路由、候选筛选、schema 压缩和能力封装，做到“按需暴露工具”。**

37 Mcp server的构建方式

MCP Server 的构建方式，我一般会理解成 **“先定义能力边界，再用官方 SDK 把这些能力按 MCP 标准暴露出去”**。MCP 官方把服务端能力主要分成三类：**Resources、Tools 和 Prompts**。其中 Resources 更像可读数据接口，适合暴露文件、文档、数据库结果；Tools 是可执行动作，适合搜索、计算、调用业务 API；Prompts 则是可复用的交互模板。官方 Python SDK 也是围绕这三类能力来构建服务端的。

具体搭建流程上，我通常会分成六步。第一步，**选 SDK 和语言**，目前官方提供多语言 SDK，Python 里最常见的是 FastMCP 方式，它会利用类型注解和 docstring 自动生成工具定义，开发效率比较高。第二步，**定义 Server 要暴露什么能力**，也就是明确哪些内容做成 Resources，哪些能力做成 Tools，哪些场景需要 Prompts。第三步，**实现业务逻辑**，比如封装数据库查询、文件读取、外部 API 调用，并做好参数校验、异常处理和权限控制。第四步，**选择传输方式**，MCP 官方和 SDK 支持像 stdio、SSE 这类标准传输。第五步，**联调用 MCP Inspector 做测试调试**，官方也专门提供了 Inspector 用来查看工具、资源和调用行为。第六步，**部署和发布**，如果要共享给更多客户端，还可以把服务元数据发布到 MCP Registry。

从工程实现角度，我会把一个 MCP Server 拆成四层：**协议层、能力层、业务层、基础设施层**。协议层负责遵守 MCP 规范和消息生命周期；能力层负责定义 Tools/Resources/Prompts；业务层是真正的数据库、文件系统、搜索或业务 API 逻辑；基础设施层则处理鉴权、日志、限流、超时和监控。这样做的好处是，MCP Server 不会变成一堆直接暴露给模型的杂乱函数，而是一个标准化、可治理、可复用的外部能力服务。这个思路也和官方对 MCP 的定位一致：它是一个把 AI 应用连接到外部系统的开放标准。

所以我一般会总结成一句话：**MCP Server 的构建，本质上就是用官方 SDK 按 MCP 协议把外部数据和能力封装成标准化的 Resources、Tools 和 Prompts，再通过标准传输方式提供给 LLM 或 Agent 调用。**